MAESTROS A UN CLIC

Documentación del proyecto

Plataforma web que conecta clientes con profesionales verificados de servicios del hogar en Medellín

Integrantes

- Juan Pablo Castaño
- Brayan Arenas

Institución Universitaria Pascual Bravo
Tecnología en Desarrollo de Software · Sexto semestre
Ingeniería de Software 2

Docente: Maira Perez

Medellín, Octubre de 2026

Documento del proyecto

> Versión revisada del entregable del proyecto de aula (13/04/2026). Se corrigió la numeración, se actualizaron los datos técnicos y legales, y el alcance refleja lo que el sistema hace hoy. Los requisitos, los diagramas, las pruebas, el despliegue y la seguridad tienen documentos propios, enlazados en cada sección.

1. Introducción

¿Alguna vez has necesitado instalar cajones nuevos en la cocina y no sabías quién podía hacerlo? Muchas personas enfrentan situaciones parecidas con tareas del hogar: reparaciones, renovaciones, instalaciones y diseño. El problema es que siempre hay que buscar en lugares distintos o preguntar a conocidos para encontrar a alguien que haga el trabajo. **Maestros a un clic** centraliza a esos profesionales, cerca de ti, para que solicites su servicio con un clic.

La solución es una plataforma web en la que:

- **Los profesionales** se registran, eligen sus especialidades, publican sus servicios con precio y duración estimada, muestran un portafolio de trabajos y definen su horario y sus tiempos de descanso. Para generar confianza, pasan por una verificación de identidad: suben documento, selfie y certificados, que revisa un administrador.
- **Los clientes** se registran, buscan profesionales por categoría, nombre, precio y cercanía, y reservan una hora libre real de la agenda del profesional. Para reservar también deben estar verificados.
- **Las dos partes** se comunican por un chat dentro de cada reserva, reciben notificaciones en tiempo real de cada cambio y, al terminar el servicio, se califican mutuamente de 1 a 5 estrellas. Una calificación baja genera una alerta para que el administrador revise el caso.

2. Planteamiento del problema

En Medellín, encontrar un profesional confiable para el hogar es difícil. Las alternativas actuales son:

- **Empresas que tercerizan profesionales**: tienen costos altos y el cliente no elige quién hace el trabajo.
- **El voz a voz**: depende de los contactos de cada persona y los resultados no siempre son los esperados.
- **Los grupos de redes sociales**: no hay ninguna validación de identidad ni de experiencia, y es común que haya estafas.

Además, coordinar horarios por mensajes es lento y propenso a cruces: el profesional acepta dos trabajos a la misma hora o no deja tiempo para desplazarse.

3. Justificación

La plataforma busca dar **credibilidad** al profesional y **tranquilidad** al cliente, para que este vuelva a usarla. Lo logra con cuatro mecanismos:

1. **Verificación de identidad** de clientes y profesionales, con documentos y certificados revisados por un administrador.
2. **Calificación recíproca y visible**: el cliente califica al profesional y el profesional al cliente, así otros profesionales pueden evaluar a un cliente antes de aceptar un trabajo.
3. **Agenda real**: el profesional define su horario, sus bloqueos y un tiempo de descanso entre servicios. El cliente solo ve y reserva horas libres, y el sistema impide que dos servicios se crucen.
4. **Trazabilidad**: cada cambio de estado de una reserva queda registrado, con quién lo hizo, cuándo y por qué.

El flujo principal es este: el profesional se registra y se verifica, y publica sus servicios. El cliente busca un profesional y envía una solicitud para una hora libre. El profesional la acepta o la rechaza; el cliente confirma. El profesional inicia y finaliza el servicio, y las dos partes se califican.

4. Objetivos

4.1 Objetivo general

Desarrollar una plataforma web de servicios profesionales y oficios que conecte a especialistas en mantenimiento, infraestructura y diseño del hogar con clientes de Medellín, y que facilite una contratación transparente, verificada y sin intermediarios.

4.2 Objetivos específicos

1. Construir un sistema de **perfiles profesionales** en el que el prestador muestre evidencia fotográfica de su trabajo, sus especialidades, sus servicios con precio y duración, y su ubicación.
2. Implementar un **motor de reservas** que calcule la disponibilidad real del profesional (horario, bloqueos, descanso y reservas existentes), evite cruces de horario y lleve cada reserva por un ciclo de vida auditable: solicitud, aceptación, confirmación, inicio y finalización.
3. Establecer un **ecosistema de confianza** basado en verificación de identidad, ubicación por comunas de Medellín y calificación recíproca.
4. Ofrecer **comunicación en tiempo real** entre las partes: chat por reserva, estado en línea y notificaciones.
5. Aplicar prácticas profesionales de ingeniería de software: arquitectura por capas y módulos, contrato de API documentado, pruebas automatizadas, integración continua y control de versiones con revisión por pull request.

5. Referentes del medio

Referente	Cómo funciona	Limitación frente a esta propuesta
Hombre en Casa Medellín	Página informativa; la solicitud se hace por WhatsApp y la empresa contacta a un profesional	Proceso manual. El cliente no elige al profesional ni ve su agenda
Multihelpers	Se escribe a la empresa, que asigna el profesional	El cliente no elige ni conoce el perfil del profesional
Hogaru	Servicio de aseo para hogares y empresas	Un solo nicho. La empresa asigna a la persona y no se ve su perfil

Diferencial de Maestros a un clic: el cliente elige al profesional viendo su perfil verificado, su portafolio, sus calificaciones y su agenda real. La reserva es directa y queda trazada de principio a fin.

6. Marco de referencia

- **Marco contextual.** La plataforma arranca en Medellín y usa la división administrativa en 16 comunas y 5 corregimientos para ubicar a clientes y profesionales y sugerir comunas vecinas.
- **Marco tecnológico.** Se enmarca en la economía colaborativa (*gig economy*). Usa una API REST, comunicación en tiempo real por WebSockets, una base de datos relacional y geolocalización.
- **Marco conceptual.** Es un *marketplace de servicios*: intermediación digital, validación de identidad (*KYC, Know Your Customer*) para reducir fraudes y reputación basada en calificación recíproca.

7. Metodología

7.1 Sustento teórico

El proyecto se fundamenta en **Scrum** (Schwaber y Sutherland, 2020): entregas iterativas e incrementales, colaboración del equipo y adaptación al cambio. Las historias de usuario, con criterios de aceptación verificables, forman el *product backlog*.

7.2 Cómo se aplicó

Práctica	Implementación en el proyecto
Backlog	Cada historia de usuario es un issue de GitHub con sus criterios de aceptación como lista verificable, su módulo y sus dependencias
Reparto del trabajo	Por módulos completos (backend y frontend de cada uno) entre dos desarrolladores *full stack*; cada uno es dueño de sus módulos
Fase 0 (base común)	Antes de repartir: esquema completo de la base de datos, contrato de la API, sistema de diseño, convenciones, CI y un módulo de referencia (favoritos)
Iteraciones	Una rama por historia, pull request revisado y fusión a `main` solo con la integración continua en verde
Definición de terminado	Cumple los criterios de aceptación, tiene pruebas automatizadas (caso feliz, validación y permisos), está documentado en el contrato de la API y fusionado en los dos repositorios
Calidad automática	En cada PR, la integración continua corre las pruebas (PHP 8.2 y 8.5), el formato (Pint, Prettier), el lint (ESLint, oxlint), los tipos (TypeScript), el build, el verificador de colores del sistema de diseño y la auditoría de dependencias. El check **Guardas** impide aflojar reglas sin aprobación

Práctica	Implementación en el proyecto
Trabajo con IA	Los dos desarrolladores usan asistentes de IA (Claude Code y Antigravity) que leen las mismas reglas del repositorio (`AGENTS.md`), para mantener un estilo único

Las etapas clásicas siguen presentes (planificación, diseño, desarrollo, pruebas, despliegue y mantenimiento), pero en ciclos cortos por historia.

8. Estudio de factibilidad

8.1 Viabilidad técnica

Todas las tecnologías son de código abierto, maduras y con amplia documentación.

Capa	Tecnología	Uso
Frontend	Vue 3, TypeScript, Vite, Tailwind CSS 4, Pinia y Pinia Colada	Aplicación web de una sola página (SPA), responsive
Backend	Laravel 12 (PHP 8.2+), Laravel Sanctum	API REST versionada (`/api/v1`) y autenticación por token
Tiempo real	Laravel Reverb (WebSockets) y Laravel Echo	Chat, estado en línea, notificaciones y agenda en vivo
Base de datos	MySQL 8	Base relacional con integridad referencial
Calidad	Pest, Vitest, ESLint, oxlint, Pint, Prettier, GitHub Actions	Pruebas automatizadas, estilo y CI
Despliegue previsto	AWS EC2 con Nginx, Supervisor y certificado TLS	Servidor único para API, cola, Reverb y archivos

La arquitectura detallada está en 08-decisiones-de-arquitectura.md y en los documentos `docs/arquitectura.md` de cada repositorio.

8.2 Viabilidad económica

Costos de infraestructura (estimados, primer año):

Concepto	Costo
Vue, Laravel, Reverb, MySQL Community Edition (licencias de código abierto: MIT y GPL)	$0
Servidor AWS EC2 (instancia pequeña) y disco	≈ 25 USD/mes
Dominio y certificado TLS (Let's Encrypt)	≈ 15 USD/año y $0
Correo transaccional (recuperación de contraseña)	Plan gratuito de un proveedor SMTP en la fase inicial

Costos de talento humano (rangos de referencia en Colombia, para una operación real):

Rol	Funciones principales	Dedicación	Costo aproximado
Líder de proyecto	Coordinar el equipo, cronograma y objetivos	Completa durante el desarrollo	$4–6 millones COP/mes
Desarrollador frontend (Vue)	Interfaz, componentes, integración con la API	Completa	$3–5 millones COP/mes
Desarrollador backend (Laravel)	Lógica de negocio, API, seguridad	Completa	$3–5 millones COP/mes
Administrador de base de datos	Esquema, optimización, respaldos	Parcial	$2–3 millones COP/mes
DevOps / nube	Servidores, despliegue, CI/CD, monitoreo	Parcial	$3–4 millones COP/mes
Diseño UI/UX	Prototipos, pruebas de usabilidad	Parcial	$2–3 millones COP/mes
QA	Pruebas funcionales, de seguridad y rendimiento	Parcial	$2–3 millones COP/mes
Seguridad / KYC	Verificación de identidad, protección de datos	Parcial	$3–4 millones COP/mes
Soporte	Atención a usuarios y profesionales	Completa tras el lanzamiento	$2–3 millones COP/mes

Modelo de ingresos posible: comisión por servicio completado o suscripción de los profesionales para destacar su perfil. El cliente ahorra frente a las empresas que tercerizan, porque contrata directamente.

8.3 Viabilidad de mercado

Hay una demanda clara por la dificultad de encontrar profesionales confiables, y los referentes actuales no dejan elegir al profesional ni ofrecen procesos automatizados. Las principales dificultades previstas son:

1. **Adopción inicial**: los profesionales pueden resistirse a la verificación, y los clientes pueden preferir los métodos tradicionales.
2. **Competencia** con plataformas ya conocidas.
3. **Percepción de la verificación** como invasiva o compleja.
4. **Arranque en frío de la reputación**: al inicio no habrá reseñas, y se necesitarán incentivos para las primeras calificaciones.
5. **Cumplimiento legal** en el tratamiento de datos personales.
6. **Costos de mercadeo** para posicionar la marca.
7. **Escalabilidad de la oferta**: si la demanda crece rápido, puede faltar disponibilidad de profesionales.

8.4 Viabilidad operativa

La plataforma es **autogestionable**. Clientes y profesionales administran sus perfiles, documentos, agendas y reservas. El administrador solo interviene en los casos críticos: aprobar la verificación de documentos, atender alertas por baja calificación y revisar reportes. Así se puede atender a muchos más usuarios sin aumentar el personal en la misma proporción. Las notificaciones, la agenda y los cambios de estado son automáticos.

8.5 Viabilidad legal y normativa

Norma	Aplicación en la plataforma
Ley 1581 de 2012 (protección de datos personales)	Tratamiento de nombres, ubicación, selfies, documentos y certificados con finalidad explícita, seguridad y derechos del titular. Ver 07-seguridad-y-privacidad.md
Decreto 1377 de 2013 (hoy compilado en el Decreto 1074 de 2015)	Autorización previa e informada para el tratamiento, y procedimientos de consulta, rectificación y supresión
Ley 1266 de 2008 (hábeas data financiero)	No aplica hoy: no se maneja información crediticia. Se tendría en cuenta si se integraran pagos o validaciones financieras
Ley 527 de 1999 (comercio electrónico)	Validez de los mensajes de datos: las solicitudes, aceptaciones y confirmaciones digitales quedan registradas con fecha y autor
Ley 1273 de 2009 (delitos informáticos)	Protección de la información y de los datos: control de acceso, almacenamiento privado de documentos y registro de cambios
CONPES 3854 de 2016 y 3995 de 2020 (política de seguridad y confianza digital)	Marco de buenas prácticas de seguridad digital que orienta los controles del sistema
ISO/IEC 27001 y 27018	Referentes de gestión de seguridad de la información y protección de datos en la nube para la operación
ISO/IEC 25010	Modelo de calidad de producto usado para formular los requisitos no funcionales (02-requisitos.md)
RGPD (Unión Europea)	Referente voluntario de buenas prácticas para una posible expansión

> **Corrección respecto al entregable original.** La Ley 1621 de 2013 regula las actividades de inteligencia y contrainteligencia del Estado, no la seguridad digital, así que se reemplazó por la Ley 1273 de 2009 y los documentos CONPES. MySQL no tiene licencias de Microsoft: la edición Community es de Oracle y se distribuye bajo GPL.

9. Alcance y limitaciones

9.1 Alcance

Incluido	Estado
Registro e inicio de sesión de clientes y profesionales, cierre de sesión y validación de datos	Implementado
Perfil profesional: descripción, especialidades, tarifa, ubicación, servicios con precio y duración, y portafolio	Implementado
Agenda: horario semanal, descanso configurable, bloqueos y cálculo de horas libres en tiempo real	Implementado
Ciclo de vida de la reserva: solicitar, aceptar o rechazar, confirmar, reprogramar, cancelar, iniciar y finalizar, con historial auditable	Implementado

Incluido	Estado
Chat por reserva con imágenes, estado en línea y eliminación de mensajes	Implementado
Notificaciones en tiempo real y centro de notificaciones	Implementado
Historiales, totales, servicios en proceso e ingresos del profesional	Implementado
Favoritos y búsqueda por nombre, categoría y precio	Implementado
Verificación de identidad (cargue de documentos y revisión del administrador), bloqueo de usuarios	Implementado
Calificación recíproca, comentarios, reportes y alertas por baja calificación	Implementado
Búsqueda por cercanía con distancia, comunas aledañas, "disponible ahora" y mapa	Implementado
Gestión de la cuenta: editar perfil, foto, contraseña, recuperación, eliminación con anonimización y cierre por inactividad	Implementado

Las 49 historias de usuario están terminadas.

El detalle por historia, con sus endpoints, pantallas y pruebas, está en 04-trazabilidad.md.

9.2 Fuera del alcance

- **Pagos en línea.** El precio se acuerda en la plataforma, pero el pago se hace por fuera.
- **Consulta automática de antecedentes con un proveedor externo.** No existe una API pública de la Policía o la Procuraduría que se pueda integrar. El administrador registra el resultado manualmente, y el modelo de datos queda listo para un proveedor futuro (decisión D10).
- **Aplicaciones móviles nativas.** La aplicación web es responsive y funciona en el navegador del celular.

9.3 Limitaciones

- La plataforma facilita la conexión y la verificación, pero no garantiza la calidad del trabajo del profesional.
- La verificación depende de la revisión humana de un administrador.
- En la fase inicial, la cobertura geográfica se limita a Medellín.
- Los costos de infraestructura en la nube pueden variar con la demanda.

10. Requisitos

Los 49 requisitos funcionales (historias de usuario con sus criterios de aceptación y su estado) y los requisitos no funcionales, medibles y con su forma de verificación, están en **02-requisitos.md**.

11. Análisis y diseño

- **Diagramas** (casos de uso, clases del dominio, entidad-relación, estados de la reserva, secuencias, componentes y despliegue), generados a partir del sistema actual: **03-diagramas.md**. Reemplazan a los diagramas PNG del entregable original, que eran previos al modelo definitivo.
- **Decisiones de arquitectura** y su justificación: **08-decisiones-de-arquitectura.md**.
- **Modelo de datos** completo: docs/modelo-de-datos.md.
- **Contrato de la API**: docs/api/endpoints.md, convenciones y la especificación OpenAPI exportada en docs/api/openapi.json.

12. Pruebas

La estrategia, las herramientas y el reporte de resultados están en **05-pruebas.md**.

13. Despliegue y operación

La guía para poner el sistema en producción y operarlo está en **06-despliegue-y-operacion.md**.

14. Seguridad y privacidad

Los controles implementados y el cumplimiento de la Ley 1581 de 2012 están en **07-seguridad-y-privacidad.md**.

15. Conclusiones

- El proyecto es **viable** técnica, económica, operativa y legalmente: usa tecnologías de código abierto maduras, un costo de infraestructura bajo y un modelo autogestionable.
- La propuesta se **diferencia** de los referentes porque el cliente elige al profesional viendo su perfil verificado, su reputación y su agenda real, y la reserva queda trazada de principio a fin.
- Construir primero una **base común** (esquema completo, contrato de API, sistema de diseño, convenciones y CI) permitió que dos personas trabajaran en paralelo por módulos sin conflictos y con un estilo uniforme.
- Las **pruebas automatizadas y la integración continua** en cada pull request detectan regresiones antes de llegar a `main`. Junto con los documentos de requisitos y trazabilidad, dejan evidencia verificable de que cada historia cumple sus criterios.
- La implementación inicial en Medellín permite validar el modelo antes de escalarlo a otras ciudades.

16. Glosario

Término	Significado
Cliente	Persona que busca y reserva servicios
Profesional	Persona que ofrece servicios; también "maestro"
Servicio	Lo que ofrece un profesional, con precio y duración estimada
Reserva	Solicitud de un cliente para un servicio en una fecha y hora; pasa por varios estados
Disponibilidad / horas libres	Horas en las que se puede reservar: el horario del profesional menos sus bloqueos y reservas, con su descanso
Descanso (*buffer*)	Tiempo que se bloquea antes y después de cada reserva para desplazarse o descansar
Verificación (KYC)	Revisión de la identidad del usuario a partir de sus documentos
API REST	Interfaz por la que el frontend se comunica con el backend mediante HTTP y JSON
WebSocket	Conexión permanente que permite enviar eventos en tiempo real
CI	Integración continua: verificaciones automáticas en cada cambio
PR	*Pull request*: propuesta de cambio que se revisa antes de unirla a la rama principal

17. Bibliografía

- Congreso de Colombia. (1999). *Ley 527 de 1999*, por medio de la cual se define y reglamenta el acceso y uso de los mensajes de datos, del comercio electrónico y de las firmas digitales.
- Congreso de Colombia. (2008). *Ley 1266 de 2008*, disposiciones generales del hábeas data.
- Congreso de Colombia. (2009). *Ley 1273 de 2009*, de la protección de la información y de los datos.
- Congreso de Colombia. (2012). *Ley Estatutaria 1581 de 2012*, disposiciones generales para la protección de datos personales.
- Presidencia de la República de Colombia. (2013). *Decreto 1377 de 2013*, reglamentario de la Ley 1581 de 2012.
- Departamento Nacional de Planeación. (2016). *Documento CONPES 3854: Política Nacional de Seguridad Digital*.
- Departamento Nacional de Planeación. (2020). *Documento CONPES 3995: Política Nacional de Confianza y Seguridad Digital*.
- International Organization for Standardization. (2022). *ISO/IEC 27001: Information security management systems*.
- International Organization for Standardization. (2023). *ISO/IEC 25010: Product quality model*.
- OWASP Foundation. (2021). *OWASP Top 10* y *Application Security Verification Standard (ASVS) 4.0*.
- Parlamento Europeo y Consejo. (2016). *Reglamento (UE) 2016/679 (RGPD)*.
- Schwaber, K. y Sutherland, J. (2020). *The Scrum Guide*.
- Sommerville, I. (2016). *Software Engineering* (10.ª ed.). Pearson.
- Documentación oficial de Laravel 12, Vue 3 y Tailwind CSS 4 (laravel.com, vuejs.org, tailwindcss.com).

Requisitos

Requisitos funcionales (historias de usuario) y no funcionales del sistema. La fuente de verdad del estado de cada historia es su issue en GitHub; este documento muestra el estado al momento de generarlo.

1. Actores

Actor	Descripción
Visitante	Persona sin sesión: explora la búsqueda y los perfiles públicos
Cliente	Busca profesionales, reserva servicios, chatea y califica. Debe estar verificado para reservar
Profesional	Publica servicios, gestiona su agenda, responde solicitudes, presta el servicio y califica al cliente
Administrador	Verifica documentos, atiende reportes y alertas, y bloquea usuarios
Sistema	Calcula disponibilidad, emite notificaciones y eventos en tiempo real, y genera alertas automáticas

2. Requisitos funcionales

Las 49 historias de usuario del proyecto (no existe la HU015: la numeración del documento original la omite). Estado a la fecha de este documento: **las 49 están hechas**. Cada una es un issue en GitHub, donde se marcan sus criterios.

2.1 Resumen

HU	Nombre	Módulo	Responsable	Estado
HU001	Inicio de sesión	auth	LordCasta	✅ Hecha
HU002	Búsqueda por geolocalización	búsqueda	barenas12	✅ Hecha
HU003	Gestión de disponibilidad y agenda	agenda	LordCasta	✅ Hecha
HU004	Solicitud y aceptación de servicio	reservas	LordCasta	✅ Hecha
HU005	Sistema de calificación recíproca	calificaciones	barenas12	✅ Hecha
HU006	Registro de cliente	auth	LordCasta	✅ Hecha
HU007	Registro de profesional	auth	LordCasta	✅ Hecha
HU008	Cargue de documentos	verificación-admin	barenas12	✅ Hecha
HU009	Edición de perfil profesional	perfil-profesional	LordCasta	✅ Hecha
HU010	Filtros y ordenamiento de profesionales	búsqueda	barenas12	✅ Hecha
HU011	Chat y confirmación de detalles del servicio	chat-notificaciones	LordCasta	✅ Hecha
HU012	Cancelación y reprogramación de reserva	reservas	LordCasta	✅ Hecha
HU013	Validación de identidad y antecedentes	verificación-admin	barenas12	✅ Hecha
HU014	Revisión y aprobación de documentos del profesional	verificación-admin	barenas12	✅ Hecha
HU016	Historial de servicios del cliente	reservas	LordCasta	✅ Hecha
HU017	Historial de servicios del profesional	reservas	LordCasta	✅ Hecha
HU018	Notificaciones del sistema	chat-notificaciones	LordCasta	✅ Hecha
HU019	Cambio de contraseña	cuenta	barenas12	✅ Hecha
HU020	Recuperación de contraseña	cuenta	barenas12	✅ Hecha
HU021	Cerrar sesión	auth	LordCasta	✅ Hecha
HU022	Registro de ubicación manual	cuenta	barenas12	✅ Hecha
HU023	Eliminación de cuenta	cuenta	barenas12	✅ Hecha
HU024	Reporte de usuarios	calificaciones	barenas12	✅ Hecha

HU	Nombre	Módulo	Responsable	Estado
HU025	Visualización de disponibilidad del profesional	agenda	LordCasta	✅ Hecha
HU026	Estado del servicio	reservas	LordCasta	✅ Hecha
HU027	Confirmación de inicio de servicio	reservas	LordCasta	✅ Hecha
HU028	Confirmación de finalización de servicio	reservas	LordCasta	✅ Hecha
HU029	Visualización de comentarios	calificaciones	barenas12	✅ Hecha
HU030	Filtro por disponibilidad inmediata	búsqueda	barenas12	✅ Hecha
HU031	Edición de perfil del cliente	cuenta	barenas12	✅ Hecha
HU032	Subida de foto de perfil	cuenta	barenas12	✅ Hecha
HU033	Visualización de servicios por categoría	búsqueda	barenas12	✅ Hecha
HU034	Guardar profesionales favoritos	favoritos	LordCasta	✅ Hecha
HU035	Visualización de favoritos	favoritos	LordCasta	✅ Hecha
HU036	Confirmación de reserva por el cliente	reservas	LordCasta	✅ Hecha
HU037	Visualización de ingresos del profesional	perfil-profesional	LordCasta	✅ Hecha
HU038	Bloqueo de usuarios problemáticos	verificación-admin	barenas12	✅ Hecha
HU039	Búsqueda por nombre	búsqueda	barenas12	✅ Hecha
HU040	Visualización de distancia	búsqueda	barenas12	✅ Hecha
HU041	Indicador de estado en línea	chat-notificaciones	LordCasta	✅ Hecha
HU042	Adjuntar archivos en el chat	chat-notificaciones	LordCasta	✅ Hecha
HU043	Eliminación de mensajes	chat-notificaciones	LordCasta	✅ Hecha
HU044	Visualización de perfil del cliente	calificaciones	barenas12	✅ Hecha
HU045	Alerta por baja calificación	calificaciones	barenas12	✅ Hecha
HU046	Ordenamiento por precio	búsqueda	LordCasta	✅ Hecha
HU047	Visualización de tiempo estimado	perfil-profesional	LordCasta	✅ Hecha
HU048	Validación de datos obligatorios	auth	LordCasta	✅ Hecha
HU049	Visualización de servicios en proceso	reservas	LordCasta	✅ Hecha
HU050	Cierre de sesión por inactividad	cuenta	barenas12	✅ Hecha

2.2 Detalle

HU001 · Inicio de sesión

Yo como cliente, quiero iniciar sesión en el sistema, para validar los datos de mi cuenta

Criterios de aceptación:

1. El sistema valida las credenciales
2. El sistema muestra error si son incorrectas
3. Redirige al panel principal

Estado: ✅ Hecha

HU002 · Búsqueda por geolocalización

Yo como cliente, quiero buscar profesionales en mi comuna mediante un mapa o listado, para contratar un servicio rápido y eficiente cerca de mi ubicación.

Criterios de aceptación:

1. El sistema solicita permisos de ubicación al usuario.
2. El buscador filtra profesionales activos dentro del radio de acción configurado.
3. Se despliega una lista ordenada por proximidad (km).

4. Si no hay profesionales en la comuna, el sistema sugiere los más cercanos en comunas aledañas.

Estado: ✅ Hecha

HU003 · Gestión de disponibilidad y agenda

Yo como profesional, quiero configurar mis días y horas de trabajo en un calendario, para evitar que los clientes reserven en momentos donde no tengo disponibilidad.

Criterios de aceptación:

1. El profesional puede marcar bloques de horas como "ocupado" o "disponible".
2. El sistema bloquea automáticamente un tiempo adicional (configurable) después de cada servicio para descanso.
3. Los cambios en la agenda se reflejan en tiempo real para los clientes.
4. El calendario impide que se solapen dos servicios distintos.

Estado: ✅ Hecha

HU004 · Solicitud y aceptación de servicio

Yo como cliente, quiero solicitar una reserva seleccionando una hora disponible del profesional, para concretar la contratación sin necesidad de intermediarios.

Criterios de aceptación:

1. El cliente visualiza la disponibilidad real del profesional.
2. Al enviar la solicitud, el profesional recibe una alerta en su panel.
3. El profesional tiene la opción de "Aceptar" o "Rechazar" la solicitud.
4. El estado de la reserva cambia a "Confirmado" solo si el profesional acepta.

Estado: ✅ Hecha

HU005 · Sistema de calificación recíproca

Yo como usuario (cliente o profesional), quiero calificar mi experiencia con la otra parte tras finalizar el servicio, para mantener la reputación y calidad dentro de la plataforma.

Criterios de aceptación:

1. La opción de calificar solo se habilita una vez que el servicio ha sido marcado como "Completado".
2. El sistema permite una calificación de 1 a 5 estrellas.
3. Se permite dejar un comentario opcional sobre el servicio o la atención.
4. El promedio de estrellas se actualiza automáticamente en el perfil del usuario calificado.

Estado: ✅ Hecha

HU006 · Registro de cliente

Yo como cliente, quiero registrarme en la plataforma con mi correo y contraseña elegida., para acceder al sistema.

Criterios de aceptación:

1. El correo debe ser válido
2. Se debe validar la existencia del correo
3. La contraseña debe ser mayor a 5 caracteres
4. Me debe permitir iniciar sesión inmediatamente después de crear el usuario

Estado: ✅ Hecha

HU007 · Registro de profesional

Yo como profesional., quiero registrarme en la plataforma con mi correo y contraseña elegida., para acceder al sistema y poder cargar mis documentos.

Criterios de aceptación:

1. El correo debe ser válido
2. Se debe validar la existencia del correo
3. La contraseña debe ser mayor a 5 caracteres

4. Me debe permitir iniciar sesión inmediatamente después de crear el usuario

Estado: ✅ Hecha

HU008 · Cargue de documentos

Yo como profesional., quiero después de registrarme, subir mis documentos como certificaciones, fotos, diplomas requeridos., para poder que validen mis documentos y ser profesional verificado.

Criterios de aceptación:

1. El sistema permite cargar múltiples archivos (imágenes y PDF) por categoría (certificados, documento de identidad, evidencias de trabajos).
2. El sistema valida formato y tamaño máximo de archivo antes de subirlo.
3. El sistema solicita como mínimo: documento de identidad, selfie y al menos un certificado o evidencia de trabajo.
4. Al finalizar el cargue, el estado del perfil queda en "Pendiente de verificación" y no puede ofrecer servicios hasta ser aprobado.
5. El profesional puede ver el estado de revisión (Pendiente, Aprobado, Rechazado) y el motivo si es rechazado.
6. El profesional puede reemplazar documentos rechazados y reenviar a verificación.

Estado: ✅ Hecha

HU009 · Edición de perfil profesional

Yo como profesional, quiero editar mi perfil (descripción, especialidades, tarifa por hora, ubicación, agenda y portafolio), para mantener mi información actualizada y atraer más clientes.

Criterios de aceptación:

1. El sistema permite actualizar campos básicos del perfil sin perder información previamente aprobada.
2. El sistema valida que la tarifa por hora sea un valor numérico mayor a 0.
3. El sistema permite agregar o quitar especialidades dentro del catálogo de servicios.
4. Los cambios de ubicación actualizan la zona de cobertura para búsquedas por geolocalización.
5. Las fotos del portafolio se muestran en una galería dentro del perfil.

Estado: ✅ Hecha

HU010 · Filtros y ordenamiento de profesionales

Yo como cliente, quiero filtrar y ordenar profesionales por servicio, precio, calificación y distancia, para encontrar la mejor opción según mis necesidades.

Criterios de aceptación:

1. El cliente puede filtrar por categoría de servicio y subcategoría.
2. El cliente puede definir un rango de precio por hora.
3. El cliente puede ordenar por calificación (descendente) y por distancia (ascendente).
4. Los resultados muestran información clave: nombre, calificación promedio, precio, distancia y disponibilidad próxima.
5. Al aplicar filtros, el listado y el mapa se actualizan sin recargar toda la página.

Estado: ✅ Hecha

HU011 · Chat y confirmación de detalles del servicio

Yo como cliente, quiero comunicarme con el profesional dentro de la plataforma antes del servicio, para aclarar detalles del trabajo, materiales y condiciones.

Criterios de aceptación:

1. El sistema habilita el chat solo cuando exista una solicitud enviada o una reserva confirmada.
2. El chat guarda el historial de mensajes por solicitud o reserva.
3. El cliente puede enviar fotos o descripciones del problema (con validación de tamaño y formato).
4. El profesional puede confirmar por el chat detalles como materiales requeridos y tiempo estimado.
5. El sistema envía notificaciones cuando llega un mensaje nuevo.

Estado: ✅ Hecha

HU012 · Cancelación y reprogramación de reserva

Yo como cliente o profesional, quiero cancelar o reprogramar una reserva, para gestionar cambios de disponibilidad o imprevistos.

Criterios de aceptación:

1. El sistema permite cancelar antes de la hora programada con un motivo obligatorio.
2. Al cancelar, el sistema libera automáticamente el bloque de agenda correspondiente.
3. Para reprogramar, el usuario selecciona un nuevo horario disponible del profesional.
4. El estado de la reserva cambia a "Cancelado" o "Reprogramado" y queda registro del motivo y la fecha anterior.
5. Ambas partes reciben notificación de la cancelación o reprogramación.

Estado: ✅ Hecha

HU013 · Validación de identidad y antecedentes

Yo como usuario (cliente o profesional), quiero completar un proceso de verificación de identidad y antecedentes, para aumentar la seguridad y confianza en la plataforma.

Criterios de aceptación:

1. El sistema solicita documento de identidad y selfie para validación.
2. El sistema valida coincidencia entre selfie y foto del documento mediante el proveedor de verificación definido.
3. El sistema registra el resultado de la verificación (Aprobado/Rechazado/Pendiente) y la fecha.
4. Si la verificación es rechazada, el usuario recibe el motivo y puede reenviar la documentación.
5. Mientras el usuario esté "Pendiente" o "Rechazado", el sistema limita acciones críticas (publicar servicios, confirmar reservas) según el rol.

Estado: ✅ Hecha

HU014 · Revisión y aprobación de documentos del profesional

Yo como gestor de la aplicación (administrador/validador), quiero revisar las selfies y documentos cargados por el profesional, para aprobar o rechazar la verificación y asegurar la autenticidad de la información.

Criterios de aceptación:

1. El sistema muestra una bandeja de solicitudes "Pendiente de verificación" con filtros por fecha, tipo de documento y estado.
2. Al abrir una solicitud, el gestor visualiza: selfie, documento de identidad, certificados y evidencias; con opción de zoom/descarga.
3. El gestor puede marcar cada documento como "Aprobado" o "Rechazado" y agregar observaciones obligatorias cuando rechaza.
4. El gestor puede aprobar la solicitud solo si los documentos mínimos requeridos están completos y aprobados.
5. Al aprobar, el estado del profesional cambia a "Verificado" y se habilita la publicación/aceptación de servicios.
6. Al rechazar, el sistema notifica al profesional el motivo y mantiene el estado en "Rechazado" hasta que reenvíe la información.
7. El sistema registra auditoría de la revisión: quién revisó, fecha/hora y decisión tomada.

Estado: ✅ Hecha

HU016 · Historial de servicios del cliente

Yo como cliente, quiero ver mi historial de servicios solicitados, para llevar control de lo que he contratado.

Criterios de aceptación:

1. El sistema muestra lista de servicios realizados y pendientes.
2. Se visualiza fecha, profesional y estado.
3. Se puede acceder al detalle de cada servicio.
4. Los servicios cancelados también aparecen en el historial.

Estado: ✅ Hecha

HU017 · Historial de servicios del profesional

Yo como profesional, quiero ver mi historial de trabajos realizados, para llevar control de mis servicios.

Criterios de aceptación:

1. Se muestran todos los servicios realizados y cancelados.
2. Se visualiza fecha, cliente y estado.
3. Se puede ver el detalle de cada servicio.
4. Se muestra el total de servicios realizados.

Estado: ✅ Hecha

HU018 · Notificaciones del sistema

Yo como usuario, quiero recibir notificaciones, para estar informado sobre solicitudes y cambios.

Criterios de aceptación:

1. El sistema envía notificación cuando hay una nueva solicitud.
2. Notifica cuando un servicio es aceptado o rechazado.
3. Notifica mensajes nuevos en el chat.
4. Las notificaciones se pueden visualizar dentro del sistema.

Estado: ✅ Hecha

HU019 · Cambio de contraseña

Yo como usuario, quiero cambiar mi contraseña, para mantener mi cuenta segura.

Criterios de aceptación:

1. Debo ingresar mi contraseña actual.
2. La nueva contraseña debe cumplir requisitos mínimos.
3. El sistema confirma el cambio exitoso.
4. Si la contraseña actual es incorrecta, muestra error.

Estado: ✅ Hecha

HU020 · Recuperación de contraseña

Yo como usuario, quiero recuperar mi contraseña, para poder acceder si la olvido.

Criterios de aceptación:

1. El usuario ingresa su correo.
2. El sistema envía un enlace de recuperación.
3. El enlace permite crear una nueva contraseña.
4. El enlace tiene un tiempo de expiración.

Estado: ✅ Hecha

HU021 · Cerrar sesión

Yo como usuario, quiero cerrar sesión, para proteger mi cuenta cuando termine de usarla.

Criterios de aceptación:

1. Existe un botón visible para cerrar sesión.
2. Al cerrar sesión se redirige al inicio.
3. Se elimina la sesión activa.

Estado: ✅ Hecha

HU022 · Registro de ubicación manual

Yo como usuario, quiero ingresar mi ubicación manualmente, para usar la plataforma si no quiero activar GPS.

Criterios de aceptación:

1. El usuario puede escribir su dirección o comuna.
2. El sistema valida la ubicación ingresada.
3. Se usa esa ubicación para búsquedas.
4. Se puede actualizar en cualquier momento.

Estado: ✅ Hecha

HU023 · Eliminación de cuenta

Yo como usuario, quiero eliminar mi cuenta, para dejar de usar la plataforma.

Criterios de aceptación:

1. El sistema solicita confirmación antes de eliminar.
2. Se valida que no tenga servicios pendientes.
3. La cuenta se elimina del sistema.
4. Se muestra mensaje de confirmación.

Estado: ✅ Hecha

HU024 · Reporte de usuarios

Yo como usuario, quiero reportar a otro usuario, para informar comportamientos indebidos.

Criterios de aceptación:

1. Se puede reportar desde el perfil o servicio.
2. Se debe ingresar un motivo.
3. El reporte queda registrado en el sistema.
4. El administrador puede revisar los reportes.

Estado: ✅ Hecha

HU025 · Visualización de disponibilidad del profesional

Yo como cliente, quiero ver los horarios disponibles del profesional, para elegir el mejor momento para mi servicio.

Criterios de aceptación:

1. Se muestra un calendario con horarios disponibles.
2. No se muestran horarios ocupados.
3. Se actualiza en tiempo real.
4. Permite seleccionar una fecha y hora fácilmente.

Estado: ✅ Hecha

HU026 · Estado del servicio

Yo como usuario, quiero ver el estado del servicio, para saber en qué etapa se encuentra.

Criterios de aceptación:

1. El servicio puede estar en estados como pendiente, confirmado, en proceso o completado.
2. El estado se actualiza automáticamente.
3. El usuario puede ver el estado en su historial.
4. Se notifica cuando cambia el estado.

Estado: ✅ Hecha

HU027 · Confirmación de inicio de servicio

Yo como profesional, quiero marcar el inicio del servicio, para indicar que ya comencé el trabajo.

Criterios de aceptación:

1. El profesional puede iniciar el servicio desde la reserva.
2. El estado cambia a "En proceso".
3. El cliente recibe notificación.
4. Solo se puede iniciar en la hora programada o después.

Estado: ✅ Hecha

HU028 · Confirmación de finalización de servicio

Yo como profesional, quiero marcar el servicio como finalizado, para cerrar el trabajo realizado.

Criterios de aceptación:

1. El profesional puede finalizar el servicio.
2. El estado cambia a "Completado".

3. Se habilita la opción de calificación.
4. El cliente recibe notificación.

Estado: ✅ Hecha

HU029 · Visualización de comentarios

Yo como usuario, quiero ver los comentarios de otros usuarios, para tomar mejores decisiones.

Criterios de aceptación:

1. Se muestran comentarios en el perfil.
2. Cada comentario incluye calificación y texto.
3. Se ordenan por fecha o relevancia.
4. Solo usuarios que usaron el servicio pueden comentar.

Estado: ✅ Hecha

HU030 · Filtro por disponibilidad inmediata

Yo como cliente, quiero filtrar profesionales disponibles de inmediato, para encontrar ayuda rápida.

Criterios de aceptación:

1. Existe un filtro de “disponible ahora”.
2. Solo muestra profesionales con horario libre cercano.
3. Se actualiza en tiempo real.
4. Se combina con otros filtros (precio, calificación, etc.).

Estado: ✅ Hecha

HU031 · Edición de perfil del cliente

Yo como cliente, quiero editar mi perfil, para mantener mis datos actualizados.

Criterios de aceptación:

1. Puedo cambiar nombre, foto y datos básicos.
2. El sistema guarda los cambios correctamente.
3. Se validan los campos obligatorios.
4. Los cambios se reflejan inmediatamente.

Estado: ✅ Hecha

HU032 · Subida de foto de perfil

Yo como usuario, quiero subir una foto de perfil, para que otros me identifiquen fácilmente.

Criterios de aceptación:

1. Se permite subir imágenes.
2. El sistema valida formato y tamaño.
3. La foto se actualiza en el perfil.
4. Se puede cambiar en cualquier momento.

Estado: ✅ Hecha

HU033 · Visualización de servicios por categoría

Yo como cliente, quiero ver servicios por categoría, para encontrar más fácil lo que necesito.

Criterios de aceptación:

1. Se muestran categorías disponibles.
2. Al seleccionar una, se listan los profesionales.
3. Se puede navegar entre categorías.
4. Se muestran resultados relacionados.

Estado: ✅ Hecha

HU034 · Guardar profesionales favoritos

Yo como cliente, quiero guardar profesionales como favoritos, para encontrarlos rápidamente después.

Criterios de aceptación:

1. Existe opción de agregar a favoritos.
2. Se pueden ver en una lista de favoritos.
3. Se pueden eliminar de favoritos.
4. Se mantienen guardados en la cuenta.

Estado: ✅ Hecha

HU035 · Visualización de favoritos

Yo como cliente, quiero ver mis profesionales favoritos, para acceder rápido a ellos.

Criterios de aceptación:

1. Se muestra lista de favoritos.
2. Se puede acceder al perfil de cada uno.
3. Se puede eliminar de la lista.
4. La lista se guarda correctamente.

Estado: ✅ Hecha

HU036 · Confirmación de reserva por el cliente

Yo como cliente, quiero confirmar una reserva, para asegurar que el servicio se realizará.

Criterios de aceptación:

1. El cliente puede confirmar después de aceptación.
2. El sistema muestra el estado confirmado.
3. Se notifica al profesional.
4. Se guarda el registro.

Estado: ✅ Hecha

HU037 · Visualización de ingresos del profesional

Yo como profesional, quiero ver mis ingresos generados, para llevar control de lo que gano.

Criterios de aceptación:

1. Se muestra total de ingresos.
2. Se puede ver por fechas.
3. Se listan servicios asociados.
4. Los datos se actualizan automáticamente.

Estado: ✅ Hecha

HU038 · Bloqueo de usuarios problemáticos

Yo como administrador, quiero bloquear usuarios, para evitar comportamientos indebidos.

Criterios de aceptación:

1. El administrador puede bloquear usuarios.
2. El usuario bloqueado no puede acceder.
3. Se guarda el motivo del bloqueo.
4. Se puede desbloquear posteriormente.

Estado: ✅ Hecha

HU039 · Búsqueda por nombre

Yo como cliente, quiero buscar profesionales por nombre, para encontrarlos directamente.

Criterios de aceptación:

1. Existe barra de búsqueda.
2. Se muestran resultados coincidentes.
3. Se puede seleccionar el perfil.
4. Funciona en tiempo real.

Estado: ✅ Hecha

HU040 · Visualización de distancia

Yo como cliente, quiero ver la distancia del profesional, para saber qué tan cerca está.

Criterios de aceptación:

1. Se muestra distancia en km.
2. Se actualiza según ubicación.
3. Aparece en listados y perfil.
4. Se usa para ordenar resultados.

Estado: ✅ Hecha

HU041 · Indicador de estado en línea

Yo como usuario, quiero ver si el otro usuario está en línea, para saber si puedo comunicarme rápido.

Criterios de aceptación:

1. Se muestra indicador de en línea o desconectado.
2. Se actualiza automáticamente.
3. Aparece en el chat.
4. Es visible para ambas partes.

Estado: ✅ Hecha

HU042 · Adjuntar archivos en el chat

Yo como usuario, quiero enviar archivos en el chat, para compartir información del servicio.

Criterios de aceptación:

1. Se permite enviar imágenes.
2. Se valida tamaño y formato.
3. El archivo se visualiza en el chat.
4. Se guarda en el historial.

Estado: ✅ Hecha

HU043 · Eliminación de mensajes

Yo como usuario, quiero eliminar mensajes enviados, para corregir errores.

Criterios de aceptación:

1. Se puede eliminar un mensaje.
2. El mensaje desaparece del chat.
3. Se actualiza para ambas partes.
4. Se mantiene la conversación.

Estado: ✅ Hecha

HU044 · Visualización de perfil del cliente

Yo como profesional, quiero ver el perfil del cliente, para conocer con quién trabajaré.

Criterios de aceptación:

1. Se muestra nombre y foto.
2. Se muestra calificación del cliente.

3. Se pueden ver comentarios.
4. Se accede desde la solicitud.

Estado: ✅ Hecha

HU045 · Alerta por baja calificación

Yo como administrador, quiero recibir alertas de usuarios con baja calificación, para revisar posibles problemas.

Criterios de aceptación:

1. El sistema detecta calificación baja.
2. Se genera una alerta automática.
3. Se puede revisar el perfil.
4. Se guarda el registro.

Estado: ✅ Hecha

HU046 · Ordenamiento por precio

Yo como cliente, quiero ordenar por precio, para elegir según mi presupuesto.

Criterios de aceptación:

1. Se puede ordenar de menor a mayor.
2. También de mayor a menor.
3. Se actualiza el listado.
4. Funciona junto con otros filtros.

Estado: ✅ Hecha

HU047 · Visualización de tiempo estimado

Yo como cliente, quiero ver el tiempo estimado del servicio, para organizar mi tiempo.

Criterios de aceptación:

1. El profesional indica duración estimada.
2. Se muestra en la reserva.
3. El cliente puede visualizarlo antes de confirmar.
4. Se guarda con el servicio.

Estado: ✅ Hecha

HU048 · Validación de datos obligatorios

Yo como usuario, quiero que el sistema valide datos obligatorios, para evitar errores al registrar información.

Criterios de aceptación:

1. El sistema detecta campos vacíos.
2. Muestra mensajes de error.
3. No permite continuar si faltan datos.
4. Indica claramente qué falta.

Estado: ✅ Hecha

HU049 · Visualización de servicios en proceso

Yo como usuario, quiero ver mis servicios en proceso, para saber cuáles están activos.

Criterios de aceptación:

1. Se muestran servicios en curso.
2. Se visualiza estado actual.
3. Se puede acceder al detalle.
4. Se actualiza en tiempo real.

Estado: ✅ Hecha

HU050 · Cierre de sesión por inactividad

Yo como usuario, quiero que el sistema cierre mi sesión por inactividad, para proteger mi cuenta.

Criterios de aceptación:

1. El sistema detecta inactividad.
2. Cierra sesión automáticamente.
3. Muestra mensaje informativo.
4. Redirige al inicio de sesión.

Estado: ✅ Hecha

3. Requisitos no funcionales

Se formulan con el modelo de calidad **ISO/IEC 25010**. Cada uno tiene una meta medible, la forma en que el sistema la cumple y cómo se verifica. El estado indica si ya está cumplido, si es una meta que se mide en el despliegue o si tiene una acción pendiente.

3.1 Seguridad

ID	Requisito	Cómo se cumple	Verificación	Estado
RNF-SEG-01	Solo usuarios autenticados acceden a sus datos	Autenticación por token (Laravel Sanctum) en todas las rutas privadas	Tests de 401 sin token en cada módulo	✅
RNF-SEG-02	Cada usuario solo opera sobre sus recursos y según su rol	Middleware `role:` (tipo de cuenta) y Policies (dueño del recurso); los usuarios bloqueados no entran (`not_blocked`)	Tests de 403 por endpoint (otro usuario, otro rol)	✅
RNF-SEG-03	Contraseñas protegidas	Hash bcrypt (cast `hashed`), mínimo 8 caracteres y confirmación	Tests de registro	✅
RNF-SEG-04	Resistencia a fuerza bruta en el acceso	Máximo 10 intentos por minuto en registro, inicio de sesión y recuperación de contraseña (`throttle:10,1`)	Revisión de rutas; prueba manual	✅
RNF-SEG-05	Toda entrada se valida en el servidor	FormRequest por endpoint con errores 422 por campo. El cliente valida además con Zod, sin reemplazar al servidor	Tests de validación en cada endpoint	✅
RNF-SEG-06	Los documentos de identidad y los adjuntos del chat no son públicos	Disco privado y URL firmada que vence (60 min); sin firma responde 403	ChatTest; prueba manual de URL sin firma	✅
RNF-SEG-07	Tráfico cifrado	HTTPS (TLS) y WebSockets seguros (`wss`) en producción	Checklist de despliegue	🎯 En despliegue
RNF-SEG-08	La API solo acepta peticiones del frontend oficial	Publicar `config/cors.php` con el dominio del frontend en producción (hoy acepta cualquier origen, valor por defecto de Laravel)	Checklist de despliegue	⚠️ Pendiente
RNF-SEG-09	Las dependencias no tienen vulnerabilidades conocidas	Auditoría de dependencias (`composer audit`, `npm audit`) en cada PR	CI (job "Seguridad de dependencias")	✅
RNF-SEG-10	No se puede averiguar qué correos tienen cuenta	La recuperación de contraseña responde igual exista o no el correo	PasswordResetTest	✅
RNF-SEG-11	Las sesiones abandonadas se cierran	Cierre automático tras 15 minutos sin actividad en el navegador (HU050)	Prueba manual	✅

3.2 Privacidad y protección de datos

ID	Requisito	Cómo se cumple	Verificación	Estado
RNF-PRI-01	El perfil público no expone datos de contacto	`ProfessionalResource` excluye correo, teléfono, dirección y coordenadas	ProfessionalsTest	✅
RNF-PRI-02	El chat solo lo ven las partes de la reserva	Policy `chat` (ni otros usuarios ni el administrador)	ChatTest	✅
RNF-PRI-05	El perfil de un cliente solo lo ve el profesional que tiene una reserva con él	Policy `viewClientProfile` (HU044)	UserReportsAndModerationTest	✅

ID	Requisito	Cómo se cumple	Verificación	Estado
RNF-PRI-03	Autorización informada para el tratamiento de datos (Ley 1581)	Aceptación explícita de la política de tratamiento al registrarse	—	⚠️ Pendiente (ver 07)
RNF-PRI-04	Derecho de supresión	Eliminación de cuenta con anonimización: se borran datos de contacto, ubicación, fotos y documentos, y se conserva el historial de la otra parte (HU023, decisión D12)	AccountTest	✅

3.3 Integridad y auditoría

ID	Requisito	Cómo se cumple	Verificación	Estado
RNF-INT-01	Cada cambio de estado de una reserva queda registrado con autor, fecha y motivo	Tabla `booking_status_changes`; todo cambio pasa por `BookingService::transition()`	BookingHistoryTest, BookingsTest	✅
RNF-INT-02	No se pueden reservar dos servicios a la misma hora	Validación de cruces con descanso y bloqueo de la fila del profesional dentro de la transacción (`lockForUpdate`)	BookingsTest, ProfessionalSlotsTest	✅
RNF-INT-03	Los estados solo cambian por transiciones válidas	Máquina de estados en `BookingStatus`; una transición inválida responde 409	Tests de 409 en cada acción	✅
RNF-INT-04	Integridad referencial	Llaves foráneas en todas las relaciones; los servicios con reservas no se borran (se ocultan)	Migraciones; ProfessionalProfileTest	✅
RNF-INT-05	El precio pactado no cambia después de reservar	Copia del precio en `bookings.agreed_price`	BookingsTest, ProfessionalProfileTest	✅

3.4 Eficiencia de desempeño

ID	Requisito	Cómo se cumple	Verificación	Estado
RNF-REN-01	El 95 % de las respuestas de la API en menos de 500 ms con la carga esperada del piloto	Listados paginados (15–20, máximo 50), índices en las columnas de búsqueda, carga anticipada de relaciones (`with`), promedio de calificación precalculado (D13)	Medición con el servidor de producción	🎯 En despliegue
RNF-REN-02	Eventos en tiempo real entregados en menos de 2 s	Laravel Reverb con cola; los eventos se emiten después de confirmar la transacción	Prueba manual con dos sesiones	✅ (local)
RNF-REN-03	Carga inicial del frontend ligera	Rutas cargadas bajo demanda (*lazy loading*) y build optimizado de Vite	Build en CI	✅

3.5 Disponibilidad y fiabilidad

ID	Requisito	Cómo se cumple	Verificación	Estado
RNF-DIS-01	Disponibilidad mensual ≥ 99 % durante el piloto	Procesos supervisados (Supervisor reinicia la cola y Reverb) y verificación de salud (`/up`)	Monitoreo externo	🎯 En despliegue
RNF-DIS-02	Recuperación ante pérdida de datos	Respaldo diario de la base y los archivos, con 7 días de retención; restauración documentada	Prueba de restauración (06)	🎯 En despliegue
RNF-DIS-03	Un fallo del tiempo real no impide usar la app	Sin WebSocket la app sigue funcionando y se actualiza al recargar o al volver a la pestaña	Prueba manual con Reverb apagado	✅

3.6 Usabilidad y accesibilidad

ID	Requisito	Cómo se cumple	Verificación	Estado
RNF-USA-01	Interfaz usable en celular, tableta y computador	Diseño responsive con Tailwind; menú móvil	Revisión manual en tres anchos	✅
RNF-USA-02	Mensajes claros y en español	Mensajes de validación y de error en español, con el nombre del campo	ApiErrorsTest y tests de cada formulario	✅
RNF-USA-03	Toda vista con datos muestra cargando, error, vacío y contenido	Patrón obligatorio del frontend (`docs/arquitectura.md`)	Revisión en PR; tests de componentes	✅

ID	Requisito	Cómo se cumple	Verificación	Estado
RNF-USA-04	Accesibilidad base (meta: WCAG 2.1 nivel AA)	HTML semántico, `aria-label` en controles de solo ícono, foco visible, contraste de la paleta, diálogos que se cierran con Esc	Revisión manual y auditoría Lighthouse	🎯 Auditoría pendiente
RNF-USA-05	Identidad visual consistente	Sistema de diseño con tokens; un verificador en CI rechaza colores fuera de la paleta	`npm run lint:tokens`	✅

3.7 Mantenibilidad

ID	Requisito	Cómo se cumple	Verificación	Estado
RNF-MAN-01	Código modular y por capas	Backend: Ruta → FormRequest → Policy → Service → Resource. Frontend: módulos autocontenidos con API pública	Revisión en PR; regla de lint de imports	✅
RNF-MAN-02	Cada cambio pasa pruebas automáticas antes de unirse	CI obligatorio en `main`: tests, formato, lint, tipos y build	Protección de la rama	✅
RNF-MAN-03	Reglas de calidad protegidas	Check **Guardas**: la configuración de calidad no se modifica sin aprobación explícita	CI	✅
RNF-MAN-04	Contrato de API documentado y sincronizado	`docs/api/endpoints.md` se actualiza en el mismo PR; especificación OpenAPI generada del código	Revisión en PR	✅

3.8 Compatibilidad y portabilidad

ID	Requisito	Cómo se cumple	Verificación	Estado
RNF-COM-01	Navegadores modernos	Últimas dos versiones de Chrome, Edge, Firefox y Safari	Revisión manual	✅
RNF-COM-02	Versiones de PHP soportadas	PHP 8.2 a 8.5	CI corre las pruebas en 8.2 y 8.5	✅
RNF-COM-03	Base de datos	MySQL 8 en desarrollo y producción; SQLite en memoria para las pruebas	CI	✅

3.9 Escalabilidad

ID	Requisito	Cómo se cumple	Estado
RNF-ESC-01	La API puede crecer horizontalmente	API sin estado de sesión (tokens); colas para el trabajo diferido	✅ Diseño
RNF-ESC-02	El tiempo real puede crecer	Reverb admite varias instancias coordinadas con Redis	🎯 Futuro
RNF-ESC-03	Los archivos pueden salir del servidor	Discos de Laravel intercambiables (local → S3) sin cambiar el código	🎯 Futuro

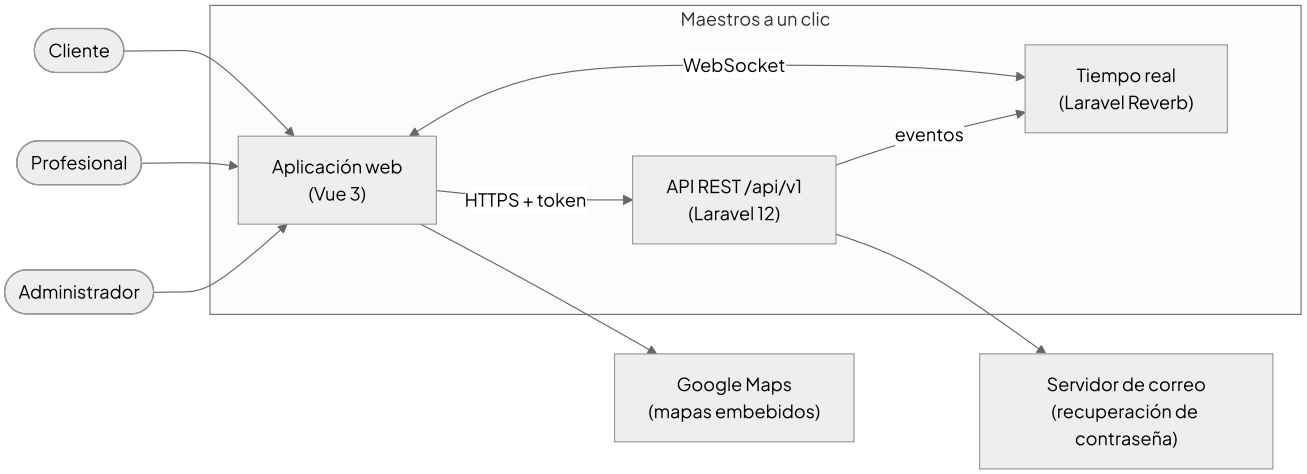
Leyenda: ✅ cumplido · 🎯 meta que se verifica en el despliegue · ⚠️ acción pendiente.

Diagramas del sistema

Diagramas de análisis y diseño hechos a partir del sistema actual: modelos, migraciones, rutas y servicios. Están en Mermaid para que se versionen con el código: GitHub los dibuja al abrir este archivo y el generador de PDF los incluye. Reemplazan a los diagramas PNG del entregable original, que eran anteriores al modelo definitivo.

1. Contexto del sistema

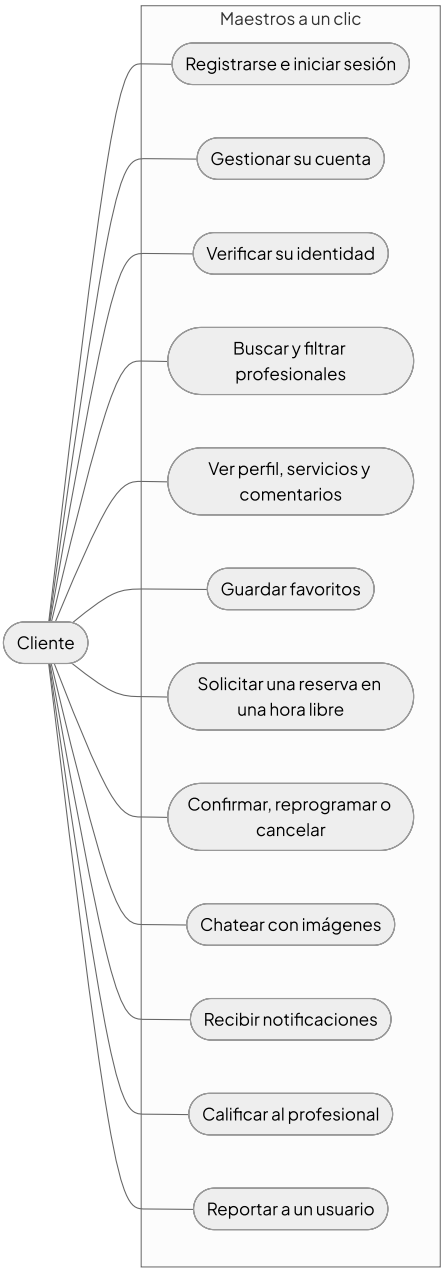
Quién usa el sistema y con qué se integra.



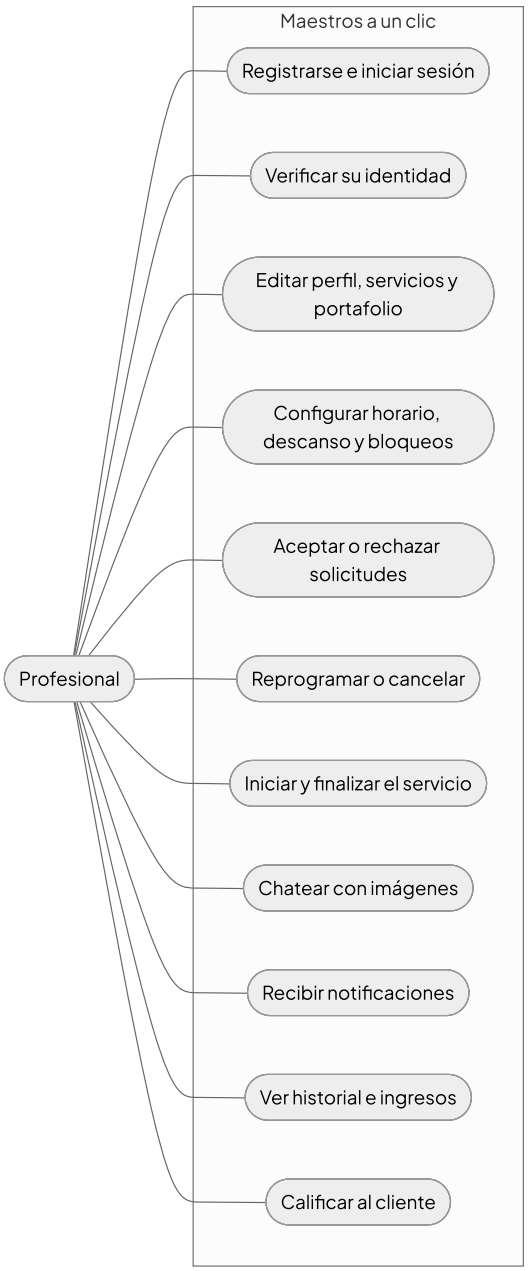
2. Casos de uso

Un diagrama por actor. Las historias que cubre cada caso están en 02-requisitos.md.

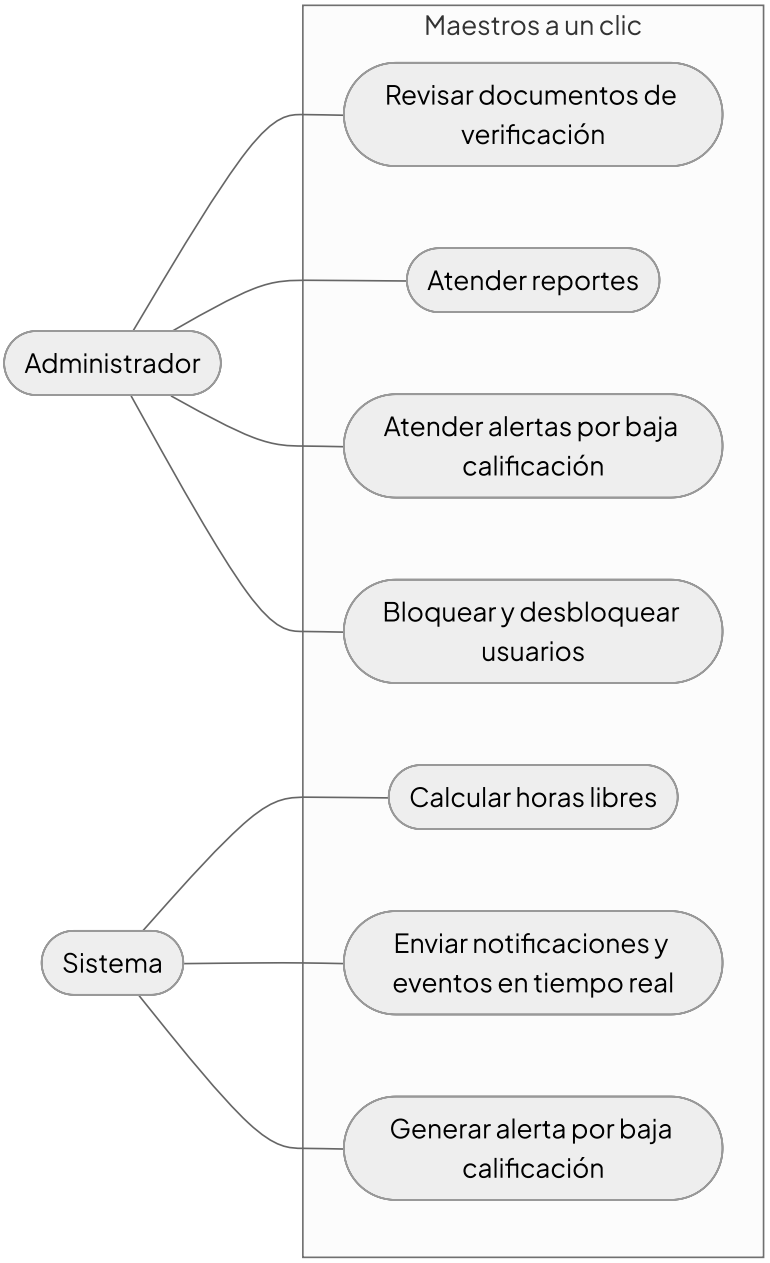
2.1 Cliente


Maestros a un clic
Cliente
Registrarse e iniciar sesión
Gestionar su cuenta
Verificar su identidad
Buscar y filtrar profesionales
Ver perfil, servicios y comentarios
Guardar favoritos
Solicitar una reserva en una hora libre
Confirmar, reprogramar o cancelar
Chatear con imágenes
Recibir notificaciones
Calificar al profesional
Reportar a un usuario

2.2 Profesional

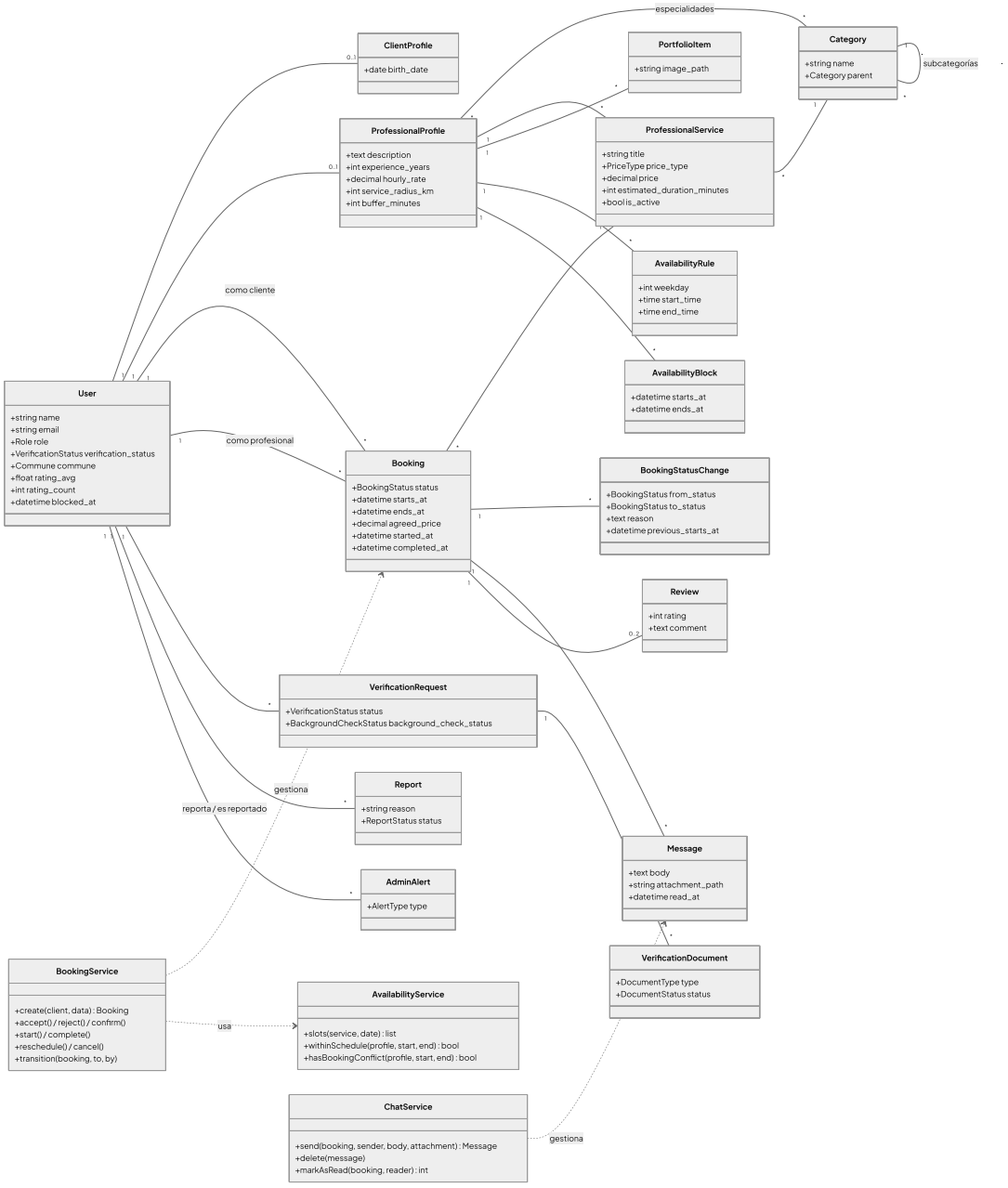


2.3 Administrador y sistema



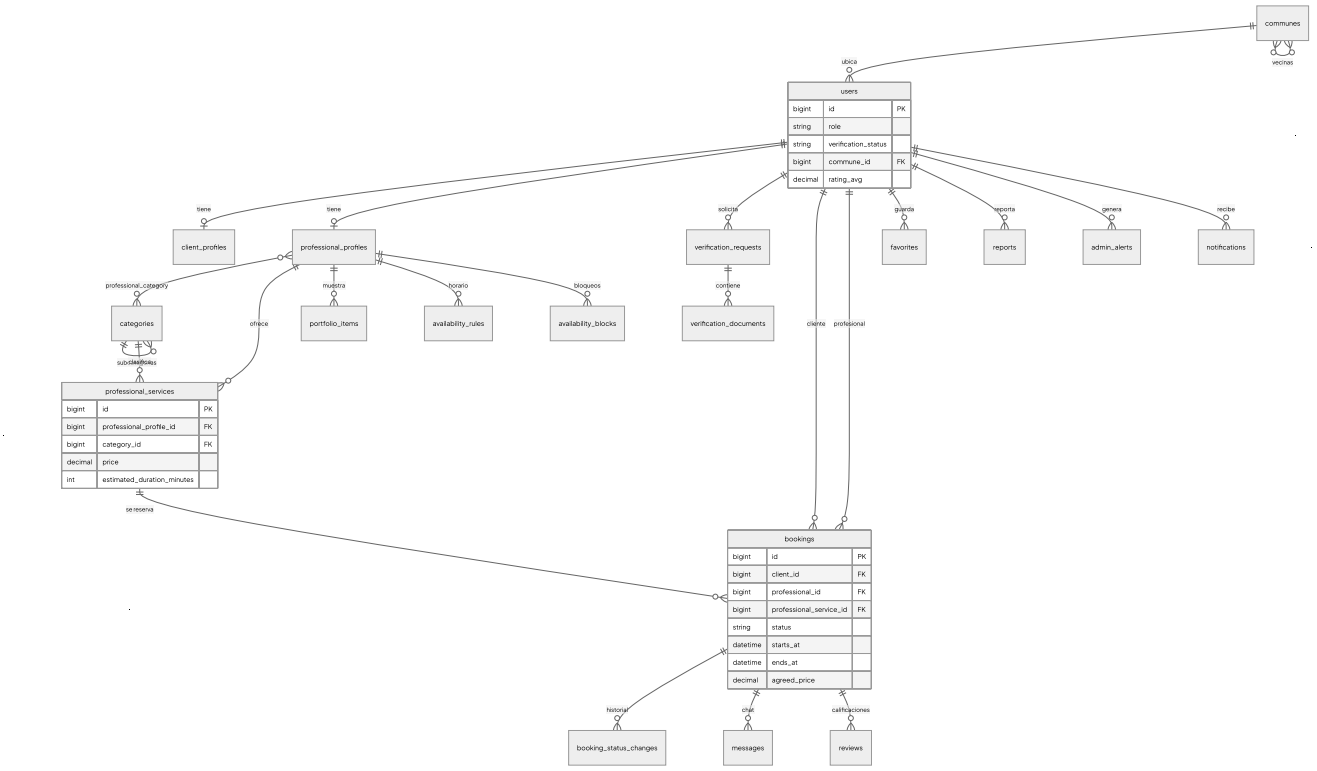
3. Clases del dominio

Modelos principales con sus atributos clave y los servicios que aplican las reglas de negocio. Los estados, roles y tipos son enumeraciones de PHP (`app/Enums`).



4. Modelo entidad-relación

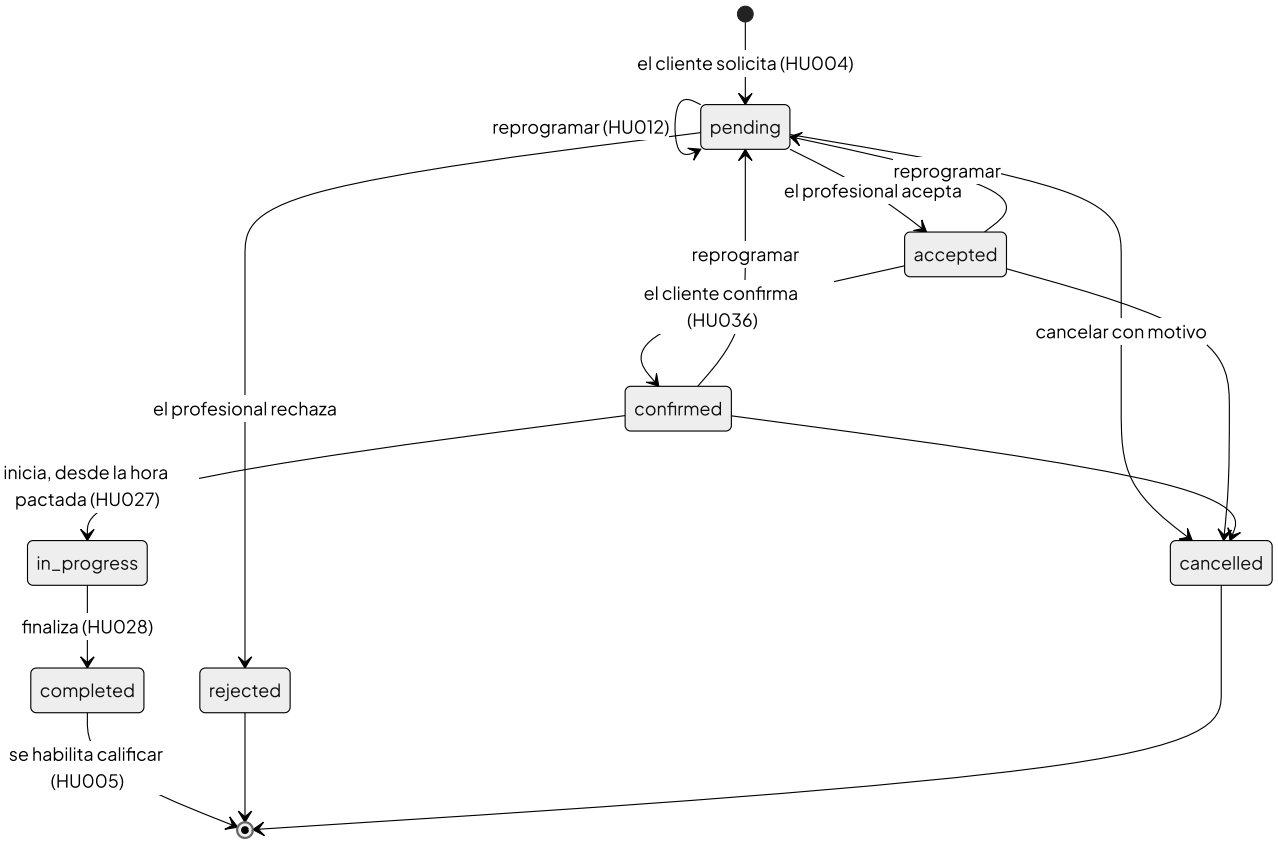
Entidades y relaciones con sus llaves. Las columnas completas de cada tabla están en docs/modelo-de-datos.md §2.



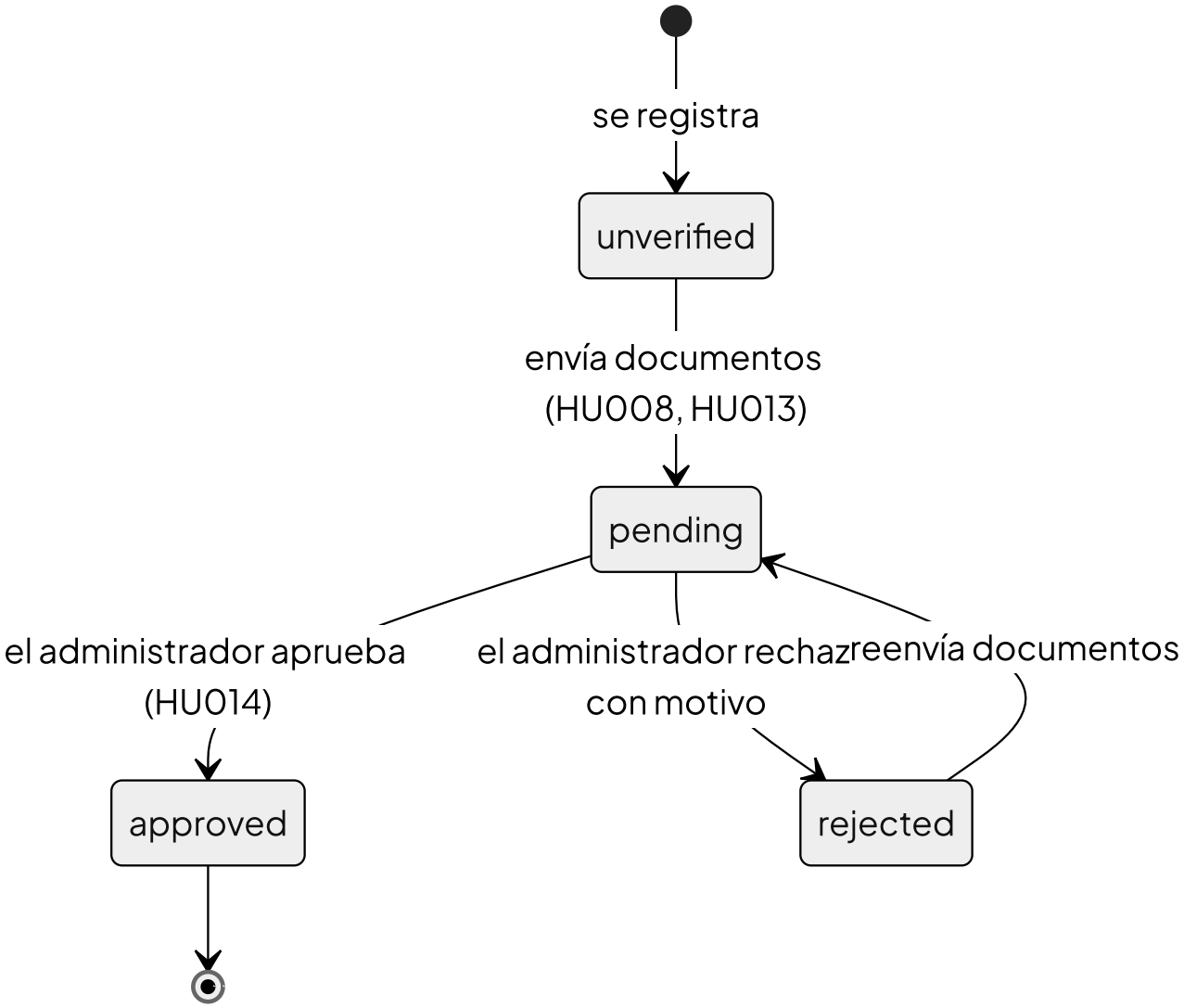
5. Estados

5.1 Reserva

Todo cambio pasa por `BookingService::transition()` y queda en `booking_status_changes`.

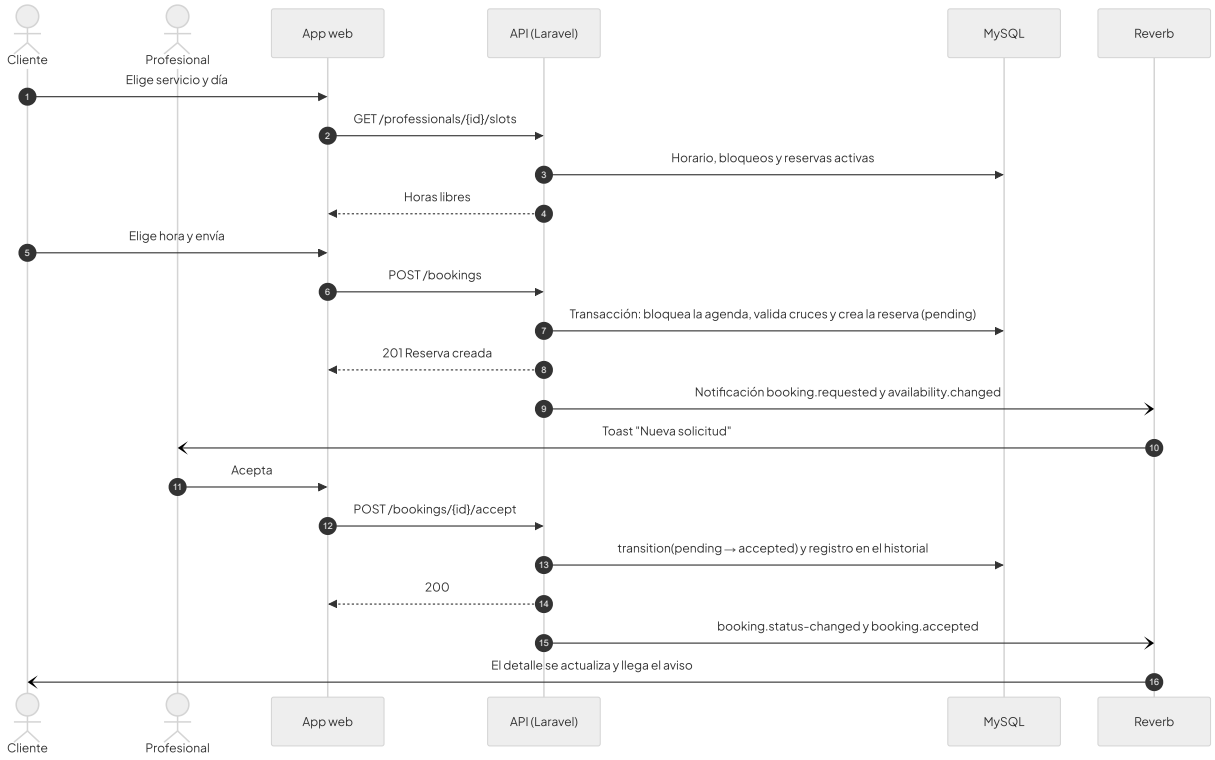


5.2 Verificación de identidad

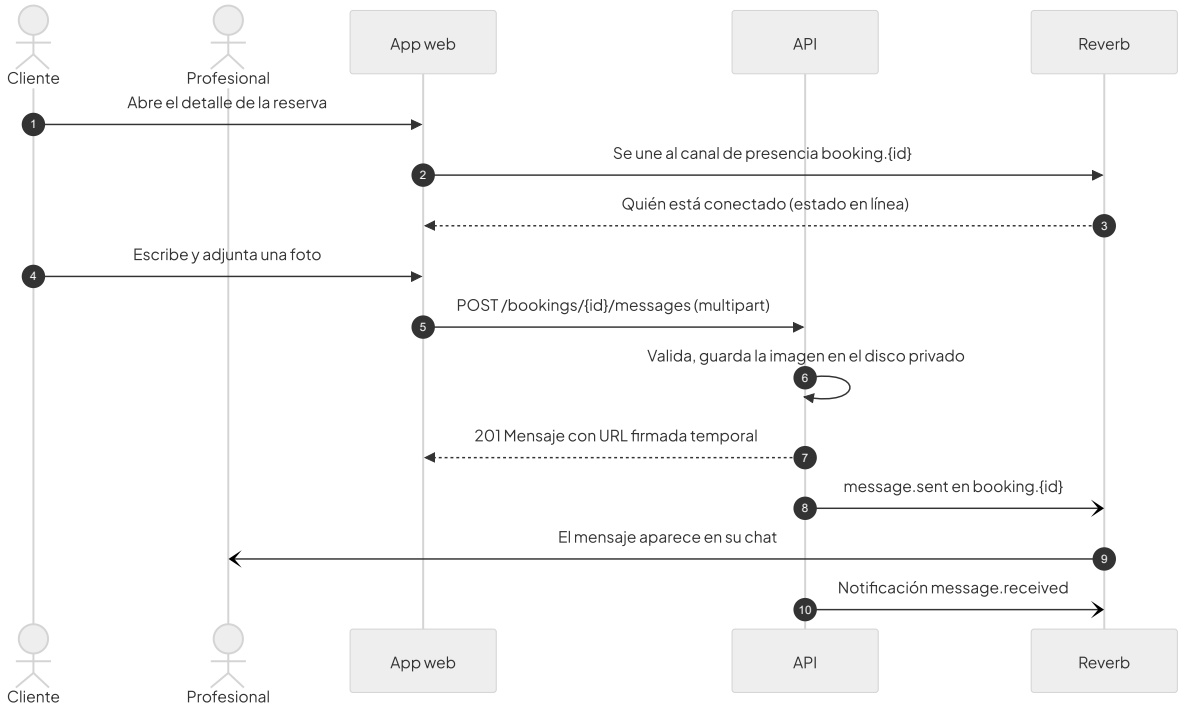


6. Secuencias

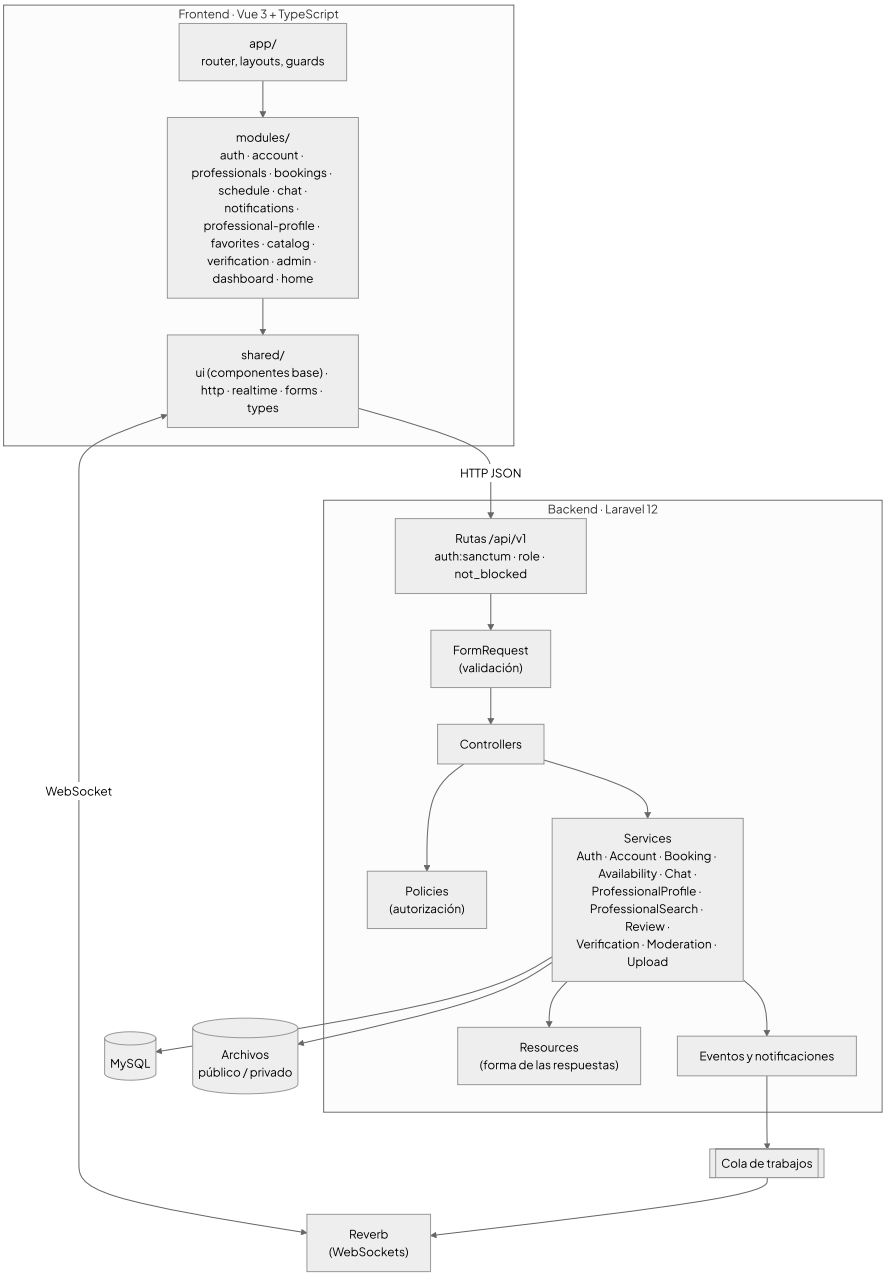
6.1 Solicitar y aceptar una reserva



6.2 Mensaje en el chat

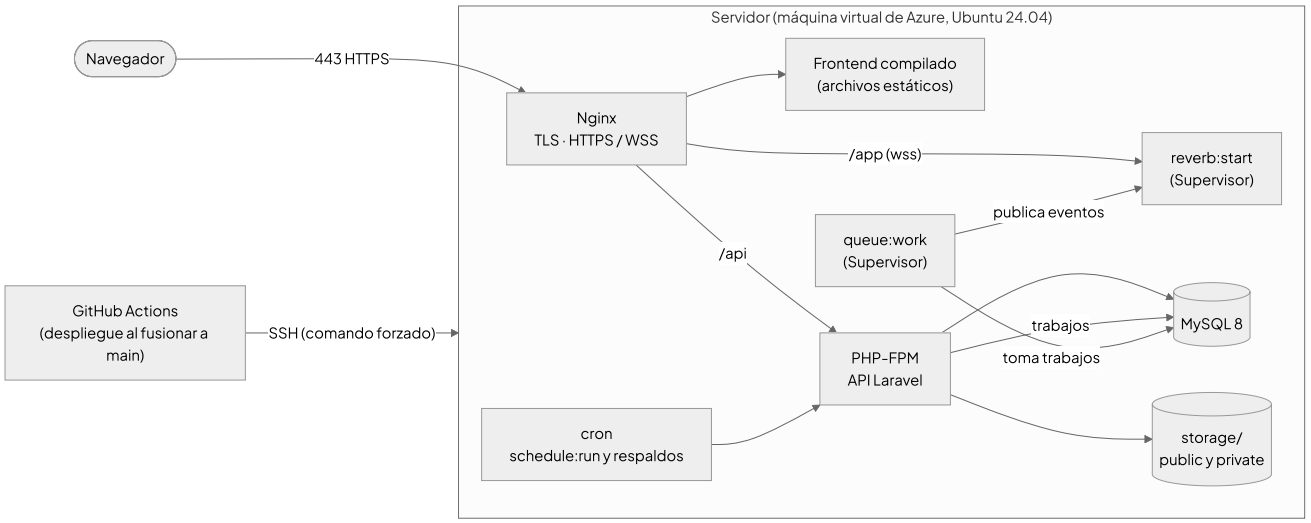


7. Componentes



8. Despliegue

Topología de producción en Azure, con despliegue continuo desde GitHub. Los pasos están en 06-despliegue-y-operacion.md.


Servidor (máquina virtual de Azure, Ubuntu 24.04)
Navegador
443 HTTPS
Nginx
TLS · HTTPS / WSS
Frontend compilado
(archivos estáticos)
/app (wss)
reverb:start
(Supervisor)
publica eventos
queue:work
(Supervisor)
/api
GitHub Actions
(despliegue al fusionar a
main)
SSH (comando forzado)
trabajos
toma trabajos
MySQL 8
PHP-FPM
API Laravel
cron
schedule:run y respaldos
storage/
public y private

Matriz de trazabilidad

Relaciona cada historia de usuario con lo que la implementa (endpoints y eventos del backend, vistas y componentes del frontend) y con las pruebas automatizadas que la verifican. Sirve para comprobar que cada requisito está cubierto y para ubicar rápido el código de una funcionalidad.

- **Backend:** rutas bajo `/api/v1` (contrato) y eventos de tiempo real (tiempo-real.md).
- **Frontend:** vistas en `src/modules/<módulo>/views` y componentes en `src/modules/<módulo>/components` del repositorio del frontend.
- **Pruebas:** `*Test` son pruebas de API del backend (`tests/Feature`); `*.spec` son pruebas del frontend (`__tests__` de cada módulo).
- **Estado:** el de su issue en GitHub al generar el documento.

HU	Módulo	Backend (API y eventos)	Frontend	Pruebas	Estado
HU001 Inicio de sesión	auth	POST /auth/login	LoginView (/login)	Auth/LoginTest · auth/schemas.spec	✅ Hecha
HU002 Búsqueda por geolocalización	búsqueda	GET /professionals?lat=&lng= (radio de acción, comunas aledañas)	SearchResultsView (geolocalización del navegador)	ProfessionalsTest	✅ Hecha
HU003 Gestión de disponibilidad y agenda	agenda	GET /professional/availability · PUT …/rules · POST/DELETE …/blocks · evento availability.changed	AvailabilityView (WeeklyScheduleEditor, BlocksManager)	AvailabilityTest · WeeklyScheduleEditor.spec · weekly.spec	✅ Hecha
HU004 Solicitud y aceptación de servicio	reservas	POST /bookings · POST /bookings/{id}/accept · /reject · GET /professionals/{id}/slots	NewBookingView · BookingResponseActions · PendingRequestsPanel	BookingsTest · ProfessionalSlotsTest · BookingResponseActions.spec · SlotPicker.spec	✅ Hecha
HU005 Sistema de calificación recíproca	calificaciones	POST /bookings/{id}/review	ReviewBookingModal (detalle de la reserva)	ReviewsTest · ReviewBookingModal.spec	✅ Hecha
HU006 Registro de cliente	auth	POST /auth/register/client	RegisterClientView	RegisterClientTest · auth/schemas.spec	✅ Hecha
HU007 Registro de profesional	auth	POST /auth/register/professional	RegisterProfessionalView	Auth/RegisterProfessionalTest · auth/schemas.spec	✅ Hecha
HU008 Cargue de documentos	verificación-admin	GET/POST /verification	VerificationView	VerificationTest · verification/schemas.spec	✅ Hecha
HU009 Edición de perfil profesional	perfil-profesional	GET/PATCH /professional/profile · PUT /professional/categories · /professional/services · /professional/portfolio	AboutMeView · MyServicesView	ProfessionalProfileTest · professional-profile/schemas.spec · ServiceFormModal.spec	✅ Hecha
HU010 Filtros y ordenamiento de profesionales	búsqueda	GET /professionals (categoría, precio, orden)	SearchResultsView	ProfessionalsTest	✅ Hecha
HU011 Chat y confirmación de detalles del servicio	chat-notificaciones	GET/POST /bookings/{id}/messages · POST …/messages/read · evento message.sent	BookingChat (detalle de la reserva)	ChatTest · BookingChat.spec	✅ Hecha
HU012 Cancelación y reprogramación de reserva	reservas	POST /bookings/{id}/reschedule · POST /bookings/{id}/cancel	RescheduleBookingModal · CancelBookingModal	BookingRescheduleTest · BookingsTest · RescheduleBookingModal.spec	✅ Hecha
HU013 Validación de identidad y antecedentes	verificación-admin	POST /verification/documents/{id} · PATCH /admin/verification-documents/{id}	VerificationView · ReplaceDocumentModal	VerificationTest · AdminVerificationTest · ReplaceDocumentModal.spec	✅ Hecha

HU	Módulo	Backend (API y eventos)	Frontend	Pruebas	Estado
HU014 Revisión y aprobación de documentos del profesional	verificación-admin	/admin/verifications/* (aprobar, rechazar, ver archivo)	AdminVerificationsView · AdminVerificationDetailModal	AdminVerificationTest · AdminVerifications.spec	✅ Hecha
HU016 Historial de servicios del cliente	reservas	GET /bookings	MyBookingsView (filtros por estado)	BookingsTest · BookingList.spec	✅ Hecha
HU017 Historial de servicios del profesional	reservas	GET /bookings · GET /bookings/summary	ProfessionalBookingsView (totales y filtros)	BookingsTest · BookingList.spec	✅ Hecha
HU018 Notificaciones del sistema	chat-notificaciones	GET /notifications · POST /notifications/{id}/read · /read-all · canal App.Models.User.{id}	NotificationBell (barra superior)	NotificationsTest · NotificationBell.spec	✅ Hecha
HU019 Cambio de contraseña	cuenta	PUT /me/password	SecuritySettingsView · ChangePasswordForm	AccountTest · ChangePasswordForm.spec	✅ Hecha
HU020 Recuperación de contraseña	cuenta	POST /auth/forgot-password · POST /auth/reset-password	ForgotPasswordView · ResetPasswordView	Auth/PasswordResetTest · PasswordRecovery.spec	✅ Hecha
HU021 Cerrar sesión	auth	POST /auth/logout	AppNavbar (Salir)	Auth/SessionTest · router.spec	✅ Hecha
HU022 Registro de ubicación manual	cuenta	PATCH /me (comuna, dirección, coordenadas)	LocationSettingsView · ManualLocationForm	AccountTest · ManualLocationForm.spec	✅ Hecha
HU023 Eliminación de cuenta	cuenta	DELETE /me (anonimiza, D12)	SecuritySettingsView · DeleteAccountModal	AccountTest · DeleteAccountModal.spec	✅ Hecha
HU024 Reporte de usuarios	calificaciones	POST /reports · GET /admin/reports · POST /admin/reports/{id}/resolve	ReportUserModal · AdminReportsView	UserReportsAndModerationTest · ReportUserModal.spec	✅ Hecha
HU025 Visualización de disponibilidad del profesional	agenda	GET /professionals/{id}/slots · evento availability.changed	SlotPicker (en NewBookingView y Reprogramar)	ProfessionalSlotsTest · AvailabilityTest · SlotPicker.spec · dates.spec	✅ Hecha
HU026 Estado del servicio	reservas	GET /bookings/{id} · GET /bookings/{id}/history · evento booking.status-changed	BookingDetailView · BookingHistory	BookingHistoryTest · BookingHistory.spec · presence.spec	✅ Hecha
HU027 Confirmación de inicio de servicio	reservas	POST /bookings/{id}/start	ServiceProgressCard	BookingsTest · ServiceProgressCard.spec	✅ Hecha
HU028 Confirmación de finalización de servicio	reservas	POST /bookings/{id}/complete	ServiceProgressCard	BookingsTest · ServiceProgressCard.spec	✅ Hecha
HU029 Visualización de comentarios	calificaciones	GET /professionals/{id}/reviews	ProfessionalDetailView (comentarios)	UserReportsAndModerationTest · ReviewsTest	✅ Hecha
HU030 Filtro por disponibilidad inmediata	búsqueda	GET /professionals?available_now=1	SearchResultsView (filtro de disponibles ahora)	UserReportsAndModerationTest	✅ Hecha
HU031 Edición de perfil del cliente	cuenta	PATCH /me	ProfileSettingsView	AccountTest · account/schemas.spec	✅ Hecha

HU	Módulo	Backend (API y eventos)	Frontend	Pruebas	Estado
HU032 Subida de foto de perfil	cuenta	POST /me/avatar	ProfileSettingsView	AccountTest	✅ Hecha
HU033 Visualización de servicios por categoría	búsqueda	GET /categories · GET /professionals?category_id=	SearchResultsView (categoría)	CatalogTest · ProfessionalsTest	✅ Hecha
HU034 Guardar profesionales favoritos	favoritos	POST/DELETE /favorites/{professionalId}	FavoriteButton	FavoritesTest · FavoriteButton.spec	✅ Hecha
HU035 Visualización de favoritos	favoritos	GET /favorites	MyFavoritesView	FavoritesTest · FavoriteButton.spec	✅ Hecha
HU036 Confirmación de reserva por el cliente	reservas	POST /bookings/{id}/confirm	ConfirmBookingButton	BookingsTest · ConfirmBookingButton.spec	✅ Hecha
HU037 Visualización de ingresos del profesional	perfil-profesional	GET /professional/earnings?from=&to=	EarningsView	EarningsTest · EarningsView.spec · ranges.spec	✅ Hecha
HU038 Bloqueo de usuarios problemáticos	verificación-admin	GET /admin/users · POST /admin/users/{id}/block · /unblock · middleware not_blocked	AdminUsersView	UserReportsAndModerationTest	✅ Hecha
HU039 Búsqueda por nombre	búsqueda	GET /professionals?q=	SearchResultsView (barra de búsqueda)	ProfessionalsTest	✅ Hecha
HU040 Visualización de distancia	búsqueda	GET /professionals?lat=&lng= (distance_km)	ProfessionalCard · ProfessionalDetailView	ProfessionalsTest · ProfessionalCard.spec	✅ Hecha
HU041 Indicador de estado en línea	chat-notificaciones	Canal de presencia booking.{id}	BookingChat (En línea / Desconectado)	BroadcastingTest · presence.spec	✅ Hecha
HU042 Adjuntar archivos en el chat	chat-notificaciones	POST /bookings/{id}/messages (attachment)	BookingChat	ChatTest · BookingChat.spec	✅ Hecha
HU043 Eliminación de mensajes	chat-notificaciones	DELETE /messages/{id} · evento message.deleted	BookingChat	ChatTest · BookingChat.spec	✅ Hecha
HU044 Visualización de perfil del cliente	calificaciones	GET /clients/{id} (solo el profesional con reserva)	ClientProfileModal (detalle de la reserva)	UserReportsAndModerationTest	✅ Hecha
HU045 Alerta por baja calificación	calificaciones	GET /admin/alerts · POST /admin/alerts/{id}/resolve	AdminAlertsView	UserReportsAndModerationTest	✅ Hecha
HU046 Ordenamiento por precio	búsqueda	GET /professionals?sort=price_asc	price_desc	SearchResultsView	ProfessionalsTest
HU047 Visualización de tiempo estimado	perfil-profesional	/professional/services (estimated_duration_minutes)	MyServicesView · NewBookingView · BookingDetailView	ProfessionalProfileTest · professional-profile/schemas.spec	✅ Hecha
HU048 Validación de datos obligatorios	auth	FormRequest → 422 con errores por campo	Formularios con VeeValidate + Zod	ApiErrorsTest · forms.spec · auth/schemas.spec	✅ Hecha

HU	Módulo	Backend (API y eventos)	Frontend	Pruebas	Estado
HU049 Visualización de servicios en proceso	reservas	GET /bookings?status=in_progress	InProgressPanel (paneles de inicio)	BookingsTest · InProgressPanel.spec	✅ Hecha
HU050 Cierre de sesión por inactividad	cuenta	— (cierre por inactividad en el cliente)	useInactivityTimer (App.vue)	Sin prueba automática; verificado a mano	✅ Hecha

Cobertura por módulo

Módulo	Responsable	Historias	Hechas
Autenticación (fase 0)	Equipo	HU001, HU006, HU007, HU021, HU048	5 de 5
Favoritos (fase 0)	Equipo	HU034, HU035	2 de 2
Agenda	LordCasta	HU003, HU025	2 de 2
Reservas	LordCasta	HU004, HU012, HU016, HU017, HU026, HU027, HU028, HU036, HU049	9 de 9
Perfil profesional	LordCasta	HU009, HU037, HU047	3 de 3
Chat y notificaciones	LordCasta	HU011, HU018, HU041, HU042, HU043	5 de 5
Búsqueda	barenas12	HU002, HU010, HU030, HU033, HU039, HU040, HU046	7 de 7
Cuenta	barenas12	HU019, HU020, HU022, HU023, HU031, HU032, HU050	7 de 7
Verificación y administración	barenas12	HU008, HU013, HU014, HU038	4 de 4
Calificaciones	barenas12	HU005, HU024, HU029, HU044, HU045	5 de 5
Total			**49 de 49**

La única historia sin prueba automatizada es la HU050 (cierre de sesión por inactividad), que vive solo en el cliente y se verificó a mano.

Plan y reporte de pruebas

1. Objetivo

Verificar de forma automática y repetible que cada historia de usuario cumple sus criterios de aceptación, que las reglas de negocio (agenda, estados de la reserva, permisos) no se rompen al agregar funcionalidades, y que la calidad del código se mantiene.

2. Estrategia

Nivel	Qué se prueba	Herramienta	Dónde
Pruebas de API (integración)	Cada endpoint: caso feliz, validación (422), permisos (401/403), reglas de estado (409) y efectos (registros, notificaciones, eventos, archivos)	Pest (PHPUnit) sobre SQLite en memoria	Backend `tests/Feature`
Pruebas unitarias del frontend	Funciones puras: fechas en hora de Colombia, rangos, horario semanal, esquemas de validación, cliente HTTP, formato	Vitest	Frontend `__tests__`
Pruebas de componentes	Interacción principal de cada componente con datos: lo que se muestra, lo que se envía a la API y cómo se manejan los errores	Vitest + Vue Test Utils (API simulada)	Frontend `__tests__`
Análisis estático	Tipos, estilo, imports entre módulos y colores fuera del sistema de diseño	TypeScript (vue-tsc), ESLint, oxlint, Pint, Prettier y verificador de tokens	Ambos
Seguridad de dependencias	Vulnerabilidades conocidas en las librerías	`composer audit`, `npm audit`	CI
Pruebas manuales	Recorridos completos en el navegador con dos sesiones (cliente y profesional), tiempo real y responsive	Navegador	Antes de fusionar cada HU

Reglas de la estrategia:

- Toda historia nueva incluye sus pruebas en el mismo pull request. Sin pruebas no cumple la definición de terminado.
- Por endpoint, como mínimo: caso feliz, validación y permisos.
- Las pruebas no dependen de servicios externos: la API se simula en el frontend, y en el backend se simulan las notificaciones, los eventos y el almacenamiento (`Notification::fake`, `Event::fake`, `Storage::fake`).
- Las fechas se fijan en las pruebas que dependen de la hora (`Carbon::setTestNow`, `vi.useFakeTimers`), para que den el mismo resultado cualquier día.
- No se permite saltar, enfocar ni borrar pruebas para que la CI pase: el check **Guardas** lo detecta y lo marca en el PR.

3. Integración continua

Cada pull request a `main` ejecuta automáticamente, y GitHub no permite fusionar si algo falla:

Repositorio	Verificaciones obligatorias
Backend	Formato (Pint) y pruebas (Pest) en **PHP 8.2 y 8.5**; Guardas
Frontend	Lockfile, lint (oxlint, ESLint, tokens de diseño), formato, tipos, pruebas y build de producción; Guardas
Ambos (informativo)	Auditoría de dependencias

4. Reporte de resultados

Ejecución del 22 de octubre de 2026 sobre `main`, con las 49 historias terminadas:

Suite	Pruebas	Resultado	Aserciones	Duración
Backend (Pest)	220	✅ 220 aprobadas	992	≈ 23 s
Frontend (Vitest)	158 en 38 archivos	✅ 158 aprobadas	—	≈ 60 s (en Windows)

4.1 Backend: pruebas por archivo

Archivo	Qué cubre
BookingsTest	Crear reservas (horario, bloqueos, cruces, precio), aceptar o rechazar, confirmar, iniciar y finalizar, listados y filtros, resumen, notificaciones
ProfessionalProfileTest	Perfil, especialidades, servicios con duración, portafolio y permisos
AvailabilityTest	Horario semanal, descanso, bloqueos, cruces y evento de agenda
BookingRescheduleTest	Reprogramar y cancelar con sus reglas y notificaciones
ChatTest	Enviar texto e imágenes, chat cerrado, permisos, leídos y eliminar mensajes
ApiErrorsTest	Formato estándar de errores (404, 405, 422, 401, 403)
ProfessionalsTest	Búsqueda pública, filtros, orden, cercanía, comunas aledañas, próxima hora libre y privacidad del perfil
ProfessionalSlotsTest	Cálculo de horas libres
NotificationsTest	Listado, contador y marcar como leídas
EarningsTest	Ingresos por rango y validaciones
BookingHistoryTest	Historial de estados y evento de estado
Auth/*, RegisterClientTest	Registro, inicio y cierre de sesión, y recuperación de contraseña sin revelar qué correos existen
AccountTest	Editar datos y ubicación, foto, cambio de contraseña y eliminación de cuenta con anonimización
VerificationTest, AdminVerificationTest	Cargue y reemplazo de documentos, y revisión del administrador
ReviewsTest	Calificación recíproca y promedio
UserReportsAndModerationTest	Reportes, bloqueo de usuarios, alertas, perfil del cliente, comentarios públicos y "disponible ahora"
FavoritesTest, CatalogTest, BroadcastingTest, ApiDocsTest, DatabaseSeederTest	Favoritos, catálogos, autorización de canales, documentación de la API y datos de demostración

4.2 Frontend: pruebas por área

Área	Archivos
Reservas	BookingResponseActions, ConfirmBookingButton, ServiceProgressCard, RescheduleBookingModal, BookingHistory, BookingList, InProgressPanel
Agenda	SlotPicker, WeeklyScheduleEditor, weekly, dates
Chat y tiempo real	BookingChat, presence
Notificaciones	NotificationBell
Perfil profesional	schemas, ServiceFormModal, EarningsView, ranges
Cuenta	ChangePasswordForm, DeleteAccountModal, ManualLocationForm, schemas, PasswordRecovery, RegisterProfessionalForm
Verificación y administración	AdminVerifications, ReplaceDocumentModal, schemas
Calificaciones y reportes	ReviewBookingModal, ReportUserModal, ProfessionalCard
Base	router (guards), cliente HTTP, formularios, formato, tokens, verificador de colores, favoritos, esquemas de auth

4.3 Pruebas funcionales sobre la aplicación

Además de las suites automatizadas, para la entrega final se ejecutó una batería de **44 pruebas funcionales de caja negra** sobre la aplicación en marcha: 28 positivas y 16 negativas, con Chrome para los flujos de pantalla. Las 44 se aprobaron el 23 de octubre de 2026. También se revisaron 29 pantallas en tres anchos (celular, tablet y escritorio). La matriz, los resultados y los hallazgos están en el informe de análisis funcional, y los scripts, en `_pruebas/`.

5. Cómo ejecutar las pruebas

```
# Backend
php artisan test
vendor/bin/pint --test

# Frontend
npm run lint
npm run type-check
npx vitest run
npm run build
```

6. Pendiente

- **Cobertura de código medida** (porcentaje de líneas): activar `--coverage` en los dos repositorios y publicar el resultado en este reporte.
- **Pruebas de extremo a extremo** de los recorridos principales con un navegador automatizado (Playwright).
- **Prueba de carga** para el requisito RNF-REN-01, sobre el servidor de producción.
- **Auditoría de accesibilidad** (Lighthouse) para RNF-USA-04.
- **Prueba automatizada del cierre por inactividad** (HU050) y de las vistas del administrador (usuarios, reportes y alertas), que hoy se prueban a mano.

Despliegue y operación

Cómo está puesta en producción Maestros a un clic, cómo se actualiza sola desde GitHub y cómo se opera. La topología está en 03-diagramas.md §8. Para el entorno de desarrollo, ver el README de cada repositorio.

Producción: https://maestrosaunclic.northcentralus.cloudapp.azure.com

1. Infraestructura

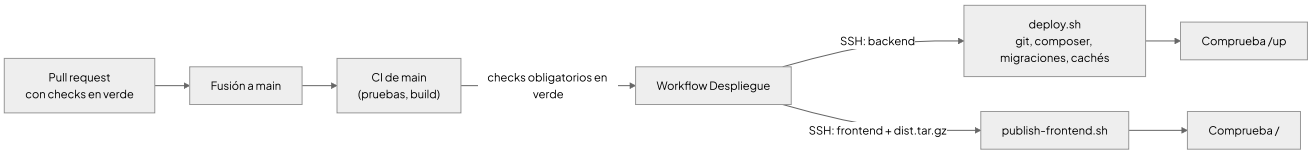
Elemento	Valor
Proveedor	Microsoft Azure, suscripción *Azure for Students* (crédito educativo, sin tarjeta)
Servidor	Una máquina virtual Linux `maestros-vm`, tamaño Standard_B2ats_v2 (2 vCPU, 1 GiB de RAM más 2 GB de intercambio), Ubuntu Server 24.04 LTS, en la región North Central US, grupo de recursos `maestros-vm_grupo`. El paso a paso para crearla está en la guía de despliegue (10)
Dominio	Nombre DNS gratuito de Azure: `maestrosaunclic.northcentralus.cloudapp.azure.com`
HTTPS	Certificado de Let's Encrypt, renovado automáticamente por Certbot
Puertos abiertos	22 (SSH con llave), 80 (redirige a HTTPS) y 443

Todo corre en la misma máquina, que es suficiente para el piloto y la demostración:

Componente	Para qué
Nginx	HTTPS, archivos del frontend, proxy de PHP y de los WebSockets
PHP-FPM 8.3	API Laravel
MySQL 8	Base de datos, ajustada a poca memoria
Supervisor	Mantiene vivos la cola (`queue:work`) y Reverb (`reverb:start`)
cron	`schedule:run` cada minuto

Un solo dominio. El frontend está en `/`; el API en `/api`; los archivos públicos en `/storage`; la salud en `/up`; y Reverb en `/app` (WebSockets del navegador) y `/apps` (Laravel publica los eventos). Como todo comparte origen, el navegador no hace peticiones entre dominios y CORS no interviene.

2. Despliegue continuo desde GitHub



Cada repositorio tiene el workflow **Despliegue** (`.github/workflows/deploy.yml`). Cuando el CI de `main` termina, revisa que hayan pasado los checks obligatorios y publica:

- **Backend:** el workflow entra por SSH y ejecuta `deploy/deploy.sh`. El script:
  1. pone la aplicación en mantenimiento;
  2. trae `main`;
  3. ejecuta `composer install --no-dev`, las migraciones y el cacheo de configuración, rutas y eventos;
  4. reinicia la cola, Reverb y PHP-FPM;
  5. levanta la aplicación, aunque algo haya fallado.
- **Frontend:** compila en GitHub con las variables de producción y envía `dist/` comprimido al servidor. `deploy/publish-frontend.sh` publica primero los archivos nuevos y al final `index.html`, así la aplicación nunca queda en blanco.

La auditoría de dependencias no bloquea el despliegue, igual que no bloquea los PRs. Un despliegue espera a que termine el anterior. Los dos workflows se pueden lanzar a mano desde *Actions → Despliegue → Run workflow*.

Seguridad del despliegue. GitHub usa una llave SSH propia (`DEPLOY_SSH_KEY`). En el servidor, esa llave tiene un **comando forzado** (`deploy/ssh-gate.sh`): solo puede pedir `backend` o `frontend` . No abre una terminal, así que si se filtrara, solo serviría para volver a desplegar `main` .

Configuración de cada repositorio (*Settings → Secrets and variables → Actions*):

Nombre	Tipo	Valor
`DEPLOY_SSH_KEY`	Secret	Llave privada de despliegue
`DEPLOY_KNOWN_HOSTS`	Secret	Huella del servidor (`ssh-keyscan <host>`)
`DEPLOY_HOST`	Variable	`maestrosaunclic.northcentralus.cloudapp.azure.com`
`DEPLOY_USER`	Variable	`maestros`
`VITE_REVERB_APP_KEY`	Variable (solo frontend)	La `REVERB_APP_KEY` del `.env` del servidor. Es pública: va en el JavaScript

Si `DEPLOY_HOST` no existe, el workflow se salta sin fallar.

3. Preparar un servidor nuevo

`deploy/provision.sh` deja lista una máquina Ubuntu 24.04 limpia en unos 10 minutos:

```
ssh maestros@<host>
sudo git clone https://github.com/LordCasta/maestros_a_un_clic_backend.git /var/www/maestros-backend
sudo DOMAIN=<host> EMAIL=<correo> \
     REVERB_APP_KEY=$(openssl rand -hex 16) REVERB_APP_SECRET=$(openssl rand -hex 32) \
     DEPLOY_PUBKEY="ssh-ed25519 AAAA… github-actions" SEED_DEMO=1 \
     bash /var/www/maestros-backend/deploy/provision.sh
```

Qué hace el script:

1. Crea 2 GB de intercambio y luego instala Nginx, PHP 8.3, MySQL, Supervisor, Certbot y Composer.
2. Ajusta MySQL y PHP-FPM a poca memoria y crea la base de datos `maestros` con un usuario propio y una contraseña aleatoria.
3. Genera el `.env` de producción (tabla siguiente).
4. Corre las migraciones y carga las comunas y las categorías.
5. Con `SEED_DEMO=1` , carga los datos de demostración y les pone una contraseña aleatoria. Con una URL pública, `password` sería una puerta abierta. El script imprime la contraseña al final.
6. Configura Nginx, el certificado HTTPS, Supervisor y cron.
7. Instala la llave de despliegue con su comando forzado.

`artisan` siempre corre como `www-data` , el usuario de PHP-FPM, para que los logs y la caché queden con un dueño que pueda escribirlos.

Variables del `.env` de producción:

Variable	Valor
`APP_ENV` / `APP_DEBUG`	`production` / `false`
`APP_URL` / `FRONTEND_URL`	`https://<host>` . El mismo dominio para las URLs de archivos y para el enlace de recuperación de contraseña
`APP_KEY`	Aleatoria, generada al preparar el servidor
`DB_*`	Base `maestros` y usuario `maestros` , con permisos solo sobre ella
`LOG_STACK` / `LOG_LEVEL`	`daily` / `warning`
`REVERB_APP_KEY` / `REVERB_APP_SECRET`	Propias de producción, no las de desarrollo
`REVERB_HOST` / `REVERB_PORT` / `REVERB_SCHEME`	`<host>` / `443` / `https` : lo que ve el navegador
`REVERB_SERVER_HOST` / `REVERB_SERVER_PORT`	`127.0.0.1` / `8080` : donde escucha Reverb, detrás de Nginx
`MAIL_MAILER`	`log` por ahora. Para que lleguen los correos de recuperación hay que configurar un SMTP (por ejemplo, Brevo tiene un plan gratuito)

4. Operación

Tarea	Cómo
Entrar al servidor	`ssh -i ~/.ssh/maestros_azure maestros@<host>`
Desplegar a mano	En el servidor: `bash /var/www/maestros-backend/deploy/deploy.sh`. Desde GitHub: *Actions → Despliegue → Run workflow*
Estado de la cola y Reverb	`sudo supervisorctl status`
Logs de la aplicación	`/var/www/maestros-backend/storage/logs/laravel-AAAA-MM-DD.log`
Logs de la cola y Reverb	`/var/log/maestros/queue.log` y `/var/log/maestros/reverb.log`
Trabajos fallidos	`sudo -u www-data php artisan queue:failed` y `queue:retry all`
Renovación del certificado	Automática (`certbot.timer`). Prueba: `sudo certbot renew --dry-run`
Detener la máquina para ahorrar crédito	Desde el portal de Azure, *Detener*. La IP y el nombre DNS se conservan. Al encenderla, los servicios arrancan solos

5. Respaldos y restauración

Qué	Cómo	Frecuencia	Retención
Base de datos	`mysqldump --single-transaction maestros > backup-$(date +%F).sql`, comprimido	Diario (cron)	7 días en el servidor y una copia fuera de él
Archivos	`storage/app/public` y `storage/app/private` (portafolio, avatares, documentos y adjuntos)	Diario	7 días

Restaurar: `mysql maestros < backup-AAAA-MM-DD.sql` y copiar de vuelta las carpetas de `storage/app`. Conviene probar una restauración al menos una vez antes del lanzamiento.

6. Monitoreo

- **Salud:** `GET https://<host>/up` responde 200 si la aplicación arranca. El despliegue lo comprueba al terminar. Para el uso continuo, conviene un monitor externo gratuito que lo consulte cada 5 minutos (RNF-DIS-01).
- **Errores:** logs diarios de Laravel (sección 4).
- **Tiempo real:** si las notificaciones no llegan, revisa `supervisorctl status` y el log de Reverb.

7. Lista de verificación antes de abrir a usuarios reales

- [x] `APP_DEBUG=false` y `APP_ENV=production`.
- [x] HTTPS activo; WebSockets por `wss`.
- [x] Frontend y API en el mismo origen (CORS no interviene).
- [x] Claves de Reverb y de la aplicación propias, no las de desarrollo.
- [x] Usuario de MySQL con permisos solo sobre su base.
- [x] Contraseñas de las cuentas demo cambiadas por una aleatoria.
- [ ] Respaldos programados y una restauración probada.
- [ ] Monitor externo de `/up`.
- [ ] SMTP para los correos de recuperación de contraseña.
- [ ] Vencimiento de los tokens (`sanctum.expiration`) y límite del chat (E-22 del informe 09).
- [ ] Política de tratamiento de datos publicada y aceptada en el registro (RNF-PRI-03).
- [ ] Datos de maqueta retirados de los paneles (E-11).

Seguridad y privacidad

Controles de seguridad implementados, cumplimiento del tratamiento de datos personales (Ley 1581 de 2012) y acciones pendientes. Se toma como referencia el **OWASP Top 10 (2021)**.

1. Datos que trata el sistema

Dato	Titular	Finalidad	Dónde se guarda	Quién lo ve
Nombre, correo, teléfono	Cliente y profesional	Cuenta y contacto	`users`	El propio usuario. El nombre lo ven además las contrapartes y, en el caso del profesional, el público
Comuna, dirección, coordenadas	Cliente y profesional	Búsqueda por cercanía y dirección del servicio	`users` , `bookings`	El propio usuario. La dirección de una reserva, solo sus partes
Selfie, documento de identidad, certificados	Cliente y profesional	Verificación de identidad (KYC)	Disco **privado**	El propio usuario y los administradores, con URL firmada temporal
Fotos del portafolio y foto de perfil	Profesional / usuario	Mostrar su trabajo y su identidad	Disco público	Cualquiera (es su finalidad)
Mensajes e imágenes del chat	Partes de la reserva	Coordinar el servicio	`messages` y disco **privado**	Solo las dos partes de la reserva
Calificaciones y comentarios	Cliente y profesional	Reputación	`reviews`	Público, en el perfil
Historial de reservas	Partes	Trazabilidad e ingresos	`bookings` , `booking_status_changes`	Las dos partes y los administradores

2. Controles frente al OWASP Top 10

Riesgo	Control en el sistema
A01 Control de acceso roto	Middleware de rol y Policies por recurso en cada acción (dueño, partes de la reserva); el perfil de un cliente solo lo ve el profesional con quien tiene una reserva; los usuarios bloqueados no acceden. Pruebas de 401 y 403 por endpoint
A02 Fallas criptográficas	Contraseñas con bcrypt; HTTPS y `wss` en producción; URLs firmadas con la clave de la app para los archivos privados
A03 Inyección	Consultas con el ORM y parámetros enlazados; validación de toda entrada con FormRequest; Vue escapa el contenido al mostrarlo (el chat no interpreta HTML)
A04 Diseño inseguro	Reglas de negocio centralizadas en servicios: máquina de estados, bloqueo de la agenda en una transacción para evitar dobles reservas, chat cerrado al terminar la reserva
A05 Configuración insegura	`APP_DEBUG=false` en producción, errores con formato estándar sin trazas, lista de verificación de despliegue. **Pendiente:** limitar CORS al dominio del frontend
A06 Componentes vulnerables	Auditoría de dependencias en cada pull request
A07 Fallas de autenticación	Tokens de Sanctum (se revocan al cerrar sesión, al restablecer la contraseña, al bloquear la cuenta y al eliminarla), máximo 10 intentos por minuto en login, registro y recuperación, contraseña de 8 caracteres o más, recuperación que no revela qué correos existen y cierre de sesión tras 15 minutos de inactividad (HU050). **Pendiente:** vencimiento de los tokens en el servidor (`sanctum.expiration`)
A08 Integridad de datos	CI obligatoria y check Guardas; las migraciones existentes no se editan; historial de estados auditable
A09 Registro y monitoreo	Log de la app con rotación diaria, historial de cambios de estado con autor y fecha, monitoreo de salud (`/up`)
A10 SSRF	El servidor no hace peticiones a URLs que entregue el usuario

3. Cumplimiento de la Ley 1581 de 2012

Principio o derecho	Cómo se atiende	Estado
Finalidad	Cada dato tiene una finalidad declarada (tabla de la sección 1)	✅

Principio o derecho	Cómo se atiende	Estado
Seguridad	Controles de la sección 2; documentos y adjuntos en almacenamiento privado	✅
Acceso y circulación restringida	El perfil público no muestra datos de contacto; el chat solo lo ven las partes	✅
Libertad (autorización previa)	Casilla obligatoria de aceptación de la *Política de tratamiento de datos* en el registro, con la fecha y la versión aceptadas	⚠️ Pendiente
Transparencia	Publicar la política (finalidades, derechos, canal de contacto del responsable)	⚠️ Pendiente
Consulta y rectificación	El usuario ve y edita sus datos, ubicación y foto desde su cuenta (HU009, HU022, HU031, HU032)	✅
Supresión	Eliminación de cuenta con anonimización: se borran datos de contacto, ubicación, fotos, documentos de verificación, favoritos y sesiones, y se conservan las reservas, reseñas y chats de la otra parte con el nombre "Usuario eliminado" (HU023, D12)	✅

4. Acciones pendientes

#	Acción	Responsable sugerido	Prioridad
1	Política de tratamiento de datos y aceptación obligatoria en el registro (campo `privacy_accepted_at` en una migración nueva)	Módulo cuenta	Alta
2	Limitar CORS al dominio del frontend en producción	Despliegue	Alta
3	Vencimiento de los tokens en el servidor (`sanctum.expiration`); el cierre por inactividad del navegador ya está (HU050)	Módulo cuenta	Media
4	Límite de frecuencia en el envío de mensajes del chat	Chat	Baja

Decisiones de arquitectura

Registro de las decisiones técnicas importantes (*Architecture Decision Records*): qué se decidió, por qué y qué consecuencias tiene. Las decisiones del modelo de datos (D1–D15) y las de negocio (N1–N3) están detalladas en docs/modelo-de-datos.md §1 y §6; aquí se resumen junto con las de arquitectura.

ADR-01 · Dos repositorios: backend y frontend

- **Decisión:** el backend (Laravel) y el frontend (Vue) viven en repositorios separados, con su propia CI, comunicados solo por la API.
- **Motivo:** responsabilidades claras, despliegues independientes y una forma de organizarse que el equipo considera más ordenada.
- **Consecuencia:** el contrato de la API (`docs/api/endpoints.md` y OpenAPI) es la frontera entre los dos, y un cambio que toca los dos lados se hace con un PR en cada repositorio: primero el backend.

ADR-02 · API REST versionada con un formato de respuesta único

- **Decisión:** API bajo `/api/v1`; toda respuesta tiene la forma `{ success, message, data, meta? }` y todo error, `{ success: false, message, errors? }`.
- **Motivo:** el frontend maneja todas las respuestas igual; una versión futura (`/v2`) no rompe a los clientes existentes.
- **Consecuencia:** los controladores solo responden con los helpers del controlador base, y los errores se formatean en un único lugar (`bootstrap/app.php`).

ADR-03 · Autenticación con tokens (Laravel Sanctum)

- **Decisión:** token Bearer por sesión, que se revoca al cerrar sesión.
- **Motivo:** la SPA y la API están en dominios distintos; los tokens evitan la complejidad de las cookies entre dominios y sirven también para autenticar los canales de WebSockets.

ADR-04 · Arquitectura por capas en el backend, sin patrón Repository

- **Decisión:** Ruta (middleware de rol) → FormRequest (validación) → Controller → Policy (autorización por recurso) → Service (reglas, transacciones, archivos) → Resource (forma de la respuesta). Las consultas reutilizables son *scopes* del modelo.
- **Motivo:** cada capa tiene una sola responsabilidad y las reglas de negocio quedan en un lugar fácil de probar. Un Repository sobre Eloquent agregaría una capa sin beneficio en este tamaño de proyecto.

ADR-05 · Frontend por módulos con API pública

- **Decisión:** `src/modules/<módulo>` autocontenidos (vistas, componentes, API, queries, rutas), que se usan entre sí solo por su `index.ts`; `src/shared` para lo común y `src/app` para el armado.
- **Motivo:** permite que dos personas trabajen en paralelo por módulos sin pisarse. ESLint bloquea los imports a archivos internos de otro módulo.

ADR-06 · Estado del servidor con Pinia Colada

- **Decisión:** los datos de la API se manejan como consultas con caché (Pinia Colada); Pinia solo guarda la sesión y los avisos.
- **Motivo:** evita duplicar el estado del servidor en *stores*. Invalidar una clave refresca todas las pantallas que la usan, que es lo que aprovechan los eventos de tiempo real.

ADR-07 · Tiempo real con Laravel Reverb (decisión N1)

- **Decisión:** WebSockets propios con Reverb: un canal privado por usuario para las notificaciones, uno de presencia por reserva para el chat, el estado y quién está conectado, y uno público por profesional para su agenda.
- **Motivo:** chat, estado en línea y agenda en vivo sin depender de un servicio de pago. Es compatible con el protocolo de Pusher, así que se usa Laravel Echo en el cliente.
- **Consecuencia:** hacen falta dos procesos más en producción: la cola y Reverb.

ADR-08 · Disponibilidad calculada, no guardada (decisión D7)

- **Decisión:** se guardan el horario semanal y los bloqueos; las horas libres se calculan al consultarlas, descontando las reservas activas y el descanso.
- **Motivo:** evita mantener miles de "cupos" sincronizados. Una reserva nueva valida los cruces dentro de una transacción que bloquea la agenda del profesional, para que dos clientes no tomen la misma hora.

ADR-09 · Máquina de estados y auditoría de la reserva (decisión D8)

- **Decisión:** los siete estados y sus transiciones válidas están en el enum `BookingStatus`; todo cambio pasa por un único método que valida, registra el historial y emite los eventos.
- **Motivo:** un solo lugar garantiza que nunca haya transiciones inválidas y que todo cambio quede auditado.

ADR-10 · Archivos privados con URL firmada (decisión D9)

- **Decisión:** los documentos de identidad y los adjuntos del chat van a un disco privado y se entregan con URLs firmadas que vencen; el portafolio y los avatares son públicos.
- **Motivo:** Ley 1581 de 2012. Nadie puede ver una cédula o una foto del chat con solo adivinar la URL.

ADR-11 · Sistema de diseño con tokens obligatorios

- **Decisión:** la paleta, la tipografía y las sombras se definen como tokens de Tailwind; un verificador en la CI rechaza los colores escritos a mano.
- **Motivo:** una identidad visual consistente aunque la construyan dos personas y asistentes de IA.

ADR-12 · Calidad protegida (CI obligatoria y check Guardas)

- **Decisión:** `main` está protegida. Solo se fusiona por PR, con todas las verificaciones en verde. El check Guardas falla si un PR modifica las reglas de calidad (lint, CI, configuración de pruebas, instrucciones para agentes) o una migración existente, salvo que una persona lo apruebe con la etiqueta `cambio-de-reglas`.
- **Motivo:** el equipo trabaja con asistentes de IA, que tienden a "arreglar" un fallo relajando la regla en lugar del código. Así eso queda bloqueado.

ADR-13 · Precio y duración por servicio (decisiones D6 y N3)

- **Decisión:** el profesional publica servicios con precio (fijo o por hora) y duración estimada. Al reservar se copia el precio a la reserva.
- **Motivo:** la duración define cuánto ocupa la reserva en la agenda, y un cambio de precio posterior no altera las reservas ya hechas.

ADR-14 · Verificación obligatoria para reservar (decisión N2)

- **Decisión:** cualquiera puede registrarse y explorar, pero solo un cliente con la identidad verificada puede crear reservas.
- **Motivo:** proteger a los profesionales, que van a la casa del cliente, sin frenar el registro.

Informe de análisis funcional del software

Este informe describe qué hace **Maestros a un clic**, cómo está construido y cómo se comporta hoy, con evidencia. Se elaboró el 23 de octubre de 2026 sobre la versión de `main` de ese día:

- backend `077976b`;
- frontend `0f6be1e`.

La validación se hizo sobre la aplicación en ejecución en un entorno local: Windows 11, PHP 8.5, MariaDB 10.4, Node.js 24 y Chrome.

El informe resume y enlaza los documentos de detalle:

- requisitos;
- diagramas;
- trazabilidad;
- pruebas;
- seguridad.

Los scripts que ejecutaron las pruebas funcionales y tomaron las capturas están en `_pruebas/`, para que la validación se pueda repetir.

1. Descripción y estructura funcional del sistema

1.1 Información general del proyecto

Campo	Valor
Nombre	Maestros a un clic
Tipo de software	Aplicación web (marketplace de servicios del hogar): API REST + aplicación de una sola página
Institución	Institución Universitaria Pascual Bravo
Programa y curso	Tecnología en Desarrollo de Software, sexto semestre · Ingeniería de Software 2
Docente	Maira Perez
Integrantes	Juan Pablo Castaño (LordCasta) y Brayan Arenas (barenas12)
Repositorios	Backend (Laravel) · Frontend (Vue)
Ámbito geográfico	Medellín: 16 comunas y 5 corregimientos
Estado	Las 49 historias de usuario implementadas; en evaluación funcional para la entrega final

1.2 Problema o necesidad que resuelve

En Medellín, conseguir un plomero, electricista o carpintero confiable depende del voz a voz, de grupos de redes sociales sin ninguna validación o de empresas que cobran más y no dejan elegir quién hace el trabajo. Coordinar la cita por mensajes es lento y produce cruces de horario. El cliente no sabe a quién deja entrar a su casa, y el profesional no sabe a qué cliente va a atender.

El sistema responde a esa necesidad con cuatro mecanismos:

- **Verificación de identidad:** documentos revisados por un administrador.
- **Calificación recíproca:** el cliente califica al profesional y el profesional al cliente.
- **Agenda real:** no se pueden reservar dos servicios a la misma hora.
- **Historial auditable** de cada reserva.

1.3 Objetivo y alcance del software

Objetivo general. Conectar a clientes de Medellín con profesionales del hogar verificados mediante una plataforma web que permita buscarlos, reservar una hora libre de su agenda, comunicarse durante el servicio y calificarse mutuamente, de forma transparente y sin intermediarios.

Alcance implementado:

- Registro e inicio de sesión de clientes y profesionales, recuperación de contraseña y cierre de sesión por inactividad.
- Gestión de la cuenta: datos, foto, ubicación manual, contraseña y eliminación con anonimización.

- Perfil profesional con especialidades, tarifa, servicios con precio y duración, y portafolio.
- Agenda con horario semanal, descanso entre servicios, bloqueos y cálculo de horas libres en tiempo real.
- Búsqueda por nombre, categoría, precio, cercanía, comunas aledañas y "disponible ahora", con mapa.
- Ciclo completo de la reserva: solicitar, aceptar o rechazar, confirmar, reprogramar, cancelar, iniciar y finalizar, con historial.
- Chat por reserva con imágenes, estado en línea y eliminación de mensajes; notificaciones en tiempo real.
- Verificación de identidad (KYC) y panel de administración: verificaciones, usuarios, reportes y alertas.
- Calificaciones recíprocas, comentarios públicos, reportes de usuarios y alertas por baja calificación.
- Favoritos, historiales, totales e ingresos del profesional.

Fuera del alcance:

- **Pagos en línea:** el precio se pacta en la plataforma, pero el pago se hace por fuera.
- **Consulta automática de antecedentes con un proveedor externo:** no existe una API pública para integrar (decisión D10).
- **Aplicaciones móviles nativas:** la aplicación web es *responsive*.

1.4 Usuarios y roles

Rol	Quién es	Qué puede hacer
Visitante	Persona sin sesión	Ver la página de inicio, buscar profesionales, ver perfiles, servicios y comentarios públicos, y registrarse
Cliente	Persona que necesita un servicio	Todo lo del visitante, más: verificar su identidad, reservar (solo si está verificado), confirmar, reprogramar o cancelar, chatear, calificar, reportar, guardar favoritos y gestionar su cuenta
Profesional	Prestador de servicios del hogar	Publicar su perfil, servicios y portafolio; definir su agenda; aceptar o rechazar solicitudes; iniciar y finalizar servicios; chatear; ver el perfil del cliente; calificarlo; ver sus ingresos y verificarse. Solo aparece en las búsquedas cuando está verificado
Administrador	Equipo de la plataforma	Revisar y aprobar o rechazar verificaciones por documento, bloquear y desbloquear usuarios, atender reportes y resolver alertas. No ve los chats
Sistema	Procesos automáticos	Calcular la disponibilidad, enviar notificaciones y eventos en tiempo real, y generar alertas por reportes y bajas calificaciones

Permisos por rol sobre las acciones principales (✅ permitido · ❌ denegado):

Acción	Visitante	Cliente	Profesional	Administrador
Buscar y ver perfiles	✅	✅	✅	✅
Solicitar una reserva	❌	✅ verificado	❌	❌
Responder solicitudes, iniciar y finalizar	❌	❌	✅ el suyo	❌
Chat de una reserva	❌	✅ participante	✅ participante	❌
Ver el perfil de un cliente	❌	❌	✅ si tiene una reserva con él	❌
Panel de administración	❌	❌	❌	✅

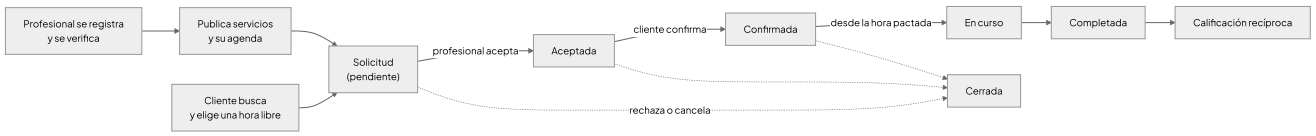
1.5 Módulos y funcionalidades

El sistema se organiza en los mismos módulos en el backend (controladores, servicios y políticas) y en el frontend (`src/modules`). Cada uno tuvo un responsable.

Módulo	Funcionalidades	Historias	Responsable
Autenticación	Registro de cliente y profesional (en dos pasos), inicio y cierre de sesión, validación de datos	HU001, HU006, HU007, HU021, HU048	Equipo (fase 0)
Cuenta	Datos y foto, ubicación manual, cambio y recuperación de contraseña, eliminación de cuenta, cierre por inactividad	HU019, HU020, HU022, HU023, HU031, HU032, HU050	barenas12
Perfil profesional	Descripción, especialidades, tarifa, servicios con precio y duración, portafolio, ingresos	HU009, HU037, HU047	LordCasta
Agenda	Horario semanal, descanso, bloqueos, horas libres en tiempo real	HU003, HU025	LordCasta
Búsqueda	Nombre, categoría y subcategoría, precio, orden, cercanía, comunas aledañas, disponible ahora, mapa	HU002, HU010, HU030, HU033, HU039, HU040, HU046	barenas12

Módulo	Funcionalidades	Historias	Responsable
Reservas	Solicitar, aceptar o rechazar, confirmar, reprogramar, cancelar, iniciar, finalizar, historial, totales, servicios en curso	HU004, HU012, HU016, HU017, HU026, HU027, HU028, HU036, HU049	LordCasta
Chat y notificaciones	Chat por reserva con imágenes, estado en línea, eliminar mensajes, centro de notificaciones	HU011, HU018, HU041, HU042, HU043	LordCasta
Verificación y administración	Cargue de documentos, revisión por documento, aprobación o rechazo, bloqueo de usuarios	HU008, HU013, HU014, HU038	barenas12
Calificaciones y reportes	Calificación recíproca, comentarios, perfil del cliente, reportes, alertas	HU005, HU024, HU029, HU044, HU045	barenas12
Favoritos	Guardar y ver profesionales favoritos	HU034, HU035	Equipo (fase 0)

Flujo principal del sistema:



1.6 Inventario de funcionalidades y estado de desarrollo

Las 49 historias de usuario están implementadas y sus issues cerrados en GitHub (la numeración del documento original no tiene HU015). Al revisar criterio por criterio para este informe se encontraron **5 historias con un criterio de aceptación incompleto**. Se marcan como *Parcial* y se detallan en la sección 3.3.

HU	Funcionalidad	Módulo	Estado
HU001	Inicio de sesión	Autenticación	✅ Completa
HU002	Búsqueda por geolocalización	Búsqueda	✅ Completa
HU003	Gestión de disponibilidad y agenda	Agenda	✅ Completa
HU004	Solicitud y aceptación de servicio	Reservas	✅ Completa
HU005	Calificación recíproca	Calificaciones	✅ Completa
HU006	Registro de cliente	Autenticación	✅ Completa
HU007	Registro de profesional	Autenticación	✅ Completa
HU008	Cargue de documentos	Verificación	✅ Completa
HU009	Edición de perfil profesional	Perfil profesional	✅ Completa
HU010	Filtros y ordenamiento	Búsqueda	✅ Completa
HU011	Chat de la reserva	Chat	✅ Completa
HU012	Cancelación y reprogramación	Reservas	✅ Completa
HU013	Validación de identidad y antecedentes	Verificación	⚠️ Parcial: la comparación selfie-documento la hace el administrador, no un proveedor automático (decisión D10)
HU014	Revisión y aprobación de documentos	Administración	✅ Completa
HU016	Historial del cliente	Reservas	✅ Completa
HU017	Historial del profesional	Reservas	✅ Completa
HU018	Notificaciones	Notificaciones	✅ Completa
HU019	Cambio de contraseña	Cuenta	✅ Completa
HU020	Recuperación de contraseña	Cuenta	✅ Completa

HU	Funcionalidad	Módulo	Estado
HU021	Cerrar sesión	Autenticación	✅ Completa
HU022	Ubicación manual	Cuenta	✅ Completa
HU023	Eliminación de cuenta	Cuenta	✅ Completa
HU024	Reporte de usuarios	Calificaciones	✅ Completa
HU025	Disponibilidad del profesional	Agenda	✅ Completa
HU026	Estado del servicio	Reservas	✅ Completa
HU027	Inicio de servicio	Reservas	✅ Completa
HU028	Finalización de servicio	Reservas	✅ Completa
HU029	Visualización de comentarios	Calificaciones	✅ Completa
HU030	Disponible ahora	Búsqueda	⚠️ Parcial: el resultado se calcula al buscar, pero la lista no se refresca sola mientras está abierta
HU031	Edición de perfil del cliente	Cuenta	✅ Completa
HU032	Foto de perfil	Cuenta	✅ Completa
HU033	Servicios por categoría	Búsqueda	✅ Completa
HU034	Guardar favoritos	Favoritos	✅ Completa
HU035	Ver favoritos	Favoritos	✅ Completa
HU036	Confirmación por el cliente	Reservas	✅ Completa
HU037	Ingresos del profesional	Perfil profesional	✅ Completa
HU038	Bloqueo de usuarios	Administración	✅ Completa
HU039	Búsqueda por nombre	Búsqueda	✅ Completa
HU040	Visualización de distancia	Búsqueda	⚠️ Parcial: la distancia aparece en el listado, pero no en el perfil del profesional
HU041	Estado en línea	Chat	✅ Completa
HU042	Adjuntos en el chat	Chat	✅ Completa
HU043	Eliminar mensajes	Chat	✅ Completa
HU044	Perfil del cliente	Calificaciones	⚠️ Parcial: muestra nombre, foto, calificación y servicios, pero no los comentarios que recibió
HU045	Alerta por baja calificación	Calificaciones	⚠️ Parcial: la alerta identifica al usuario, pero no enlaza a su perfil para revisarlo
HU046	Ordenamiento por precio	Búsqueda	✅ Completa
HU047	Tiempo estimado	Perfil profesional	✅ Completa
HU048	Validación de datos obligatorios	Autenticación	✅ Completa
HU049	Servicios en proceso	Reservas	✅ Completa
HU050	Cierre por inactividad	Cuenta	✅ Completa

Resumen: 44 completas (90 %) y 5 parciales (10 %). Ninguna historia quedó sin implementar.

1.7 Reglas de negocio

ID	Regla	Dónde se hace cumplir
RN-01	Una cuenta tiene un solo rol: cliente, profesional o administrador	`users.role` y middleware `role:` en las rutas
RN-02	El cliente debe tener la identidad verificada para reservar (decisión N2)	`BookingPolicy::create` (403)
RN-03	Solo los profesionales verificados y no bloqueados aparecen en la búsqueda y tienen perfil público	`User::publiclyListed()`

ID	Regla	Dónde se hace cumplir
RN-04	Una reserva solo puede caer dentro del horario semanal del profesional y fuera de sus bloqueos	`AvailabilityService::withinSchedule` (422)
RN-05	Dos reservas activas del mismo profesional no se pueden cruzar, contando el descanso configurado antes y después de cada una	`AvailabilityService::hasBookingConflict` y bloqueo de fila en la transacción (409)
RN-06	El precio pactado es una copia del precio del servicio al reservar; si el profesional lo cambia después, la reserva no cambia (N3)	`bookings.agreed_price`
RN-07	Los estados siguen una máquina de estados: pendiente → aceptada → confirmada → en curso → completada; rechazar o cancelar cierra la reserva	`BookingStatus::allowedTransitions` (409 si la transición no es válida)
RN-08	El servicio solo se puede iniciar desde la hora pactada	`BookingService::start` (409)
RN-09	Reprogramar devuelve la reserva a *pendiente* para que el profesional vuelva a aceptar; cancelar exige un motivo	`BookingService` y `CancelBookingRequest`
RN-10	Cada cambio de estado queda registrado con autor, fecha y motivo	`booking_status_changes` (D8)
RN-11	No se puede bloquear un horario que tiene una reserva activa	`AvailabilityService::addBlock` (409)
RN-12	El chat solo lo ven los dos participantes de la reserva y se cierra cuando la reserva termina	`MessagePolicy` y `ChatService`
RN-13	Solo se califica una reserva completada, una vez por cada parte; el promedio se actualiza solo	`ReviewService` (409)
RN-14	Una calificación de 2 o menos, o un promedio menor a 3 con 3 o más calificaciones, genera una alerta para el administrador; un reporte también	`ReviewService` , `ModerationService`
RN-15	Un reporte hecho desde una reserva solo puede venir de uno de sus participantes y apuntar al otro	`StoreReportRequest` (422)
RN-16	El perfil de un cliente solo lo ve un profesional que tiene o tuvo una reserva con él	`UserPolicy::viewClientProfile` (403)
RN-17	Un usuario bloqueado no puede entrar ni usar la API; un administrador no puede ser bloqueado	Middleware `not_blocked` , `ModerationService`
RN-18	No se puede eliminar la cuenta con reservas activas o pendientes; al eliminarla se anonimiza y la otra parte conserva su historial (D12)	`AccountService::delete`
RN-19	Los documentos de identidad y los adjuntos del chat son privados: solo se entregan con un enlace firmado que vence	Disco privado y URL firmada (D9)

2. Documentación técnica y funcionamiento del software

2.1 Matriz de requerimientos funcionales

Requerimientos funcionales agrupados a partir de las historias de usuario.

Prioridad:

- **Alta:** sin el requerimiento no existe el flujo principal.
- **Media:** mejora la confianza o la operación.
- **Baja:** comodidad.

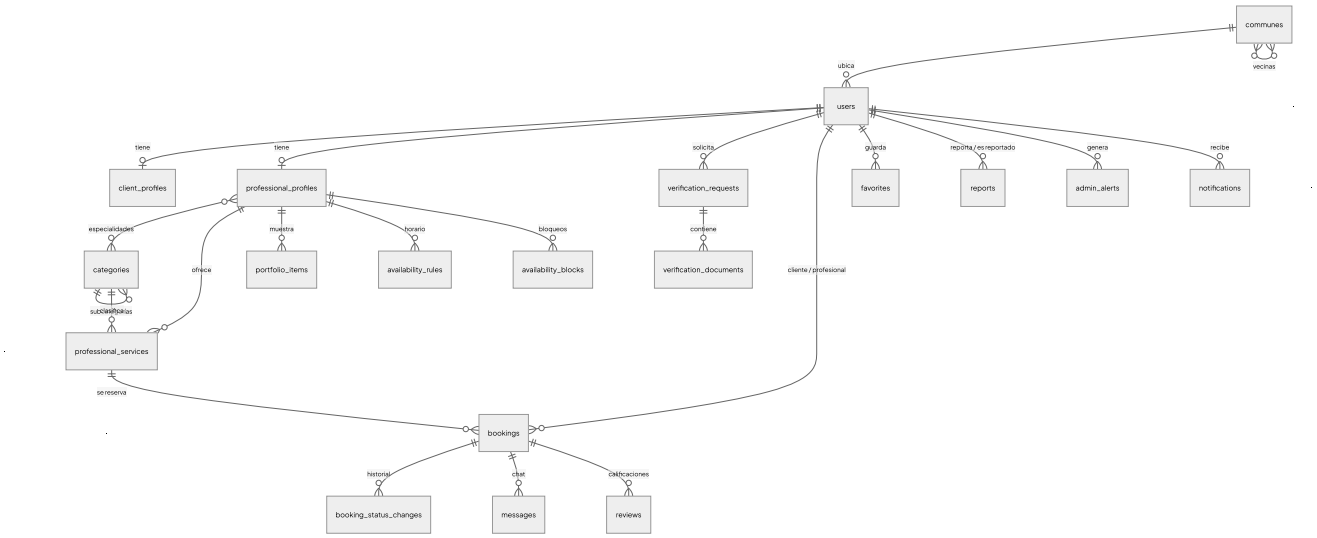
La relación detallada de cada historia con sus endpoints, pantallas y pruebas está en la matriz de trazabilidad.

ID	Requerimiento	Actor	Historias	Prioridad	Estado
RF-01	Registrar clientes y profesionales con validación de datos	Visitante	HU006, HU007, HU048	Alta	✅
RF-02	Autenticar, cerrar sesión y cerrar por inactividad	Usuario	HU001, HU021, HU050	Alta	✅
RF-03	Gestionar la cuenta: datos, foto, ubicación, contraseña, recuperación y eliminación	Usuario	HU019, HU020, HU022, HU023, HU031, HU032	Media	✅
RF-04	Gestionar el perfil profesional, sus servicios con precio y duración, y su portafolio	Profesional	HU009, HU047	Alta	✅

ID	Requerimiento	Actor	Historias	Prioridad	Estado
RF-05	Definir el horario semanal, el descanso y los bloqueos, y calcular las horas libres	Profesional, Sistema	HU003, HU025	Alta	✅
RF-06	Buscar profesionales por nombre, categoría, precio, cercanía y disponibilidad, con orden y sugerencias	Visitante, Cliente	HU002, HU010, HU030, HU033, HU039, HU040, HU046	Alta	⚠️ HU030 y HU040 parciales
RF-07	Solicitar, aceptar o rechazar y confirmar una reserva sin cruces de horario	Cliente, Profesional	HU004, HU036	Alta	✅
RF-08	Reprogramar, cancelar, iniciar y finalizar una reserva, con historial auditable	Cliente, Profesional	HU012, HU026, HU027, HU028	Alta	✅
RF-09	Consultar historiales, totales, servicios en curso e ingresos	Cliente, Profesional	HU016, HU017, HU037, HU049	Media	✅
RF-10	Comunicarse por chat con imágenes, estado en línea y eliminación de mensajes	Cliente, Profesional	HU011, HU041, HU042, HU043	Media	✅
RF-11	Notificar en tiempo real los eventos de la reserva y del chat	Sistema	HU018	Media	✅
RF-12	Verificar la identidad con documentos revisados por un administrador	Usuario, Administrador	HU008, HU013, HU014	Alta	⚠️ HU013 parcial
RF-13	Calificar recíprocamente y mostrar comentarios	Cliente, Profesional	HU005, HU029	Media	✅
RF-14	Ver el perfil del cliente antes de atenderlo	Profesional	HU044	Media	⚠️ HU044 parcial
RF-15	Reportar usuarios, generar alertas y moderar (bloquear y desbloquear)	Usuario, Administrador, Sistema	HU024, HU038, HU045	Media	⚠️ HU045 parcial
RF-16	Guardar y ver profesionales favoritos	Cliente	HU034, HU035	Baja	✅

2.2 Modelo de datos y diagrama entidad-relación

La base de datos es relacional (MySQL 8 en producción; en el entorno local se usó MariaDB 10.4, compatible). Tiene **21 tablas de dominio** más las de Laravel (caché, colas, sesiones y tokens), con llaves foráneas en todas las relaciones. Los estados, roles y tipos se guardan como texto y se modelan con *enums* de PHP (decisión D15).



Entidad	Qué guarda
`users`	Cuenta de cualquier rol: datos de contacto, comuna y coordenadas, estado de verificación, promedio de calificación y bloqueo. Se elimina con *soft delete* y anonimización (D12)
`client_profiles` / `professional_profiles`	Datos propios de cada rol: descripción, experiencia, tarifa, radio de servicio y descanso entre servicios del profesional
`communes` , `commune_neighbors`	Las 21 comunas y corregimientos de Medellín con su centroide y sus vecinas (para sugerir comunas aledañas)

Entidad	Qué guarda
`categories`, `professional_category`	Especialidades y subcategorías jerárquicas, y las especialidades de cada profesional
`professional_services`	Servicios que ofrece el profesional, con precio (por hora o fijo) y duración estimada
`portfolio_items`	Fotos de trabajos anteriores
`availability_rules`, `availability_blocks`	Horario semanal y horas bloqueadas. Las horas libres se calculan, no se guardan (D7)
`verification_requests`, `verification_documents`	Solicitudes de verificación y sus documentos (cédula, selfie, certificados, evidencias) en almacenamiento privado
`bookings`, `booking_status_changes`	Reservas con su precio pactado y el historial de cada cambio de estado
`messages`	Chat de cada reserva, con adjuntos privados
`reviews`	Calificaciones de 1 a 5 y comentarios, en los dos sentidos
`favorites`, `reports`, `admin_alerts`, `notifications`	Favoritos, denuncias, alertas para el administrador y notificaciones de cada usuario

El diccionario completo de columnas está en `docs/modelo-de-datos.md`, y el diagrama de clases y los diagramas de estados, en 03-diagramas.md.

2.3 Interfaces del sistema

Interfaz de usuario. Es una aplicación web de una sola página (Vue 3) con un sistema de diseño propio: colores, tipografía y componentes base. Se adapta a celular, tablet y escritorio. Tiene **29 pantallas**:

Rol	Pantallas
Público (7)	Inicio, búsqueda con filtros y mapa, perfil del profesional, inicio de sesión, registro de cliente, registro de profesional en dos pasos, recuperar y restablecer contraseña
Cliente (9)	Panel de inicio, mis reservas con filtros y totales, detalle de la reserva (datos, chat, historial y acciones), nueva reserva con selector de horas, favoritos, perfil, ubicación, seguridad (contraseña y eliminar cuenta) y verificación
Profesional (9)	Panel con solicitudes pendientes y servicios en curso, reservas recibidas, detalle de la reserva con perfil del cliente, disponibilidad, servicios, sobre mí (perfil, especialidades y portafolio), ingresos, cuenta y verificación
Administrador (4)	Verificaciones (con revisión por documento), usuarios, reportes y alertas

Todas las pantallas comparten la barra superior con la campana de notificaciones. Los formularios validan en el navegador antes de enviar y muestran junto a cada campo los errores que devuelve el servidor.

Interfaz de programación (API REST). Es la que usa el frontend y la que podría usar una aplicación móvil:

- **Rutas:** 74, bajo `/api/v1`.
- **Autenticación:** token Bearer (Laravel Sanctum).
- **Formato de respuesta:** JSON uniforme `{ success, message, data }`, con `meta` en los listados paginados.
- **Errores:** también uniformes. 422 con el error de cada campo, 401, 403, 404, 409 (regla de negocio) y 429 (demasiados intentos).

El contrato está en `docs/api/endpoints.md` y en la especificación OpenAPI 3.1. En local también hay documentación interactiva en `/docs/api`.

Interfaz de tiempo real. Usa WebSockets con Laravel Reverb y tres tipos de canal:

- **Privado del usuario:** notificaciones.
- **De presencia por reserva:** mensajes, mensajes eliminados, cambios de estado y quién está en línea.
- **Público de la agenda de cada profesional:** las horas libres se actualizan solas cuando alguien reserva. Ver `docs/tiempo-real.md`.

Interfaces externas:

- **Mapa:** Google Maps embebido, sin clave de API.
- **Correo electrónico:** enlace de recuperación de contraseña. En local queda en el log.
- **Almacenamiento de archivos:** un disco público para fotos y portafolio, y uno privado para documentos y adjuntos.

2.4 Capturas de pantalla de las funcionalidades implementadas

Las capturas se tomaron automáticamente con Chrome sobre la aplicación en local, con las cuentas de demostración, y reflejan su estado real. El script es `_pruebas/capturas.mjs`. Las de escritorio miden 1366 px de ancho y las de celular, 390 px.

Visitante



Figura 1. Página de inicio con el buscador principal.

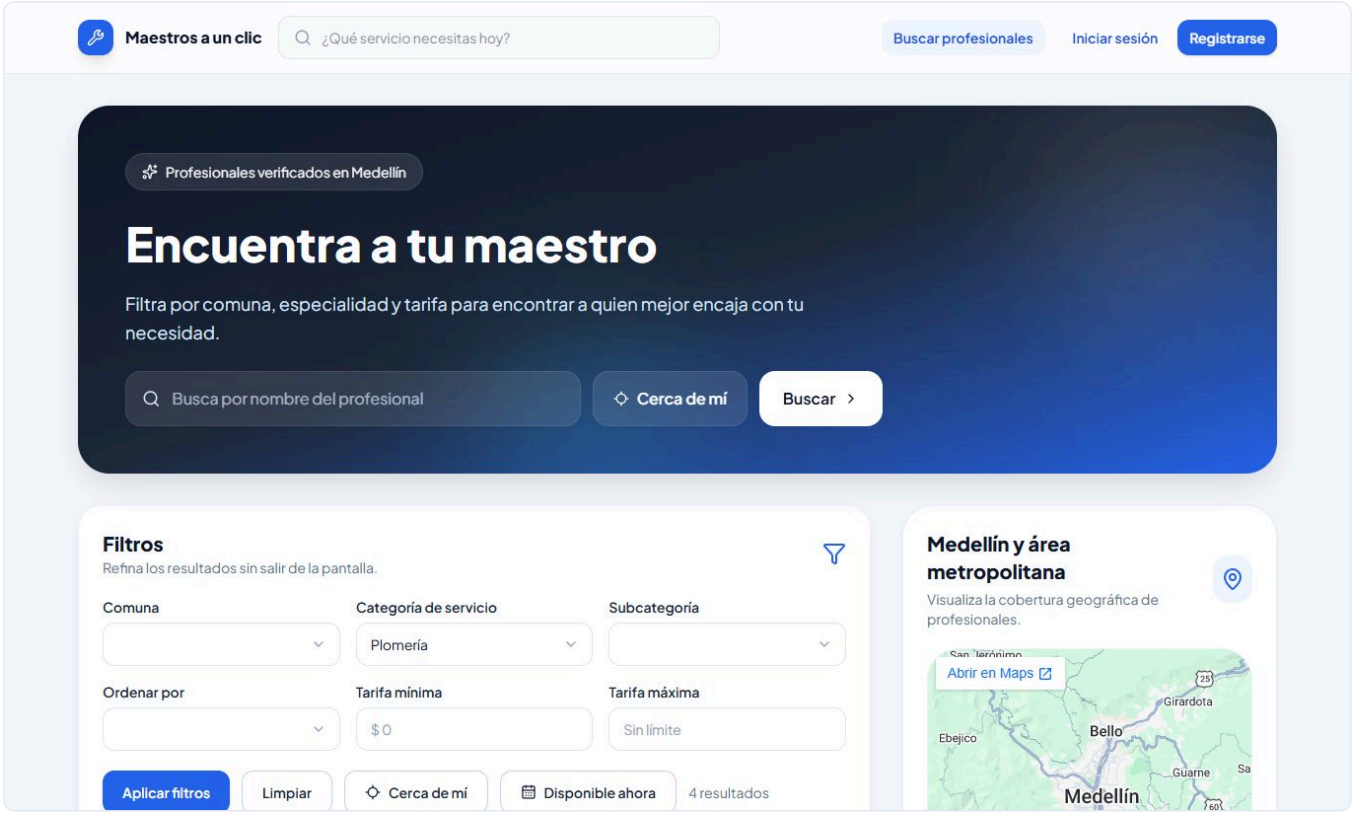


Figura 2. Búsqueda (HU010, HU033): filtros por comuna, categoría, subcategoría, orden y tarifa, botones "Cerca de mí" y "Disponible ahora", y mapa de la zona.

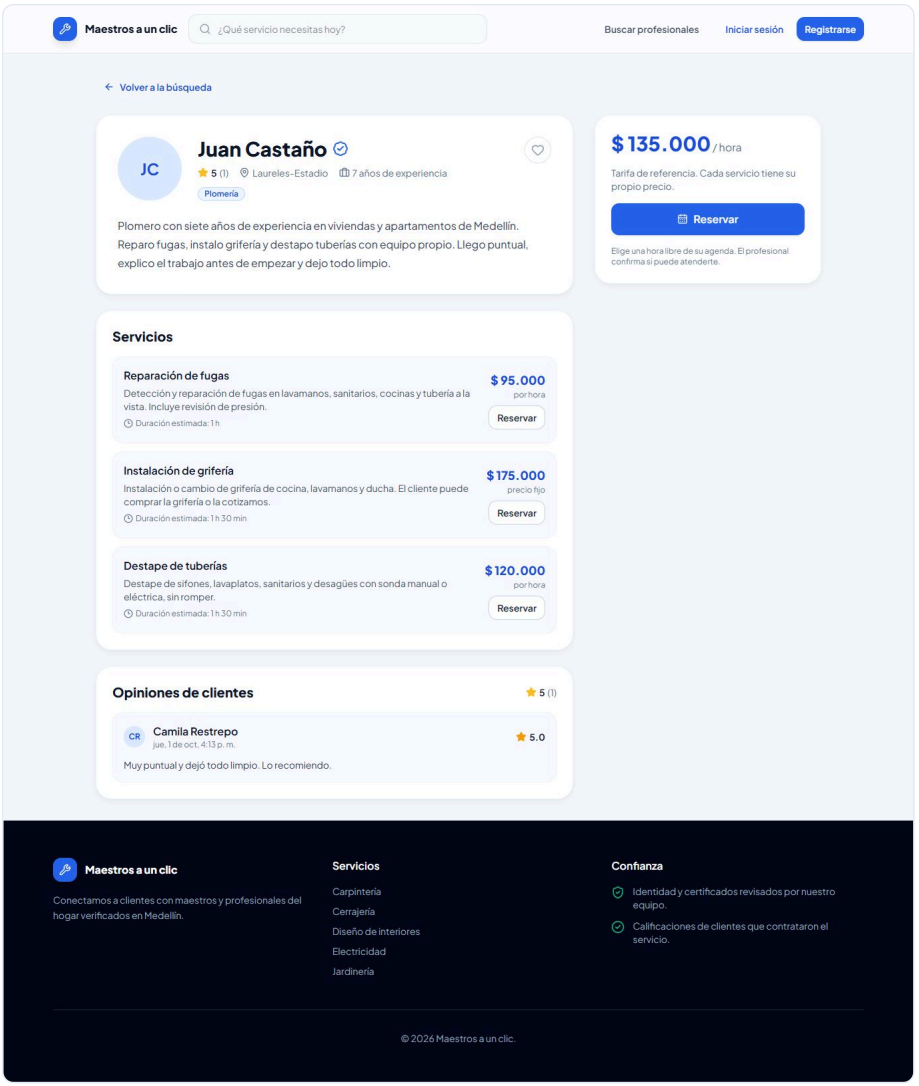


Figura 3. Perfil público del profesional: sello de verificado, calificación, experiencia, servicios con precio y duración estimada (HU047), y opiniones de clientes (HU029).

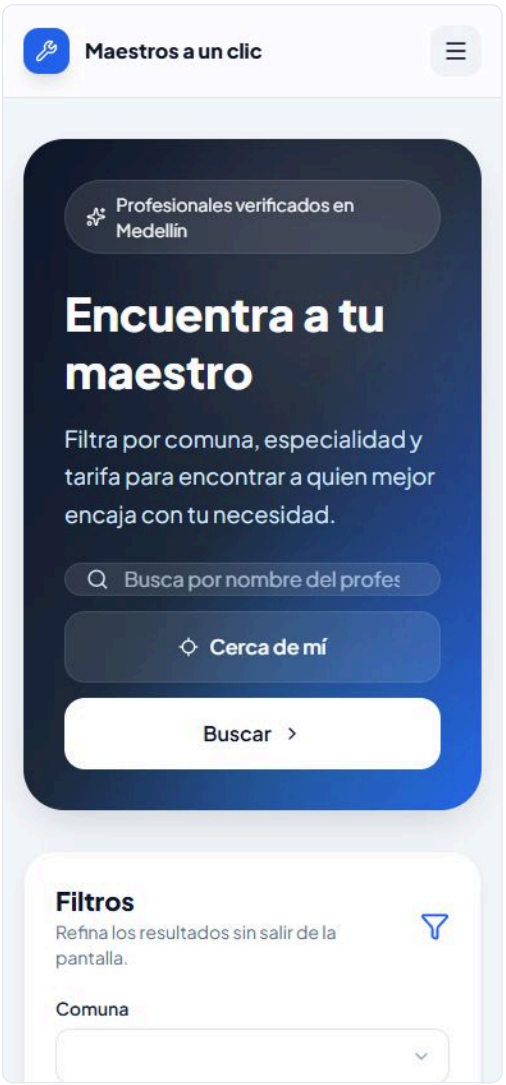


Figura 4. La búsqueda en un celular (390 px).

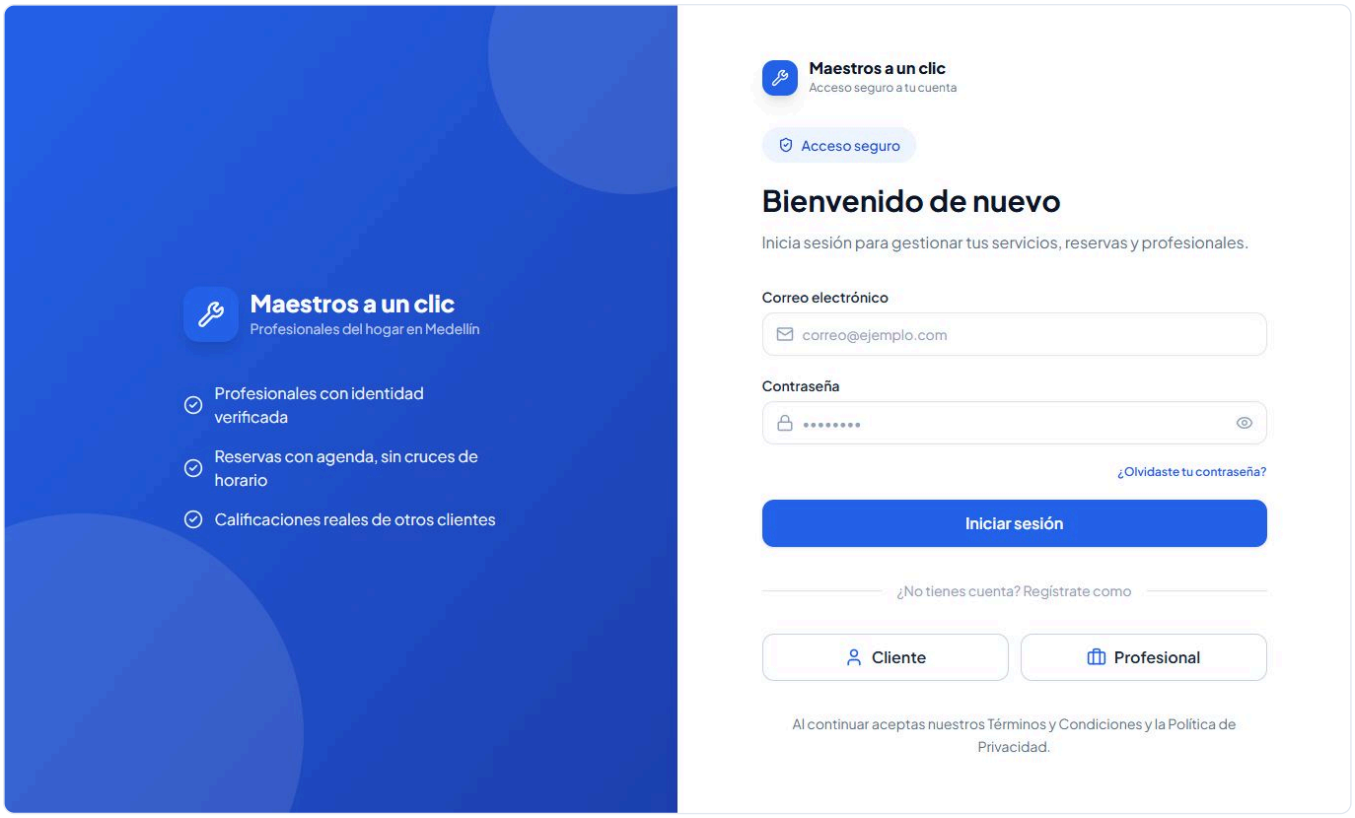


Figura 5. Inicio de sesión (HU001), con acceso al registro de cliente o profesional y a la recuperación de contraseña.

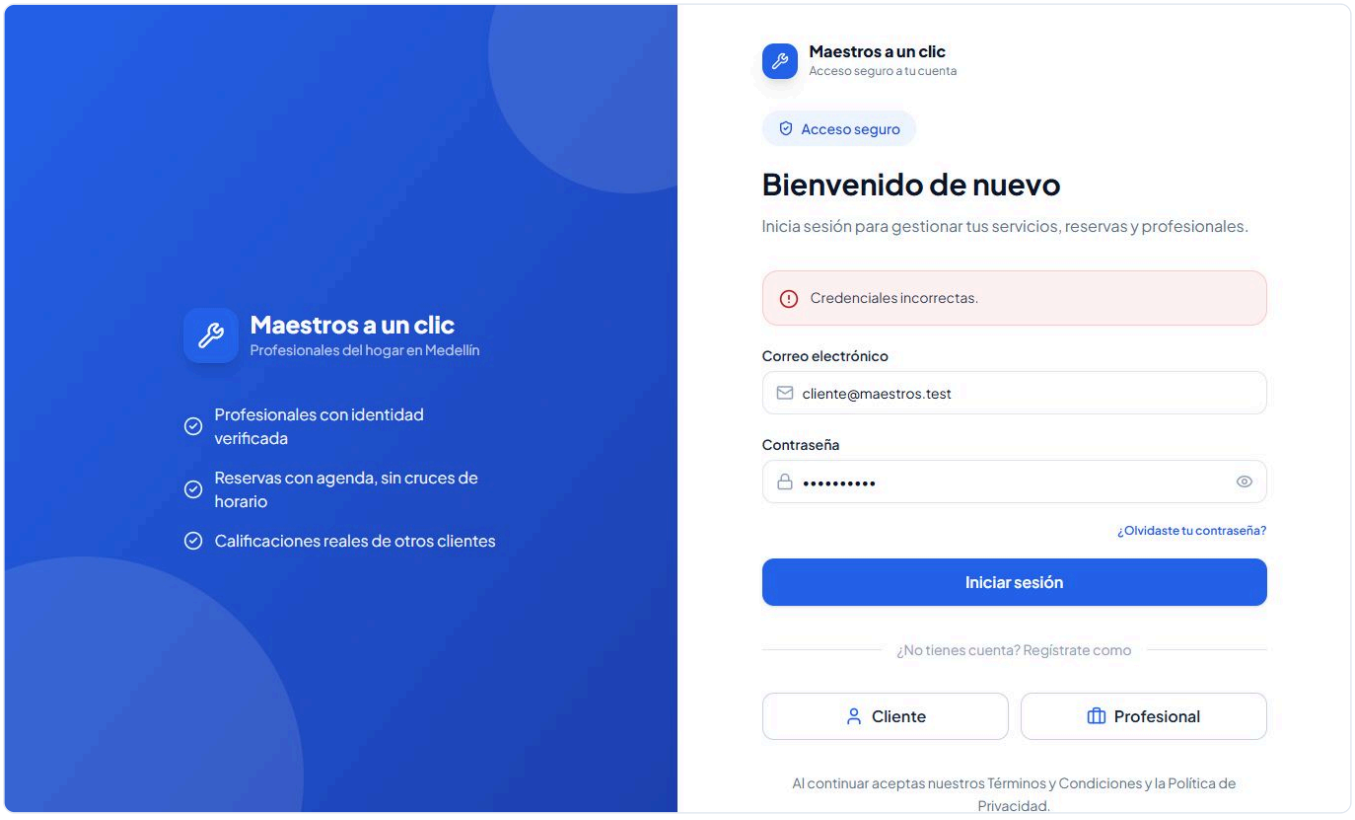


Figura 6. Validación: con una contraseña incorrecta, el sistema muestra el error y no da acceso.

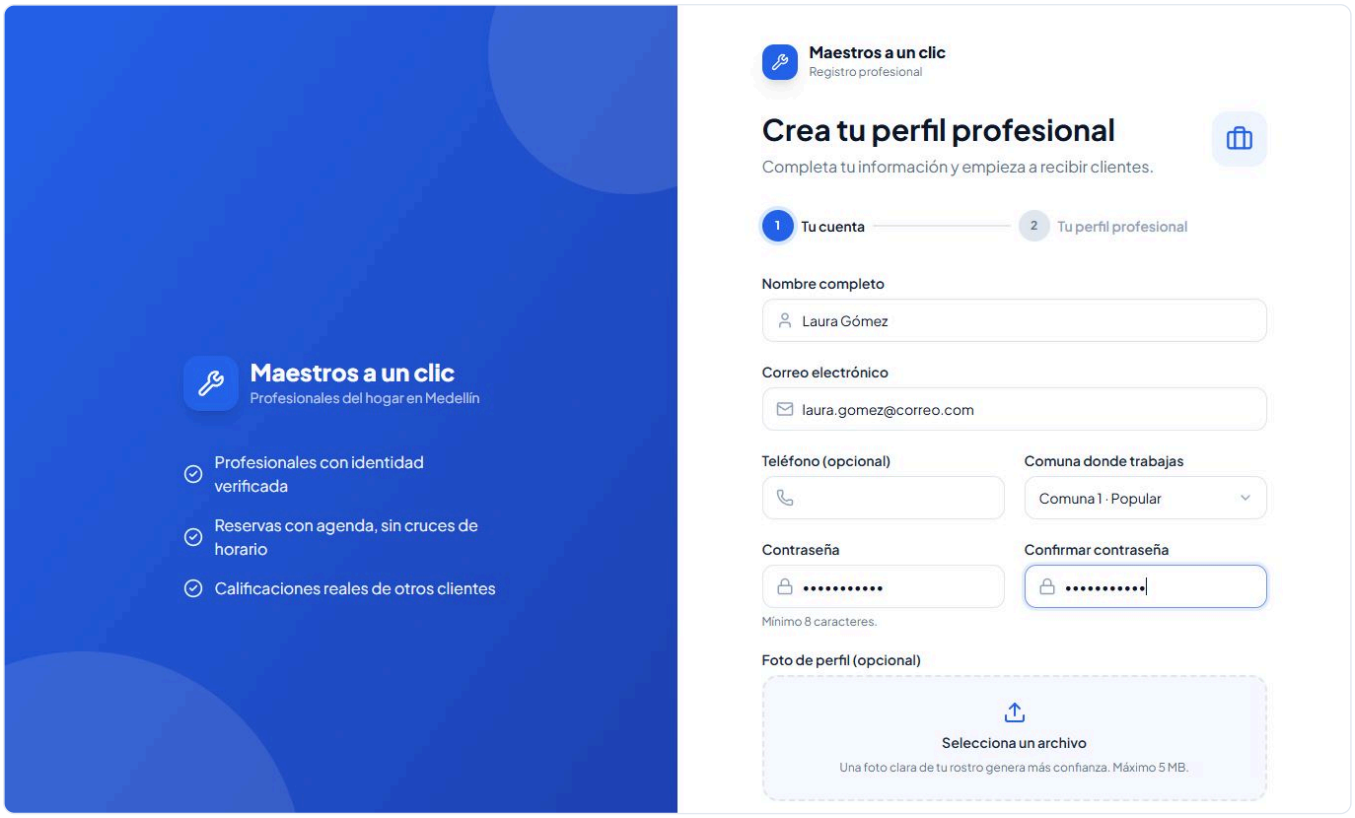


Figura 7. Registro de profesional, paso 1 (HU007): datos de la cuenta y comuna.

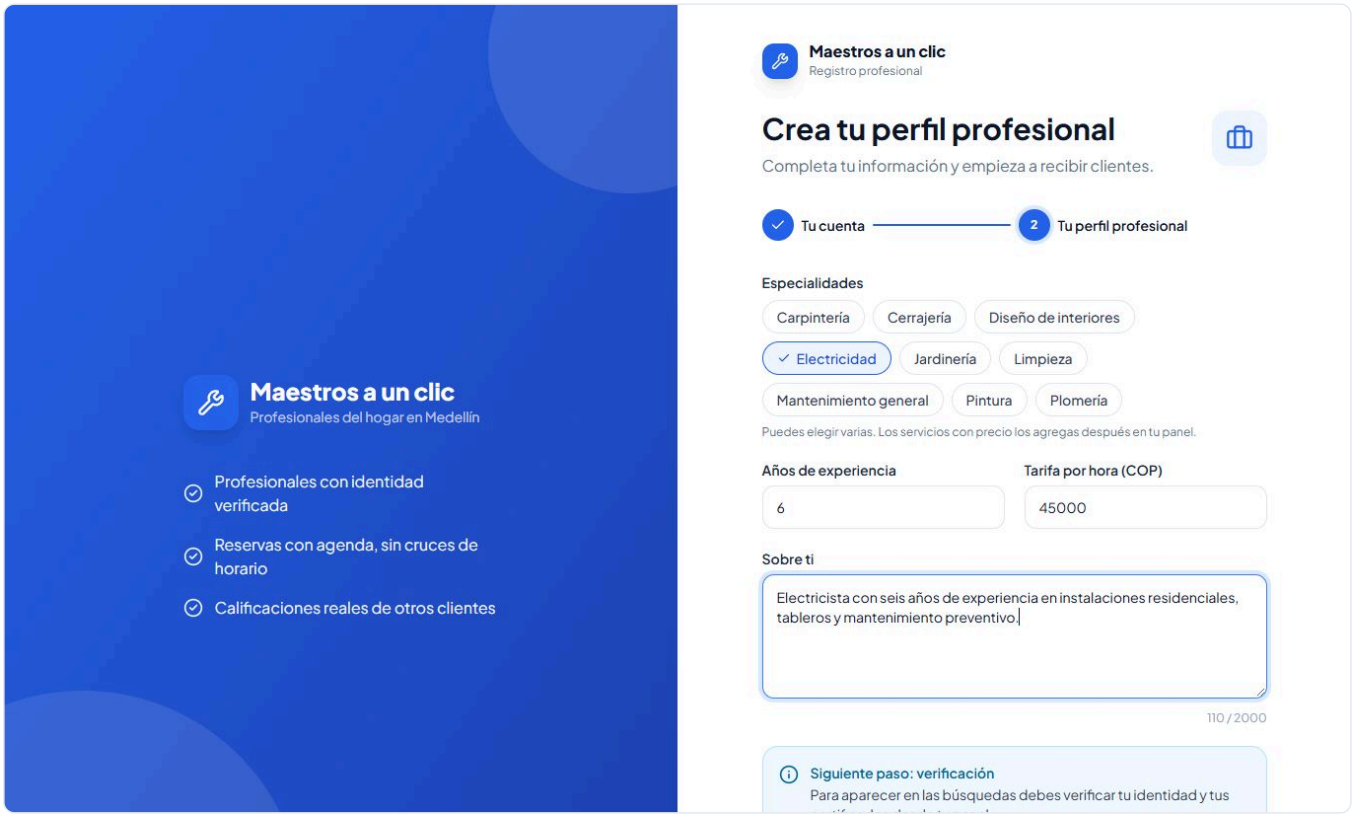


Figura 8. Registro de profesional, paso 2: especialidades, experiencia, tarifa y descripción.

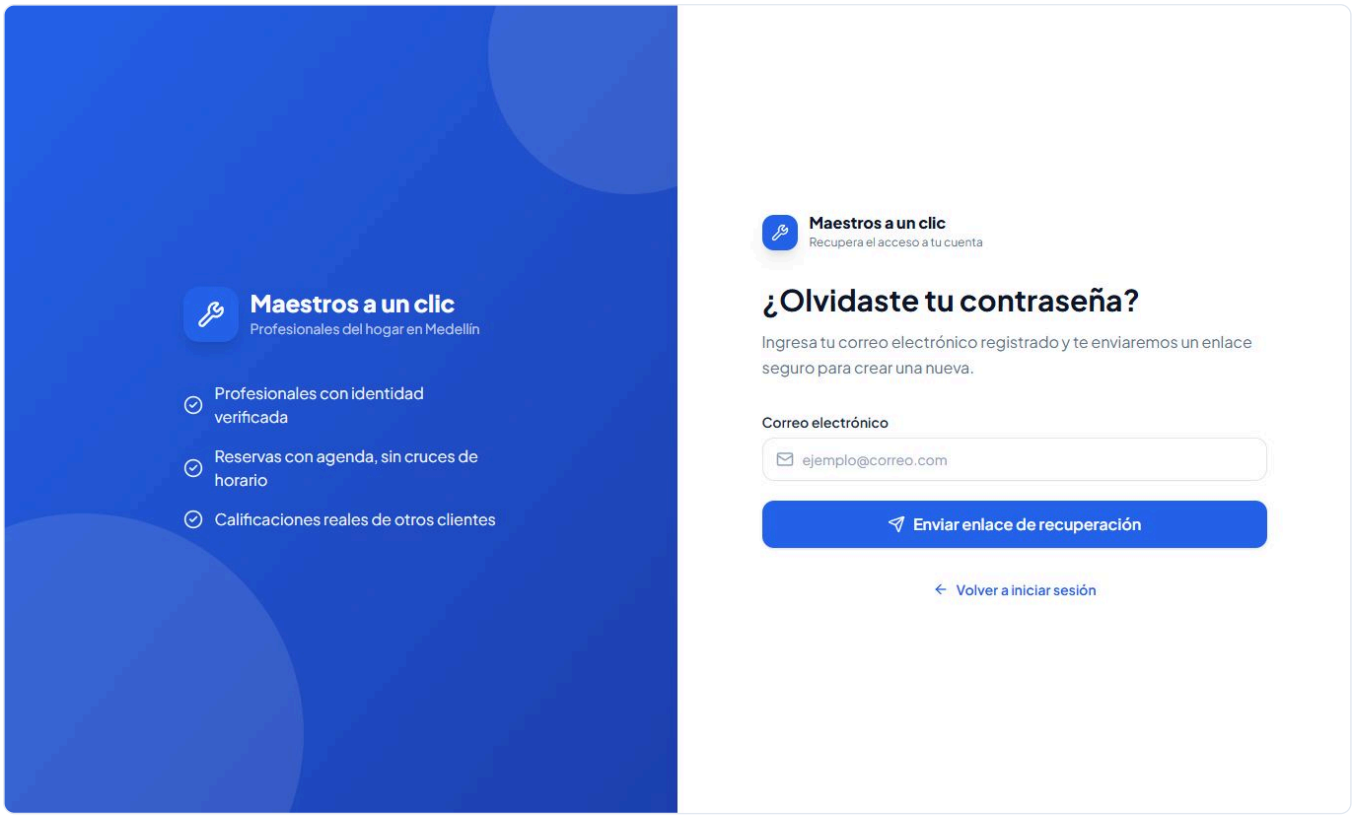


Figura 9. Recuperación de contraseña (HU020).

Cliente

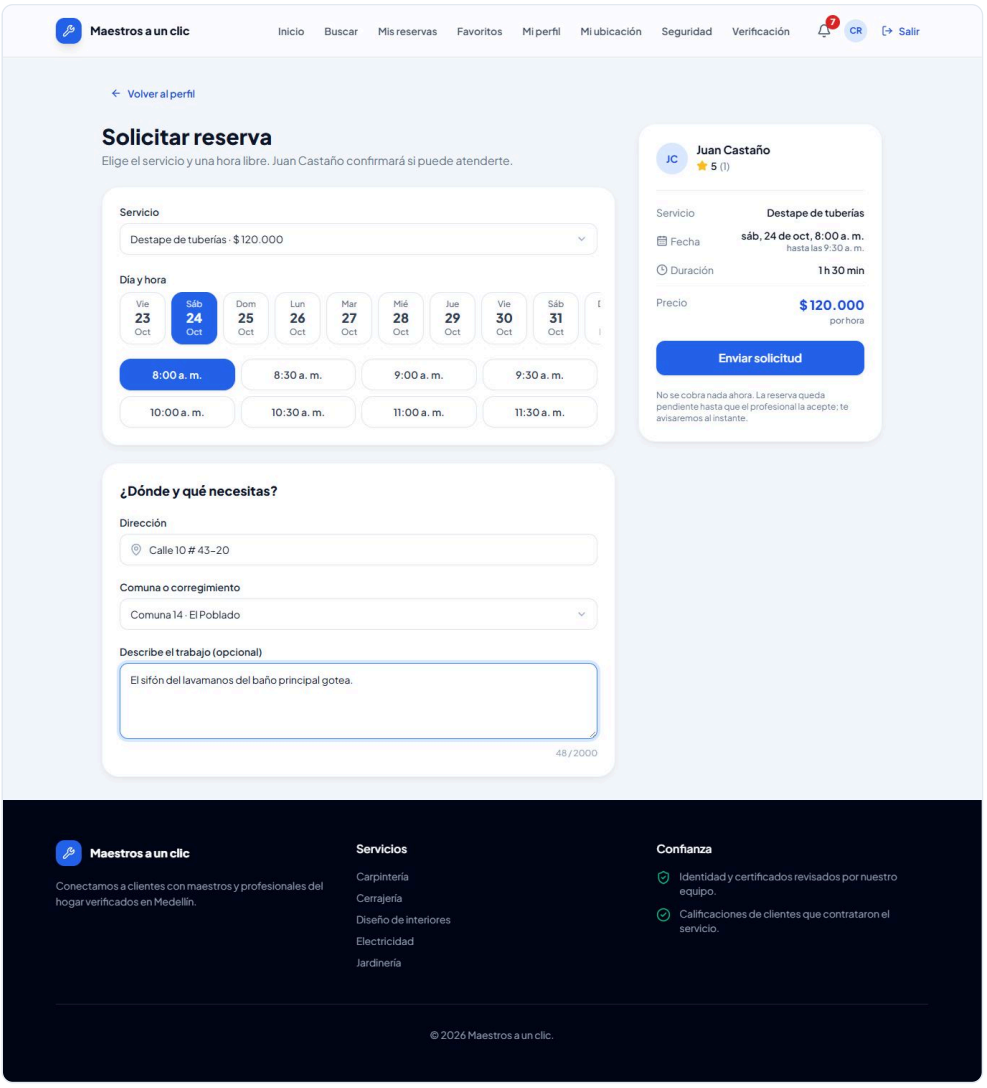


Figura 10. Solicitar una reserva (HU004, HU025): el cliente elige el servicio, el día y una hora libre de la agenda real, y ve el resumen con fecha, duración y precio.

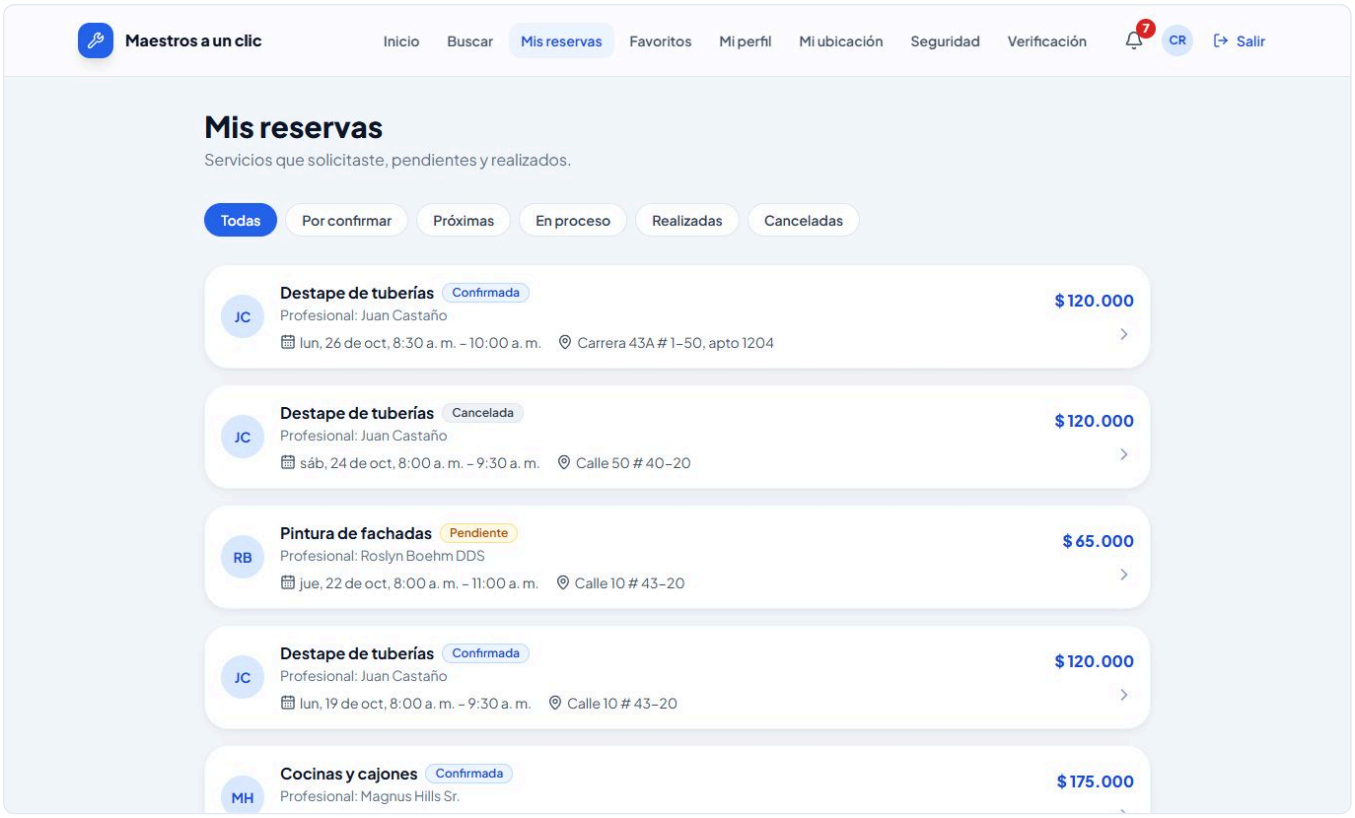


Figura 11. Mis reservas (HU016), con totales y filtros por estado.

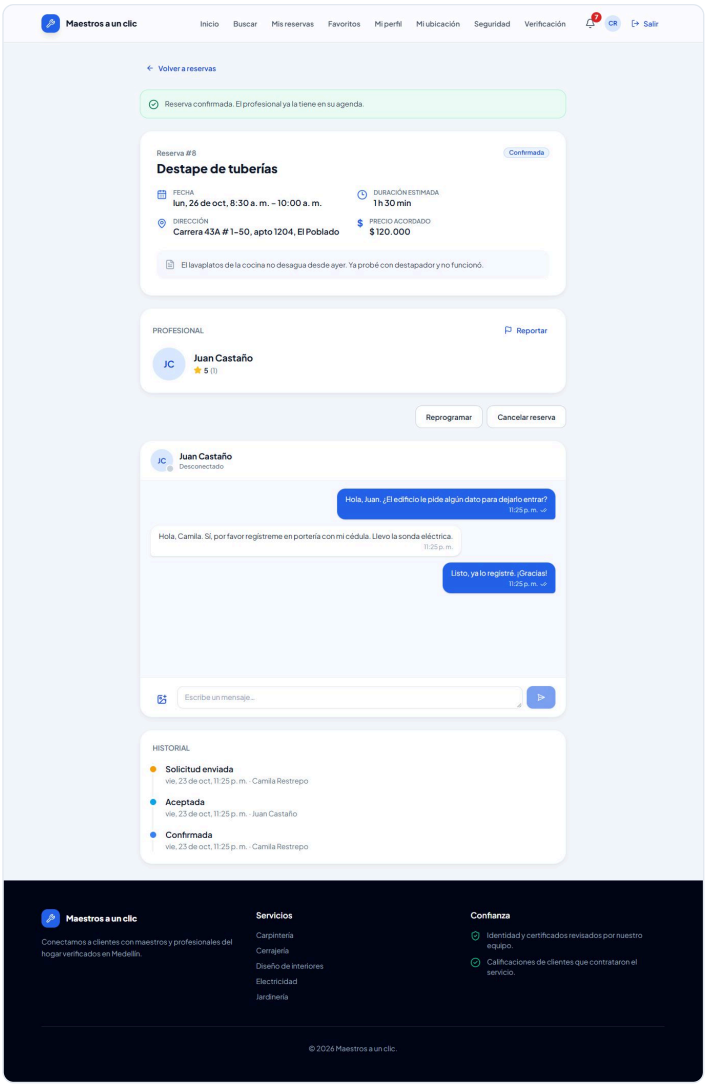


Figura 12. Detalle de una reserva confirmada: datos, profesional, acciones para reprogramar o cancelar (HU012), chat (HU011) e historial de estados con autor y fecha (HU026).

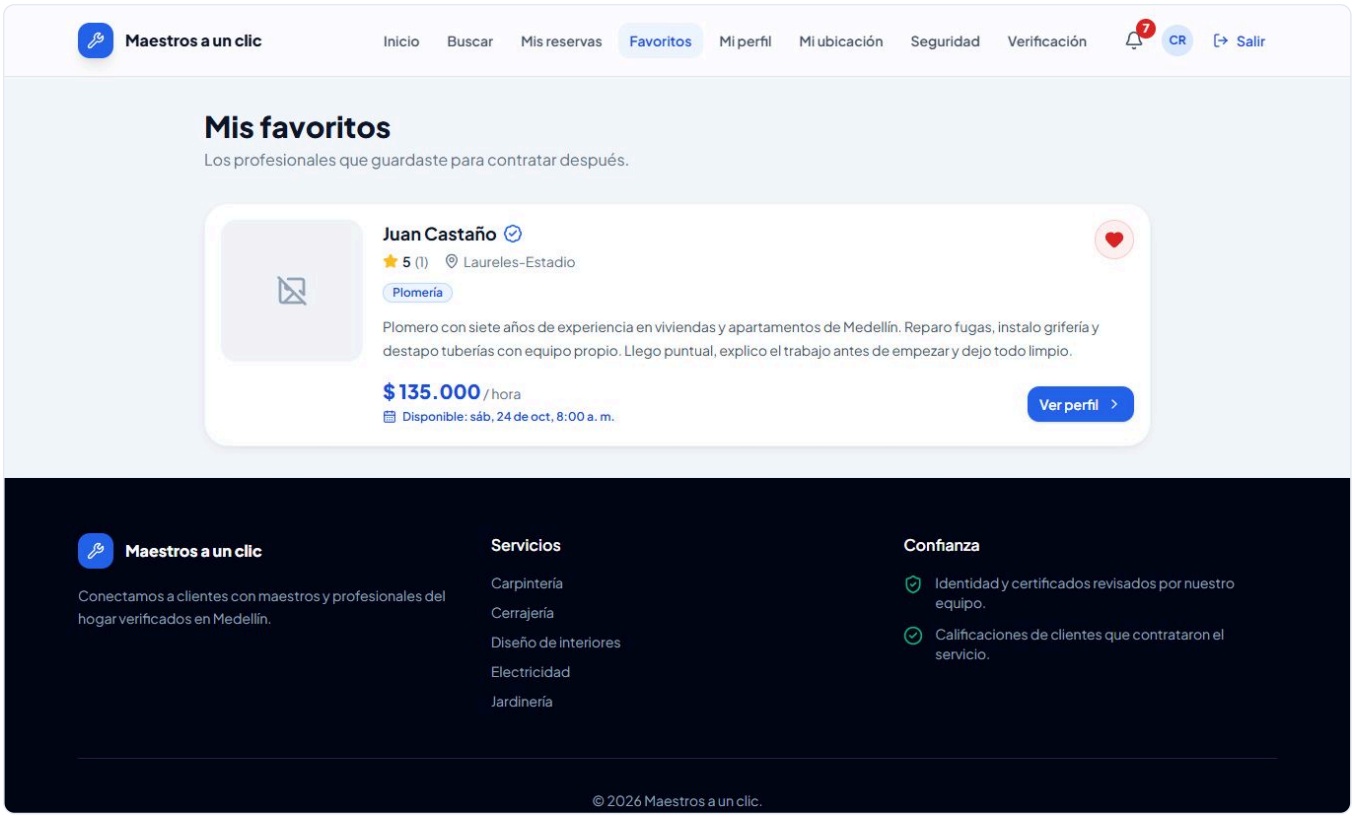


Figura 13. Profesionales favoritos (HU035).

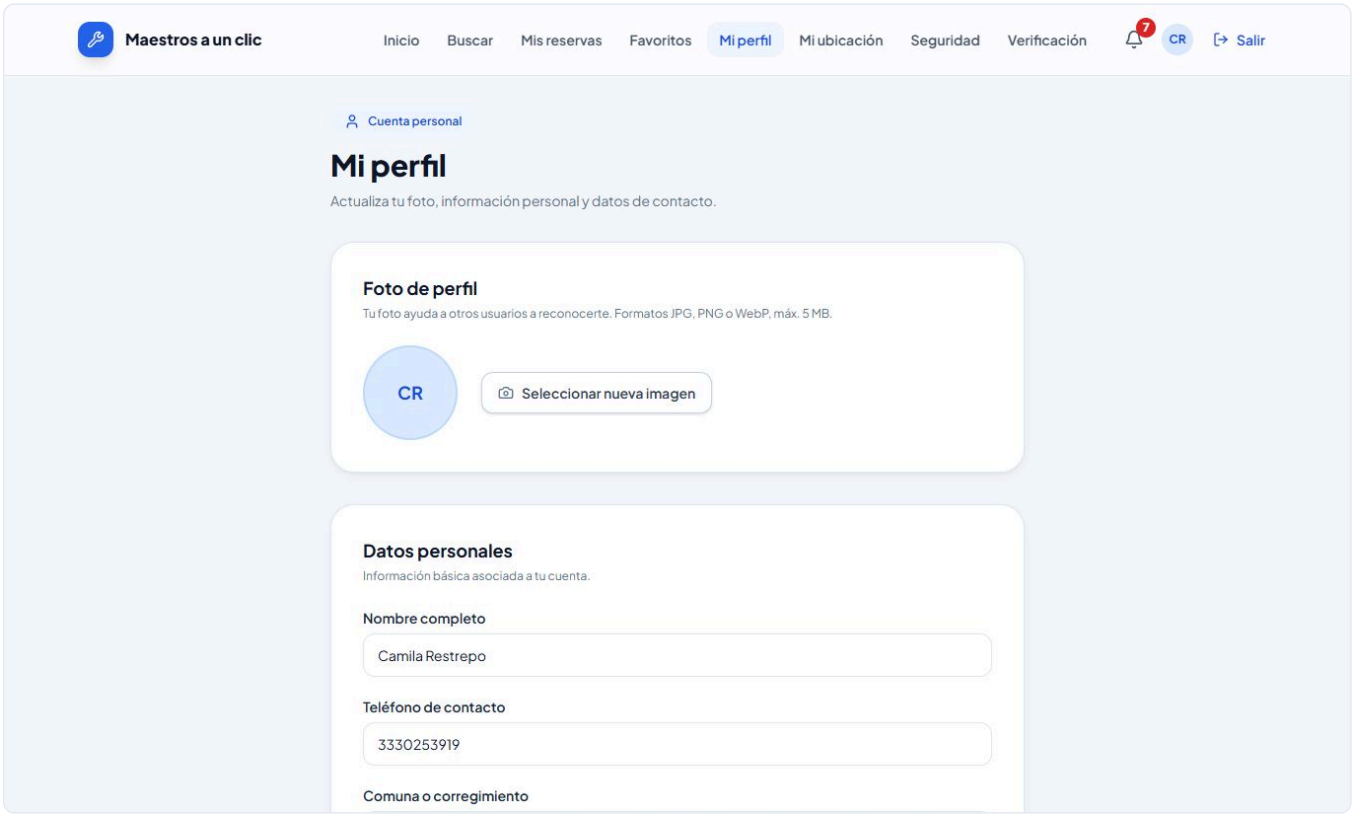


Figura 14. Edición del perfil y la foto (HU031, HU032).

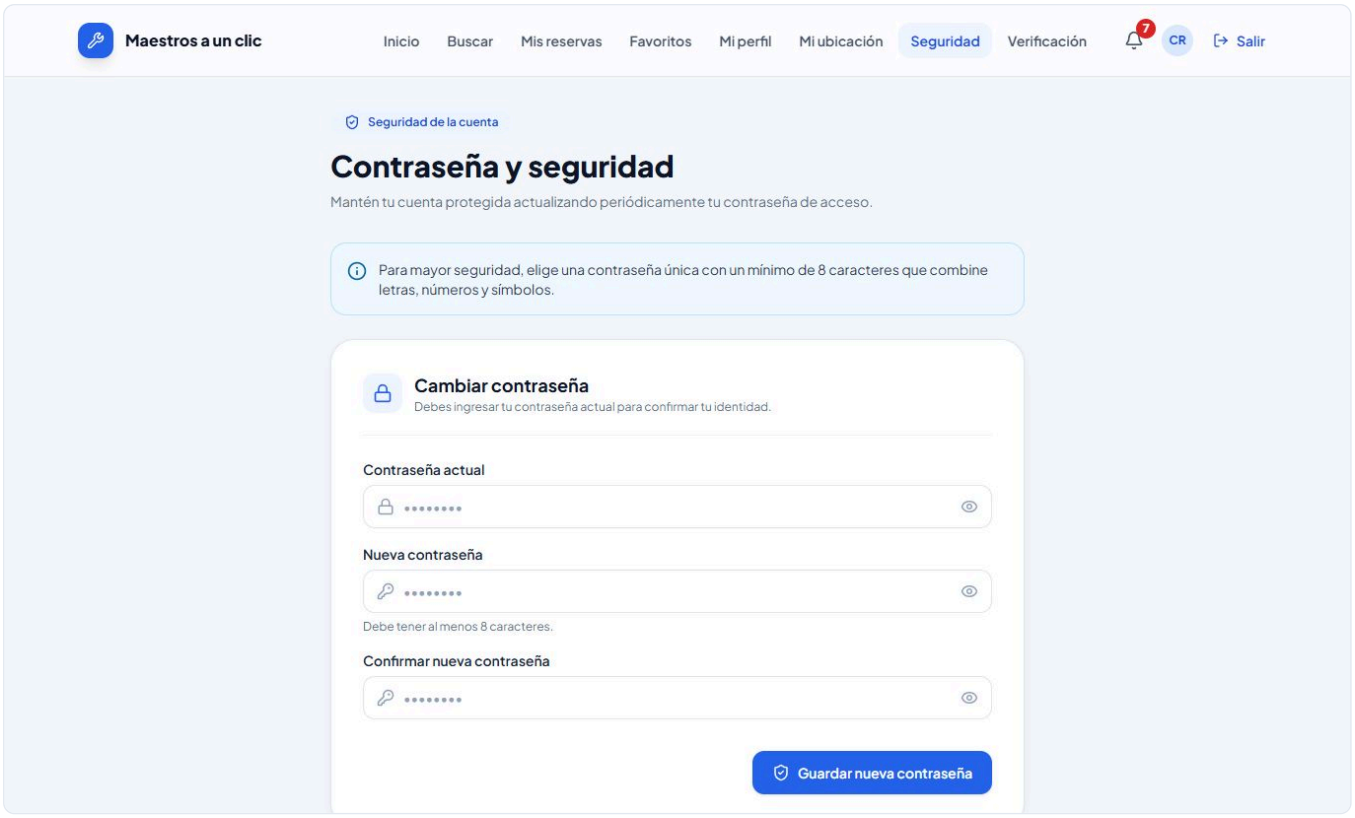


Figura 15. Seguridad: cambio de contraseña (HU019). Más abajo, en la misma página, está la eliminación de la cuenta (HU023).

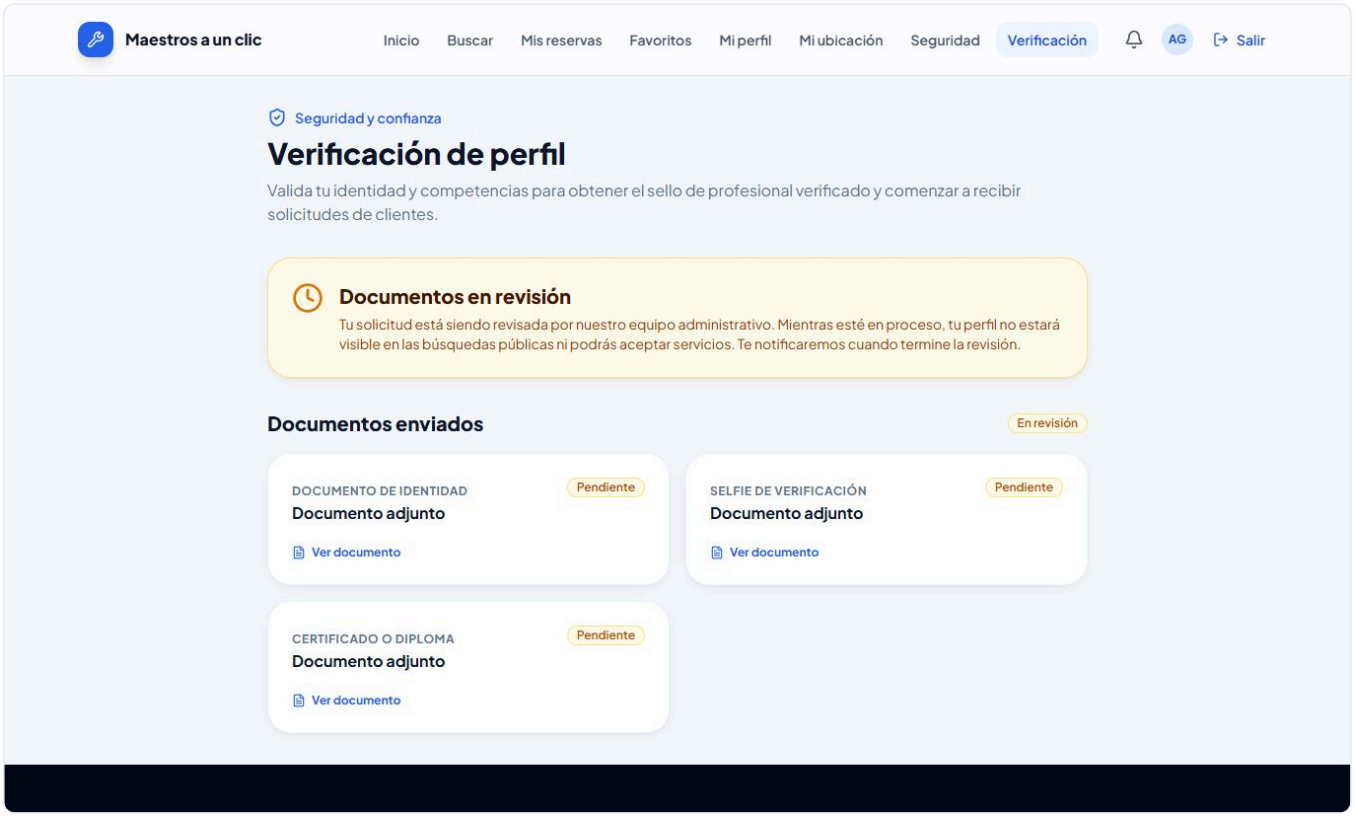


Figura 16. Verificación de identidad (HU008): documentos enviados y en revisión.

Profesional

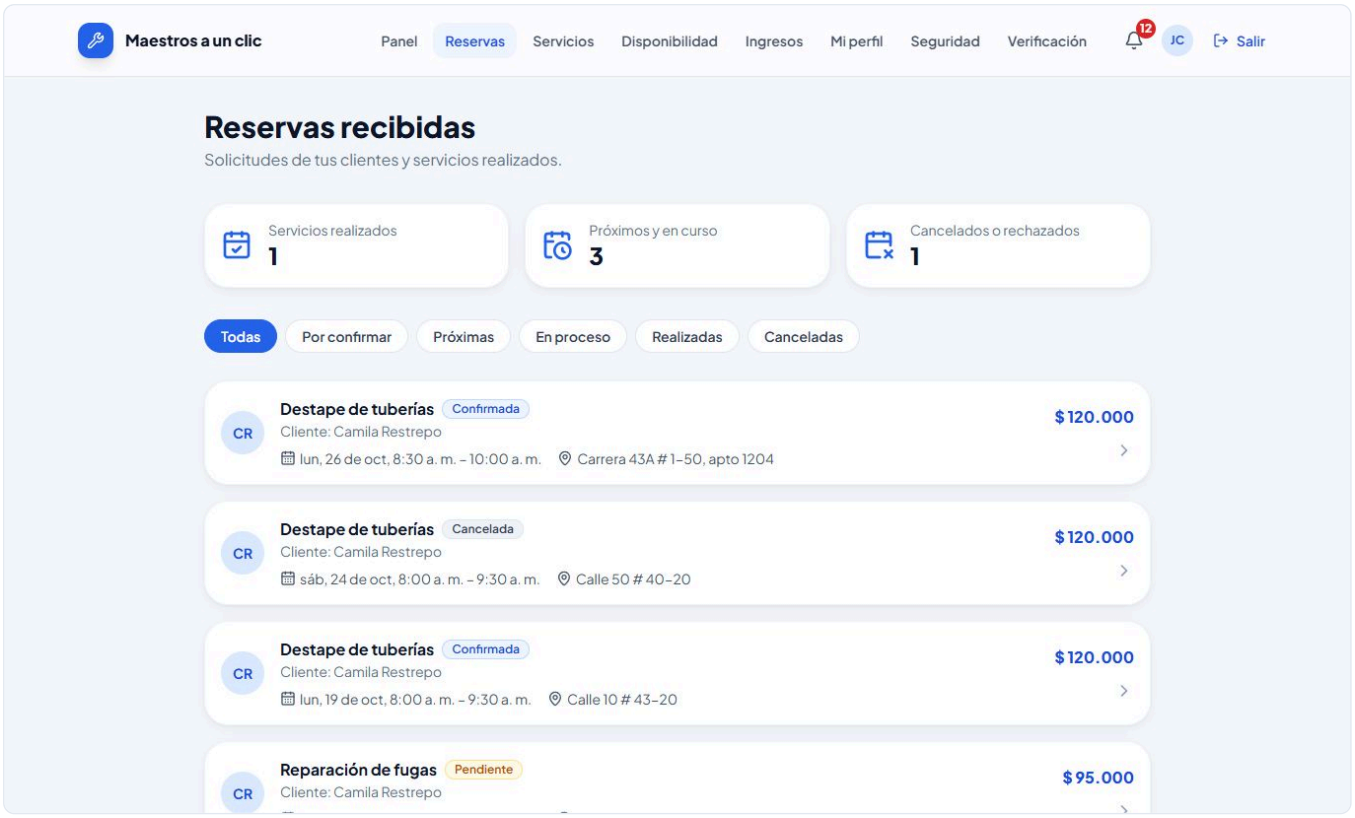


Figura 17. Reservas recibidas (HU017), con totales por estado.

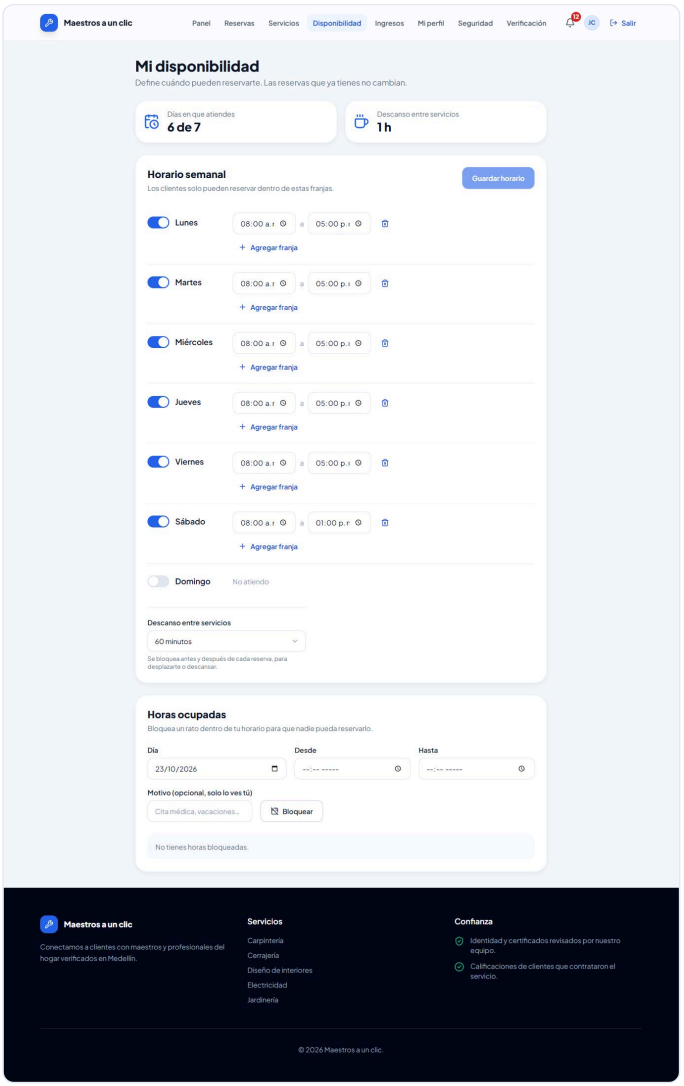


Figura 18. Disponibilidad (HU003): horario semanal con varias franjas por día, descanso entre servicios y horas bloqueadas.

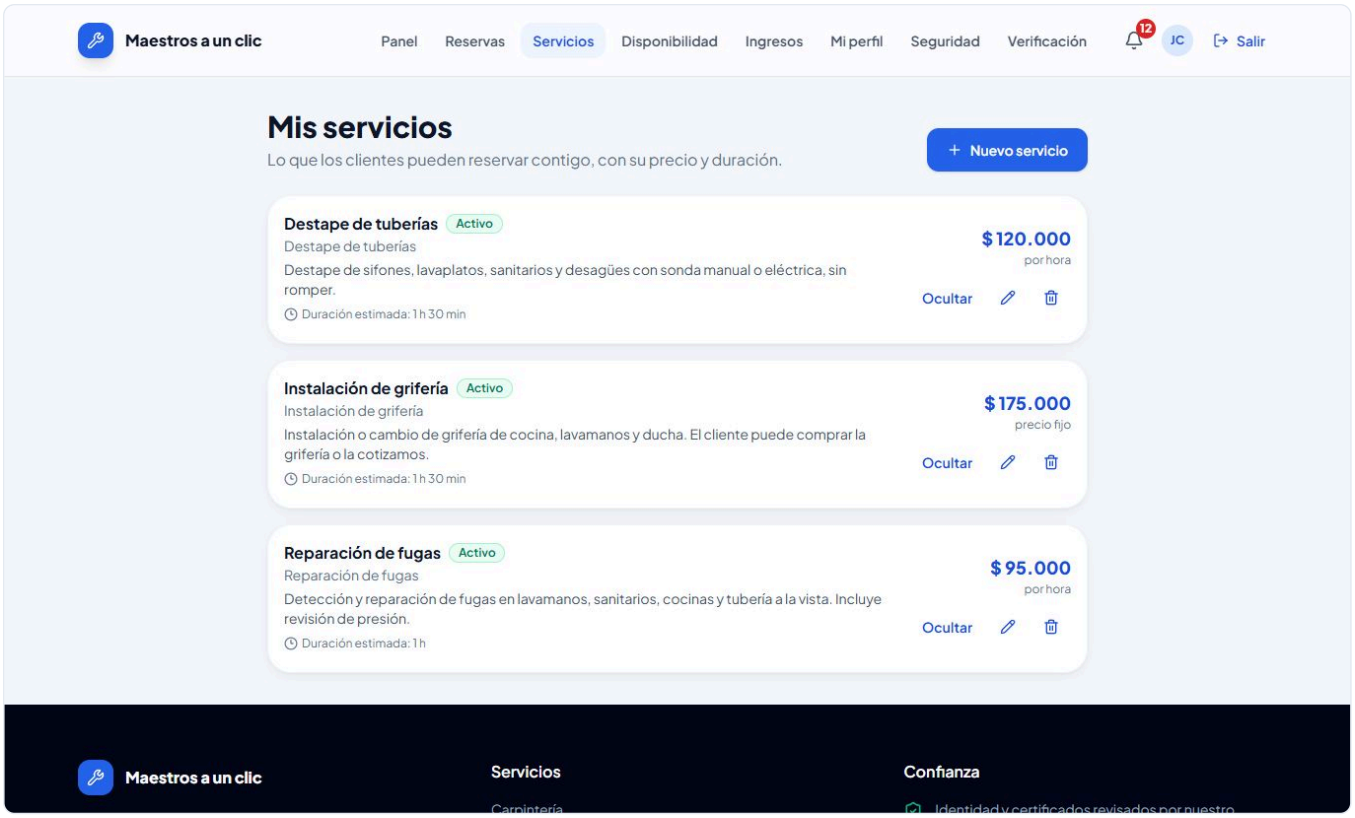


Figura 19. Servicios del profesional con precio y duración estimada (HU009, HU047).

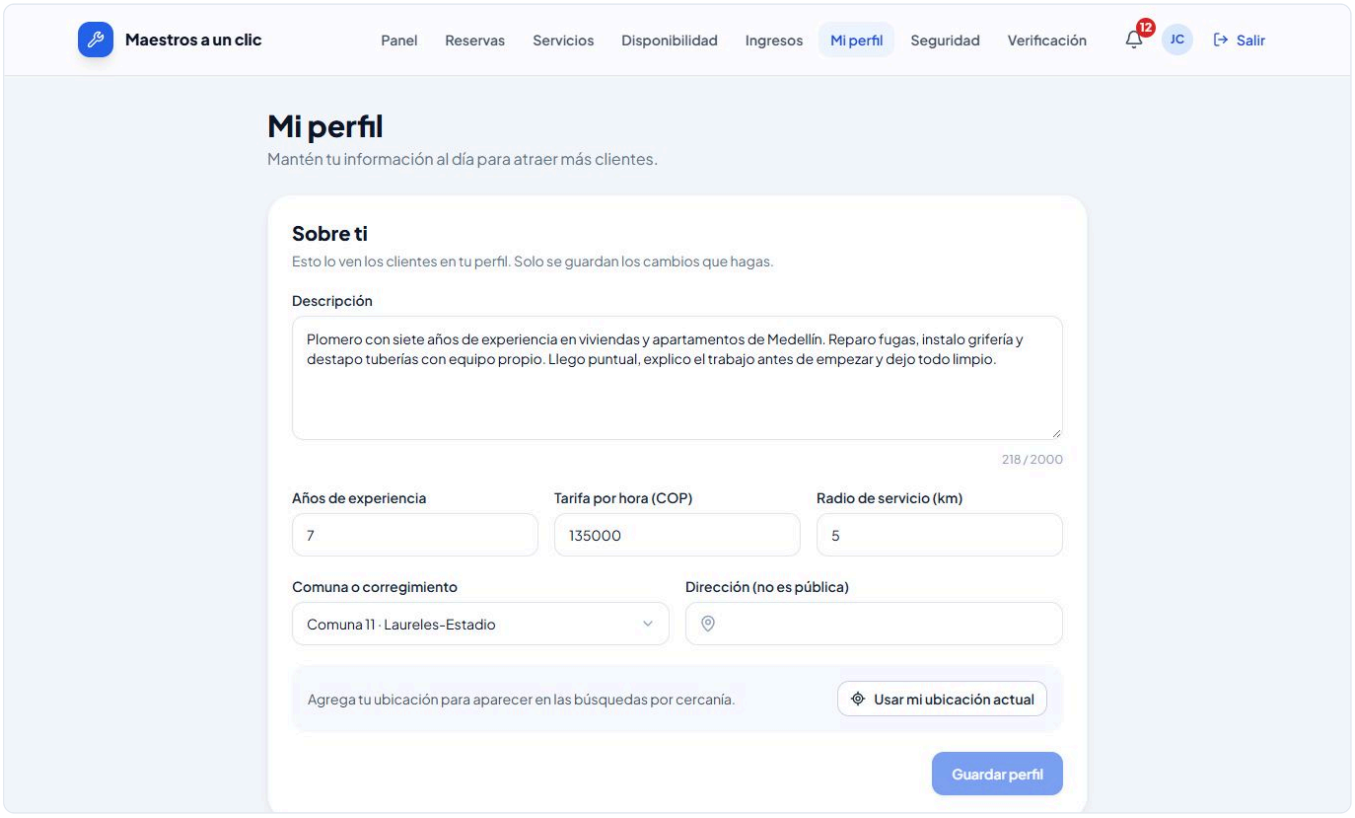


Figura 20. Perfil profesional (HU009): descripción, experiencia, tarifa, radio de servicio y ubicación. Más abajo, en la misma página, están las especialidades y el portafolio.

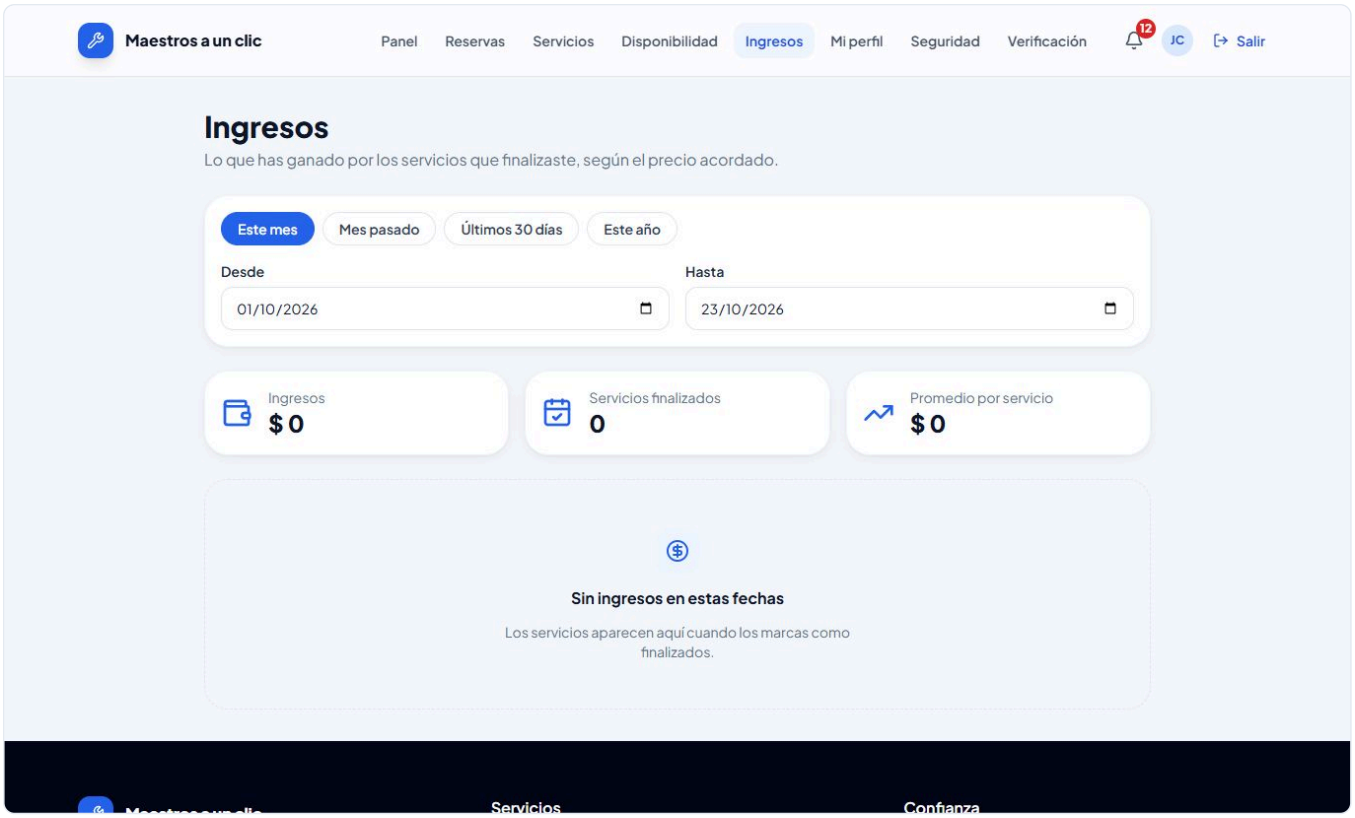


Figura 21. Ingresos por rango de fechas (HU037).

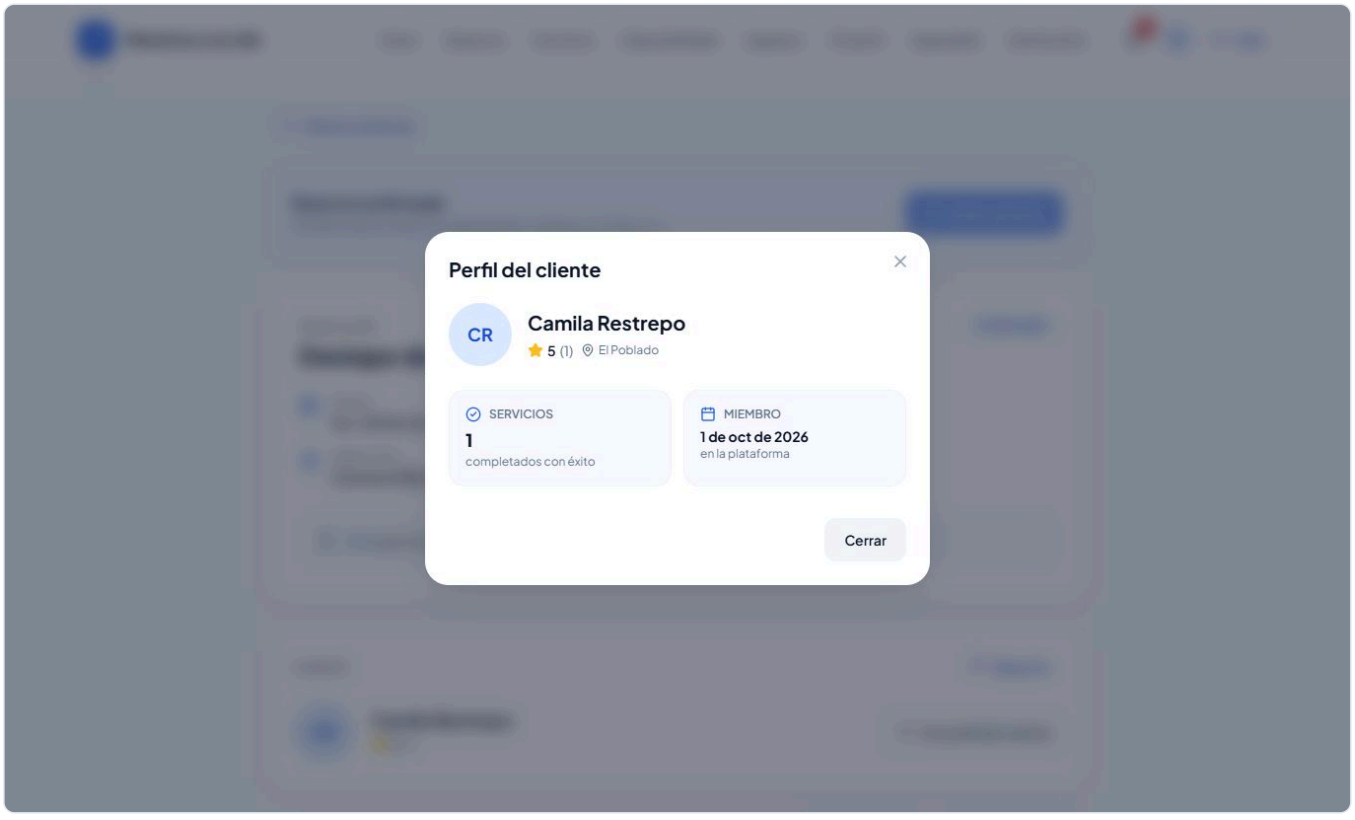


Figura 22. Perfil del cliente desde la solicitud (HU044): calificación, comuna, servicios completados y antigüedad.

Administrador

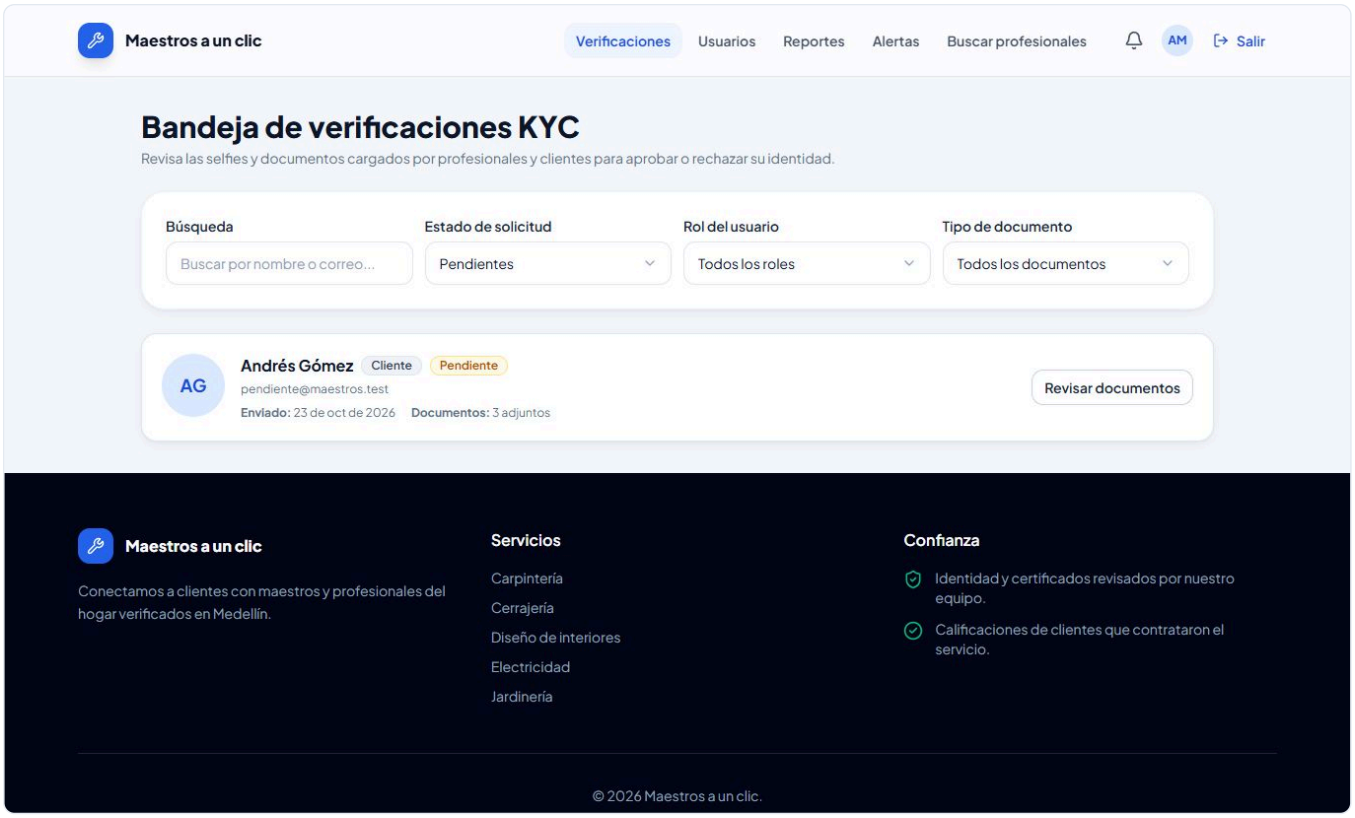


Figura 23. Bandeja de verificaciones (HU014), con filtros por estado, rol y tipo de documento.

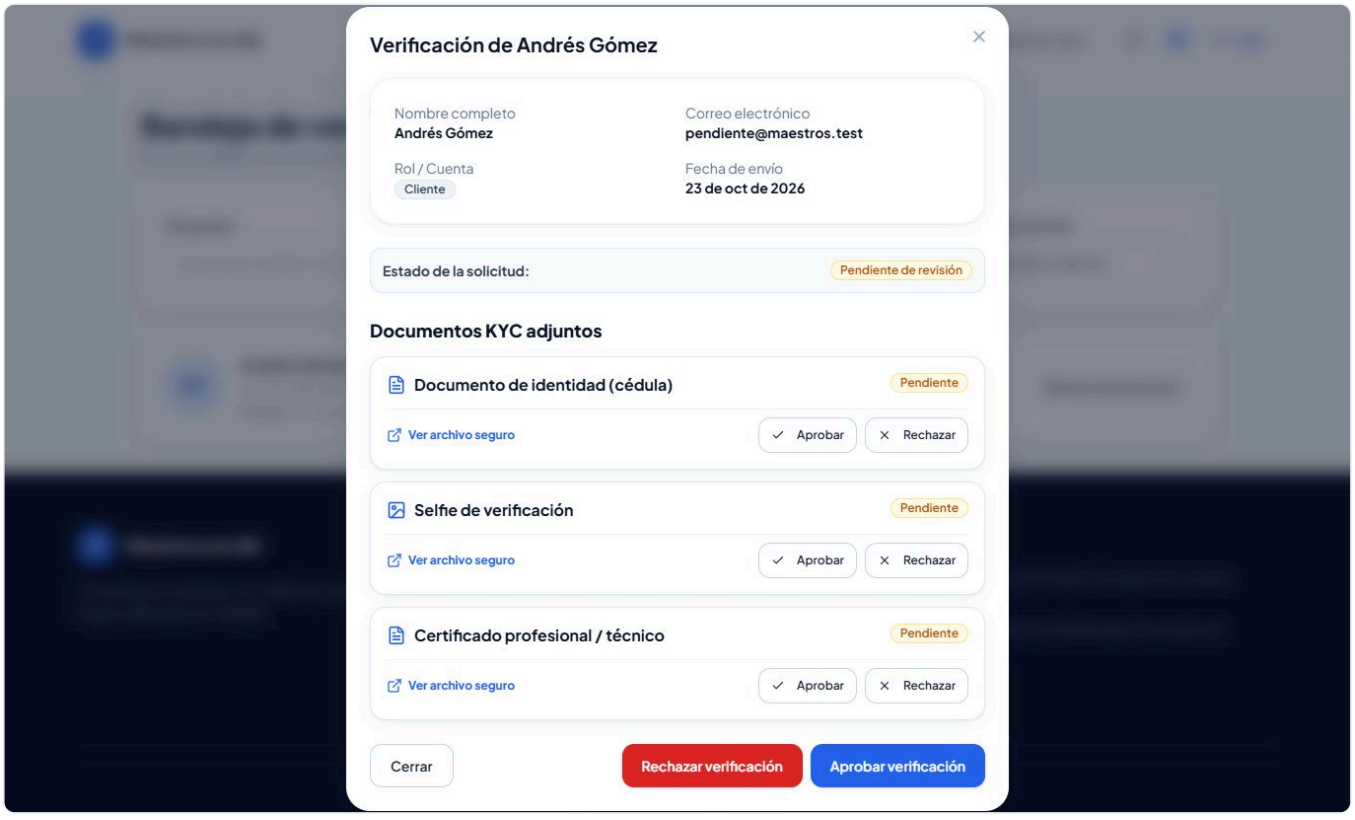


Figura 24. Revisión de una solicitud: cada documento se abre por un enlace seguro y se aprueba o rechaza por separado.

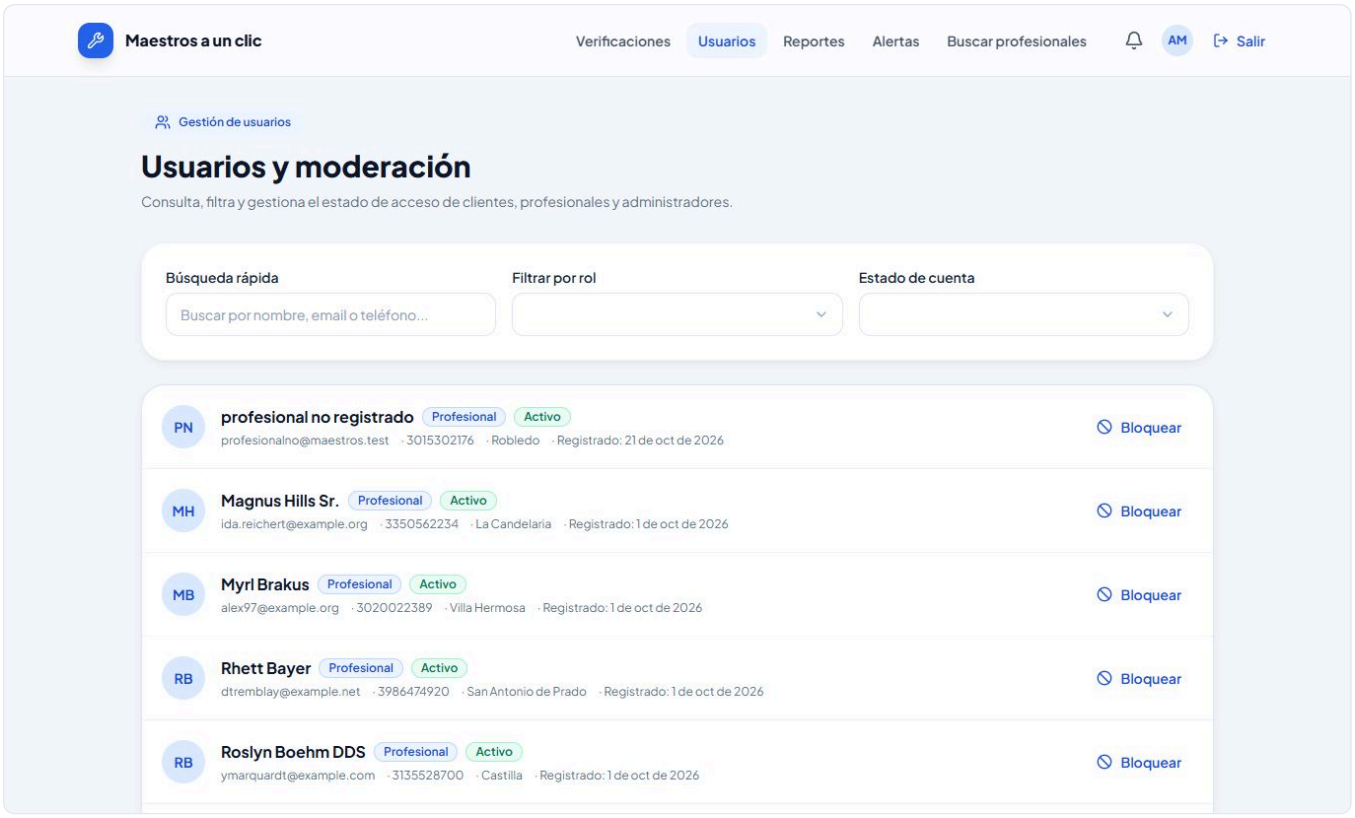


Figura 25. Moderación de usuarios (HU038): búsqueda, filtros, bloqueo y desbloqueo.

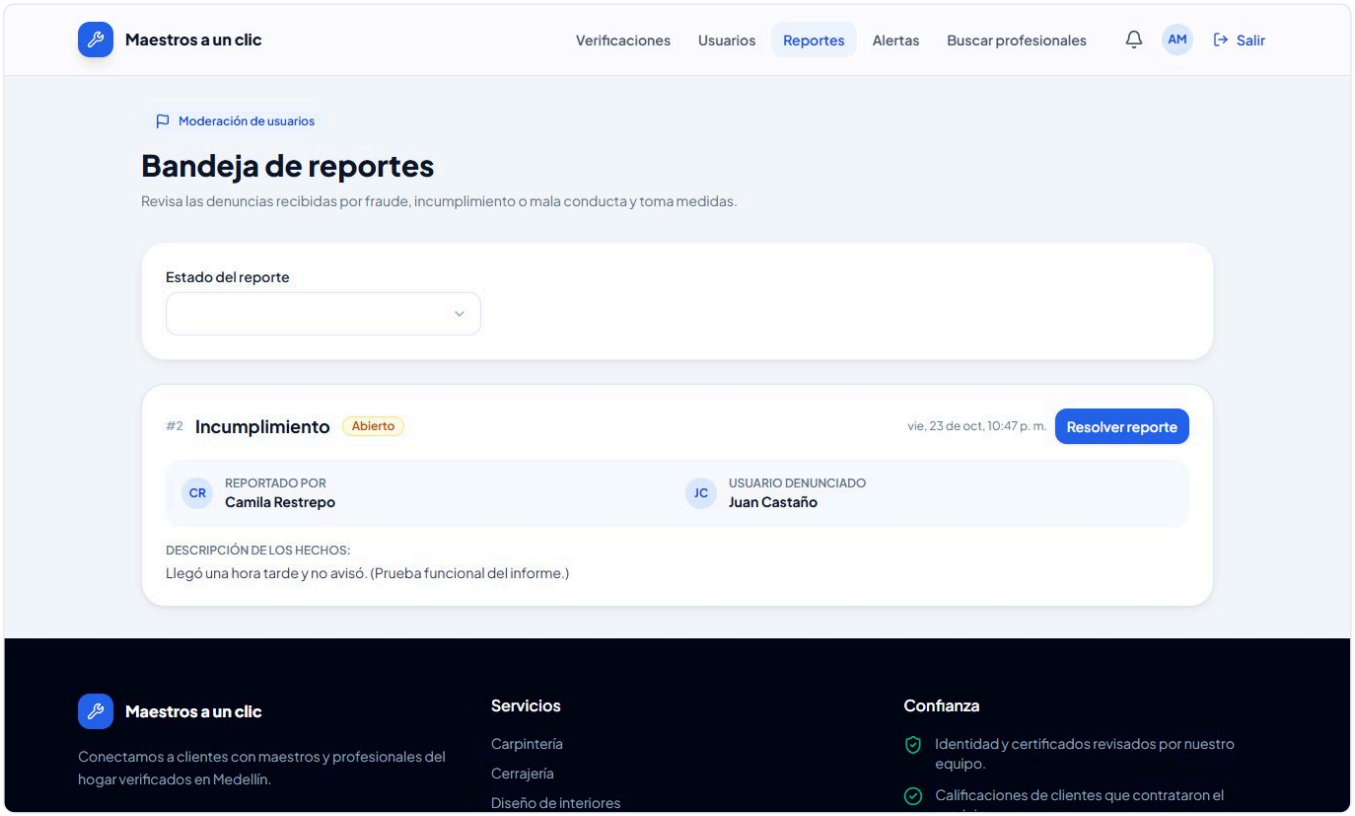


Figura 26. Bandeja de reportes (HU024): quién reporta, a quién, motivo y descripción.

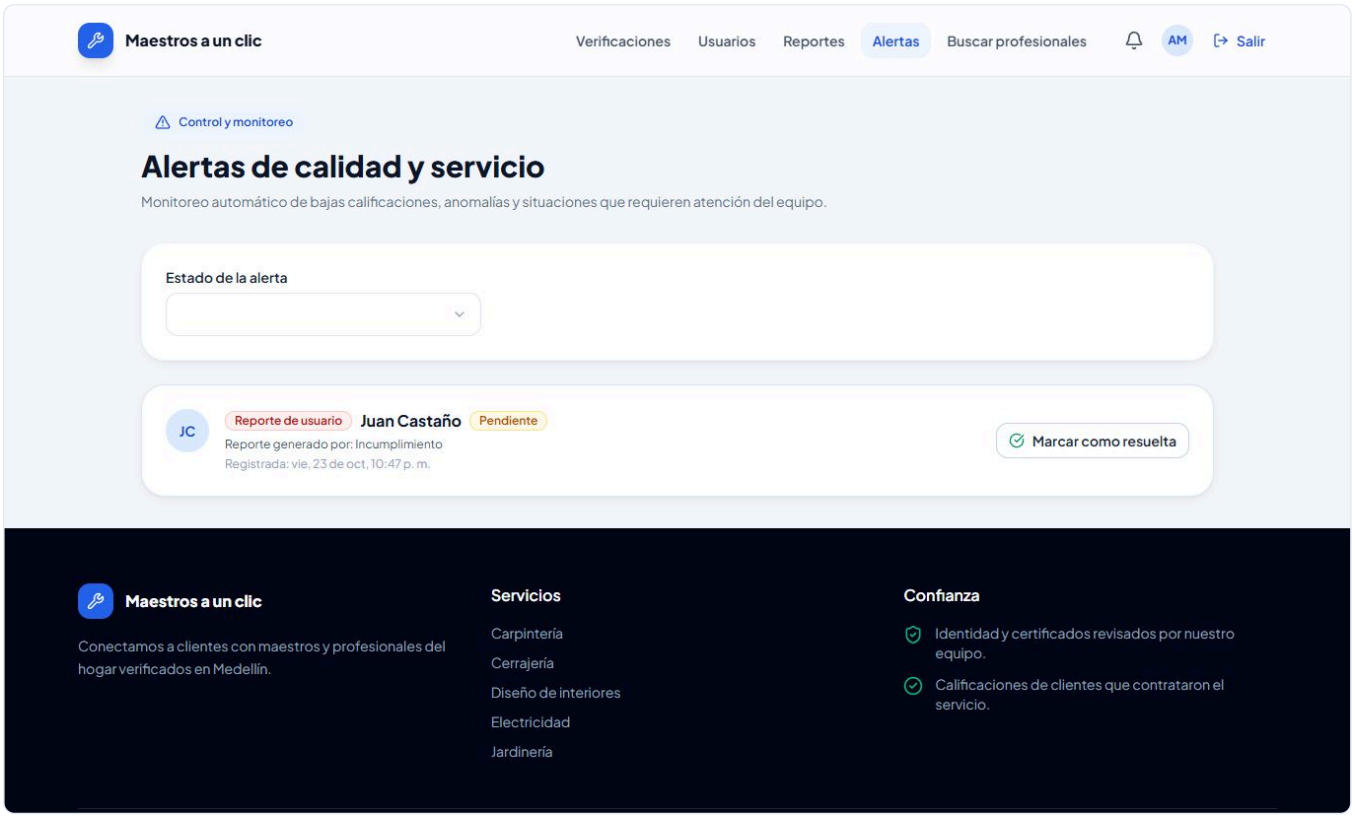


Figura 27. Alertas del sistema (HU045), generadas por reportes y bajas calificaciones.

2.5 Tecnologías y herramientas utilizadas

Capa	Tecnología	Versión	Uso
Backend	PHP	8.2 a 8.5	Lenguaje del servidor (la CI prueba en 8.2 y 8.5)
	Laravel	12.69	Framework de la API: rutas, validación, ORM, colas, notificaciones
	Laravel Sanctum	4	Autenticación por token
	Laravel Reverb	1.12	Servidor de WebSockets propio
	Scramble	0.13	Documentación OpenAPI generada desde el código
Frontend	Vue	3.5	Interfaz de una sola página
	TypeScript	5.9	Tipado estático
	Vite	8	Servidor de desarrollo y empaquetado
	Vue Router, Pinia, Pinia Colada	5 · 4 · 1.4	Navegación, estado y caché de datos del servidor
	VeeValidate + Zod	4.15 · 3.25	Formularios y validación
	Tailwind CSS	4.3	Estilos con tokens del sistema de diseño
	Laravel Echo + pusher-js	2.5 · 8.6	Cliente de tiempo real
Datos	MySQL / MariaDB	8 / 10.4	Base de datos relacional
Calidad	Pest	3.8	Pruebas de API (backend)
	Vitest + Vue Test Utils	4.1 · 2.4	Pruebas de componentes y funciones (frontend)
	Pint, ESLint, oxlint, Prettier	—	Estilo y análisis estático
	Puppeteer (Chrome)	—	Pruebas funcionales en el navegador, capturas y PDF de la documentación
Proceso	Git y GitHub	—	Ramas por historia, *pull requests* con revisión y *squash merge*
	GitHub Actions	—	CI obligatoria: pruebas, estilo, tipos, build, auditoría de dependencias y "Guardas"
	GitHub Issues	—	Una historia de usuario por issue, con sus criterios
Despliegue previsto	AWS EC2, Nginx, Supervisor, TLS	—	Ver 06-despliegue-y-operacion.md

2.6 Validaciones y restricciones del sistema

Toda entrada se valida en el servidor. Cada endpoint con datos tiene un `FormRequest`, y un error responde 422 con el mensaje de cada campo en español. El frontend valida además con Zod para avisar antes de enviar, pero nunca reemplaza al servidor.

Tipo	Validación o restricción
Registro y cuenta	Nombre y correo obligatorios, correo con formato válido y único, contraseña de 8 caracteres o más con confirmación, comuna existente. Al profesional se le exige además al menos una especialidad, una descripción de 50 caracteres o más, experiencia de 0 a 60 años y una tarifa mayor a 0
Archivos	Fotos de perfil y portafolio: JPG, PNG o WEBP de hasta 5 MB. Documentos de verificación: además PDF, de hasta 10 MB, y mínimo 3 por solicitud. Para aprobarla deben estar el documento de identidad y la selfie. Adjuntos del chat: imágenes de hasta 5 MB
Reservas	La fecha debe ser futura, estar dentro del horario, no tocar bloqueos ni cruzarse con otra reserva más el descanso. El servicio debe estar activo y el cliente verificado
Estados	Solo las transiciones permitidas: una acción fuera de orden responde 409 con el motivo (por ejemplo, iniciar antes de la hora o calificar una reserva no completada)
Agenda	Las franjas de un mismo día no se pueden cruzar, el fin debe ser posterior al inicio y el descanso va de 0 a 240 minutos
Chat	Solo los participantes. Texto de hasta 2000 caracteres, o una imagen. Se cierra cuando la reserva termina. Solo se eliminan los mensajes propios
Calificaciones y reportes	Calificación entera de 1 a 5 y comentario de hasta 1000 caracteres, una vez por parte. Reporte con motivo (3 a 100 caracteres) y descripción (10 a 2000). No se puede uno reportar a sí mismo ni usar una reserva ajena
Acceso	Rutas privadas con token (401 sin token). Rol por ruta (403). *Policies* por recurso: solo el dueño o un participante (403). Los usuarios bloqueados no entran
Abuso	Registro, inicio de sesión y recuperación aceptan como máximo 10 intentos por minuto (429). La recuperación responde igual exista o no el correo
Sesión	Se cierra tras 15 minutos sin actividad en el navegador. Los tokens se revocan al cerrar sesión, restablecer la contraseña, bloquear o eliminar la cuenta
Privacidad	El perfil público no expone correo, teléfono ni dirección. Los documentos y adjuntos solo se entregan con URL firmada que vence

3. Validación funcional y oportunidades de mejora

3.1 Matriz de pruebas funcionales

Las pruebas funcionales son de **caja negra**: se ejecutaron sobre la aplicación en marcha, sin mirar el código, contra el mismo contrato que usa la interfaz. Los casos CP-41 y CP-42 se hicieron directamente en la pantalla, con Chrome.

Cada caso registra el resultado esperado y el obtenido. Las cuentas son las de demostración, y para no afectar a esas cuentas, las pruebas de bloqueo y eliminación usan usuarios creados durante la prueba.

ID	HU / RNF	Caso	Tipo	Resultado esperado	Resultado obtenido	Estado
CP-01	HU001	Iniciar sesión con credenciales válidas	Positivo	200 con token	200, token emitido, rol cliente	✅
CP-02	HU001	Iniciar sesión con contraseña incorrecta	Negativo	401 "Credenciales incorrectas."	401 "Credenciales incorrectas."	✅
CP-03	HU006, HU048	Registro de cliente con datos vacíos	Negativo	422 en nombre, correo y contraseña	422 en `name`, `email` y `password`	✅
CP-04	HU020	Recuperar contraseña con un correo no registrado	Negativo	Misma respuesta que con un correo registrado	200 y 200 con el mismo mensaje	✅
CP-05	HU019	Cambiar la contraseña con la actual incorrecta	Negativo	422 en la contraseña actual	422 en `current_password`	✅
CP-06	HU022	Actualizar la ubicación con una comuna inexistente	Negativo	422 en la comuna	422 en `commune_id`	✅
CP-07	HU021	Cerrar sesión invalida el token	Positivo	Cierre 200 y luego 401 con el mismo token	200 y luego 401	✅

ID	HU / RNF	Caso	Tipo	Resultado esperado	Resultado obtenido	Estado
CP-08	RNF-SEG-01	Consultar reservas sin token	Negativo	401	401	✅
CP-09	RNF-SEG-02	Un cliente entra al panel de administración	Negativo	403	403	✅
CP-10	HU033, HU010	Filtrar por categoría	Positivo	Todos los resultados tienen la especialidad	2 resultados de Carpintería, todos con ella	✅
CP-11	HU046	Ordenar por precio ascendente	Positivo	Tarifas de menor a mayor	13 tarifas en orden: 40.000, 50.000, 55.000…	✅
CP-12	HU039	Buscar por nombre	Positivo	Aparece el profesional	"Juan": aparece Juan Castaño	✅
CP-13	HU002	Comuna sin profesionales	Positivo	Sugiere de comunas aledañas	Popular: 1 sugerido de una comuna vecina	✅
CP-14	HU040	Buscar por ubicación	Positivo	Distancia en km, del más cercano al más lejano	9 resultados: 1,2 · 1,5 · 2,6 · 2,9 · 3,8 km…	✅
CP-15	HU030	Filtro "disponible ahora"	Positivo	Solo quienes están en su horario en ese momento	0 resultados a las 10:47 p. m. (nadie atiende de noche)	✅
CP-16	HU029	Comentarios públicos de un profesional	Positivo	Lista de reseñas	1 reseña	✅
CP-17	HU025	Horas libres de un servicio	Positivo	Lista de horas del día	Primera hora libre: 24 oct, 8:00 a. m.	✅
CP-18	HU004	Cliente verificado solicita una reserva	Positivo	201, estado pendiente	201, reserva en `pending`	✅
CP-19	RN-02	Cliente sin verificar intenta reservar	Negativo	403	403 "Debes verificar tu identidad…"	✅
CP-20	HU025	La hora reservada deja de estar libre	Positivo	El horario ya no aparece	8:00 a. m. ya no aparece	✅
CP-21	HU018	El profesional recibe la notificación	Positivo	Notificación de nueva solicitud	"Nueva solicitud de reserva"	✅
CP-22	HU004	El profesional acepta	Positivo	200, aceptada	200, `accepted`	✅
CP-23	HU003	Bloquear un horario con una reserva activa	Negativo	409	409 "Tienes reservas activas en ese horario…"	✅
CP-24	HU036	El cliente confirma	Positivo	200, confirmada	200, `confirmed`	✅
CP-25	HU027	Iniciar antes de la hora pactada	Negativo	409	409 "Solo puedes iniciar el servicio desde la hora programada…"	✅
CP-26	HU011	Enviar un mensaje en el chat	Positivo	201	201	✅
CP-27	HU011	Un tercero intenta leer el chat	Negativo	403	403	✅
CP-28	HU026	Historial de estados	Positivo	pendiente → aceptada → confirmada	pending → accepted → confirmed	✅
CP-29	HU012	Cancelar con motivo	Positivo	200, cancelada	200, `cancelled`	✅
CP-30	HU017	Resumen de reservas por estado	Positivo	Totales por estado	Total 6: 2 pendientes, 2 confirmadas, 1 completada, 1 cancelada	✅
CP-31	HU037	Ingresos del profesional	Positivo	Total y servicios del rango	Del 1 al 23 de octubre: $0 en 0 servicios	✅
CP-32	HU005	Calificar una reserva no completada	Negativo	409	409 "Solo se pueden calificar reservas completadas."	✅
CP-33	HU024	Reportar a la otra parte de una reserva	Positivo	201	201	✅
CP-34	HU024	Reportar con una reserva ajena	Negativo	422	422 en `booking_id`	✅
CP-35	HU045	El reporte genera una alerta	Positivo	Alerta pendiente	Alerta sobre Juan Castaño	✅
CP-36	HU044	El profesional ve a su cliente	Positivo	200 con calificación y servicios	200: Camila Restrepo, 1 servicio completado	✅

ID	HU / RNF	Caso	Tipo	Resultado esperado	Resultado obtenido	Estado
CP-37	HU008	Enviar documentos de verificación	Positivo	201, en revisión	201, `pending`	✅
CP-38	HU014	La solicitud llega al administrador	Positivo	Aparece en la bandeja	Solicitud con 3 documentos	✅
CP-39	HU038	Bloquear a un usuario	Positivo	Bloqueo 200 e inicio de sesión 403	200 y 403 "Tu cuenta está bloqueada…"	✅
CP-40	HU023	Eliminar la cuenta	Positivo	200 y el correo anterior ya no entra	200 y luego 401	✅
CP-41	HU001	Contraseña incorrecta desde la pantalla	Negativo	Se muestra el error y no entra	"Credenciales incorrectas." y sigue en el login	✅
CP-42	HU007	Registro de profesional en dos pasos desde la pantalla	Positivo	Crea la cuenta y lleva a su panel	Cuenta creada y redirigida a `/professional/dashboard`	✅
CP-43	RN-16	Profesional sin reservas con el cliente ve su perfil	Negativo	403	403	✅
CP-44	RNF-SEG-04	11 intentos de inicio de sesión en un minuto	Negativo	Diez 401 y el undécimo 429	401 ×10, 429	✅

3.2 Resultados de las pruebas realizadas

Verificación	Alcance	Resultado
Pruebas funcionales (sección 3.1)	44 casos: 28 positivos y 16 negativos, de 35 historias, 2 reglas de negocio y 3 requisitos de seguridad	✅ 44 de 44 aprobados (23 de octubre de 2026)
Pruebas automatizadas del backend (Pest)	220 pruebas de API sobre SQLite en memoria	✅ 220 aprobadas, 992 aserciones
Pruebas automatizadas del frontend (Vitest)	158 pruebas en 38 archivos	✅ 158 aprobadas
Diseño adaptable	29 pantallas de los cuatro roles a 390, 1024 y 1366 px (87 combinaciones)	✅ Ninguna se desborda, después de corregir 4 problemas (3.3)
Análisis estático	Tipos, ESLint, oxlint, Pint, Prettier y verificador de colores	✅ Sin errores
Auditoría de dependencias	`composer audit` y `npm audit`	⚠️ Backend sin hallazgos. Frontend: aviso alto en `zod` (3.3, E-14)
Integración continua	Cada *pull request*	✅ Obligatoria: no se puede fusionar con un check en rojo

Observaciones de la ejecución:

- **El límite de intentos funciona.** En el primer intento de la batería, el límite de 10 intentos por minuto en autenticación (RNF-SEG-04) bloqueó 5 casos con 429, porque el script hacía demasiados registros e inicios de sesión seguidos. Se agregaron pausas de un minuto entre bloques, y se convirtió en un caso propio (CP-44).
- **Resultado según la hora.** CP-15 dio 0 resultados porque se ejecutó de noche, fuera del horario de todos los profesionales. Eso es lo esperado: la prueba automatizada `UserReportsAndModerationTest` cubre el caso positivo con una hora fija.
- **Ingresos en cero.** CP-31 dio $0 porque en los datos de demostración no hay servicios completados en octubre.

3.3 Errores y funcionalidades incompletas

Severidad:

- **Alta:** afecta la confianza, la seguridad o un flujo principal.
- **Media:** afecta una funcionalidad o la experiencia.
- **Baja:** detalle.

Los errores encontrados en la revisión final del 21 al 23 de octubre de 2026 que ya se corrigieron se dejan para constancia, con el *pull request* que los cerró.

Corregidos durante la revisión:

ID	Hallazgo	Severidad	Corrección
E-01	El registro de profesional fallaba con 422: el formulario de dos pasos enviaba solo los datos del segundo paso	Alta	Frontend #26
E-02	"Olvidé mi contraseña" revelaba qué correos tienen cuenta (respondía distinto si no existía)	Alta	Backend #76
E-03	Eliminar la cuenta no borraba los datos personales ni los archivos (incumplía D12 y la Ley 1581)	Alta	Backend #77
E-04	Al eliminar una cuenta, la otra parte veía la reserva con el cliente o profesional vacío	Alta	Backend #77
E-05	Cualquier usuario autenticado podía consultar el perfil de cualquier cliente	Media	Backend #78
E-06	Un reporte se podía asociar a una reserva ajena	Media	Backend #78
E-07	El enlace del correo de recuperación usaba `env()` fuera de la configuración: con la configuración en caché apuntaba a `localhost`	Media	Backend #76
E-08	La pantalla de nueva reserva era más ancha que la pantalla en todos los tamaños (hasta 1066 px en un celular de 390)	Media	Frontend #29
E-09	En celular, la barra superior mostraba cortados "Iniciar sesión" y "Registrarse" y escondía el botón de menú. Con sesión, desbordaba en tablet y dejaba el buscador en 62 px	Media	Frontend #29
E-10	Una prueba automatizada fallaba todas las noches entre las 10 p. m. y las 2 a. m. (dependía de la hora real)	Baja	Backend #76

Abiertos:

ID	Hallazgo	Severidad	Detalle
E-11	**Los paneles de inicio y la página principal muestran datos de maqueta**	Alta	Ver el detalle debajo de la tabla
E-12	HU044 incompleta: el perfil del cliente no muestra los comentarios que recibió	Media	`ClientProfileResource` no incluye las reseñas
E-13	Falta la política de tratamiento de datos y su aceptación al registrarse (Ley 1581)	Media	El login dice "Al continuar aceptas… la Política de Privacidad" (figura 6), pero esa política no existe. Acción pendiente en 07
E-14	Aviso de seguridad alto en `zod` < 4.6.6: consumo de memoria al validar colecciones enormes	Baja	Sin arreglo en zod 3, y `@vee-validate/zod` aún no admite zod 4. En este proyecto zod solo valida formularios en el navegador del propio usuario, así que el riesgo real es bajo
E-15	HU040 parcial: la distancia no aparece en el perfil del profesional	Baja	`GET /professionals/{id}` no recibe coordenadas
E-16	HU045 parcial: la alerta no enlaza al perfil del usuario	Baja	El administrador debe buscarlo en "Usuarios"
E-17	HU030 parcial: "disponible ahora" no se refresca solo mientras la lista está abierta	Baja	Se calcula al buscar
E-18	HU013 parcial: no hay comparación automática selfie-documento	Baja	Decisión D10: no existe un proveedor público. El administrador compara a mano
E-19	La página de verificación usa el mismo texto para clientes y profesionales: al cliente le habla de "sello de profesional verificado" y de "aceptar servicios"	Baja	Figura 16
E-20	Los selectores de filtro de las bandejas del administrador empiezan vacíos, sin una opción "Todos" visible	Baja	Figuras 26 y 27
E-21	Los datos de demostración (`DemoSeeder`) generan descripciones en latín de relleno y nombres y correos en inglés (figura 25)	Baja	Para las capturas se reemplazaron los textos del profesional principal por unos realistas
E-22	Faltan límites del lado del servidor: los tokens no vencen (`sanctum.expiration`), el chat no limita la frecuencia de mensajes y CORS acepta cualquier origen	Media	Ver 07

ID	Hallazgo	Severidad	Detalle
E-23	Sin prueba automatizada del cierre por inactividad (HU050) ni de las vistas del administrador; sin pruebas de extremo a extremo en la CI ni medición de cobertura	Media	Se verificaron a mano y con la batería funcional de este informe

Detalle de E-11. El panel del profesional muestra cifras fijas en el código:

- "8 citas hoy", "$4.850.000 de ingresos" y "4.9/5 con 127 opiniones";
- una agenda del día con clientes inventados en barrios de Bogotá;
- "Zona norte" y "74 % de la meta".

El panel del cliente tiene un texto que dice "páginas pendientes de desarrollo" y profesionales recomendados fijos. La página de inicio muestra "4.9/5, más de 2.000 reseñas" y profesionales de ejemplo. Las solicitudes pendientes y los servicios en curso de los paneles sí son reales. Al ser cifras inventadas, quien las vea puede creer que son ciertas, y eso le resta confianza a la plataforma.

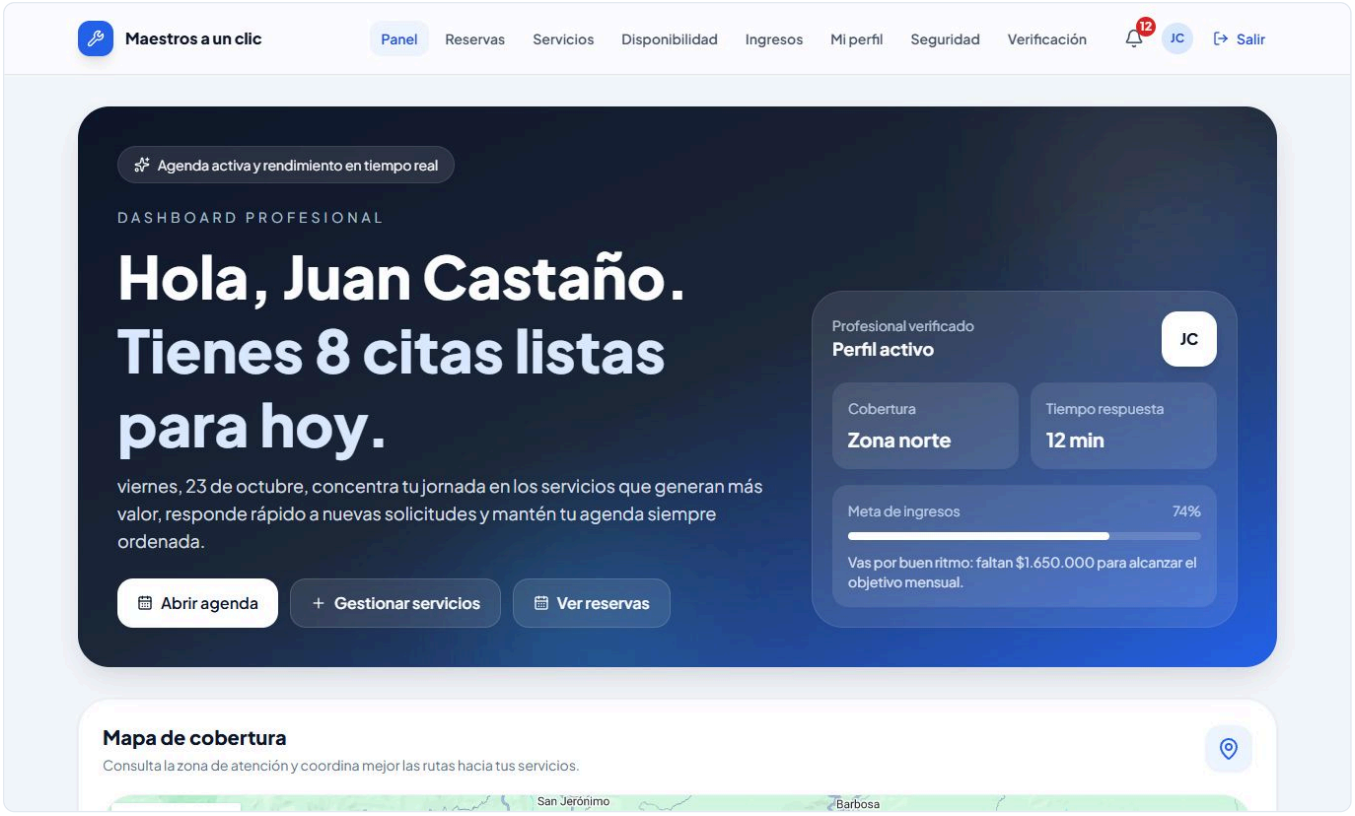


Figura 28. Panel del profesional: "8 citas listas para hoy", "Zona norte", "12 min" y "74 %" son valores fijos en el código (E-11).



Figura 29. Panel del cliente: el texto menciona "páginas pendientes de desarrollo" (E-11).

3.4 Análisis de fortalezas y debilidades

Fortalezas:

- **Cobertura funcional completa.** Las 49 historias están implementadas: 44 completas y 5 con un criterio menor por cerrar. La batería funcional pasó entera.
- **Reglas de negocio sólidas y centralizadas.** La agenda calcula la disponibilidad real, impide cruces con bloqueo de fila en la transacción y lleva un historial auditable. La máquina de estados no admite transiciones inválidas.
- **Seguridad y privacidad desde el diseño.** Hay permisos por rol y por recurso, documentos privados con enlaces que vencen y un límite de intentos. La recuperación no revela cuentas y la eliminación anonimiza. Varias de estas protecciones se comprobaron con casos negativos.
- **Tiempo real bien integrado.** El chat, el estado en línea, las notificaciones y la agenda se actualizan solos.
- **Calidad verificable.** El proyecto tiene:
  - 378 pruebas automatizadas;
  - CI obligatoria en los dos repositorios;
  - contrato de API documentado y OpenAPI;
  - trazabilidad de cada historia a su código y sus pruebas;
  - decisiones de arquitectura registradas.
- **Interfaz cuidada y adaptable.** Usa un sistema de diseño propio con validación en el navegador. Después de las correcciones, las 29 pantallas funcionan en celular, tablet y escritorio.

Debilidades:

- **Datos de maqueta en los paneles y en la página de inicio (E-11).** Es lo más visible para un usuario nuevo.
- **Pruebas en el navegador manuales.** Las pruebas de la interfaz completa no están en la CI, y la cobertura de código no se mide. Los dos errores de pantalla de la revisión (E-01 y E-08) solo se detectaron usando la aplicación.
- **Pendientes de producción:** política de datos, CORS, vencimiento de tokens y límite del chat (E-13, E-22).
- **Desarrollo desigual entre módulos.** Algunas pantallas tienen textos que no se adaptan al rol (E-19) o criterios menores sin cerrar (E-12, E-15, E-16).
- **Sin pagos en línea ni consulta automática de antecedentes.** Ambos están fuera del alcance por decisión, pero limitan el modelo de negocio.

3.5 Propuestas de mejora

#	Propuesta	Resuelve	Esfuerzo	Prioridad
1	Conectar los paneles y la página de inicio a datos reales: citas del día, ingresos del mes, solicitudes pendientes, calificación, profesionales cercanos y categorías del catálogo. Lo que no tenga dato real se quita	E-11	Medio	Alta
2	Publicar la política de tratamiento de datos y pedir su aceptación al registrarse (campo `privacy_accepted_at` en una migración nueva)	E-13	Bajo	Alta
3	Antes de desplegar: limitar CORS al dominio del frontend, hacer vencer los tokens y limitar la frecuencia de mensajes del chat	E-22	Bajo	Alta
4	Completar los criterios menores: comentarios en el perfil del cliente, distancia en el perfil del profesional, enlace de la alerta al usuario y textos de verificación según el rol	E-12, E-15, E-16, E-19	Bajo	Media
5	Llevar la batería funcional de este informe a pruebas de extremo a extremo en la CI (por ejemplo, Playwright) y medir la cobertura	E-23	Medio	Media
6	Migrar a zod 4 cuando `@vee-validate/zod` lo admita, y mantener la auditoría de dependencias en cada PR	E-14	Bajo	Media
7	Datos de demostración realistas en el `DemoSeeder` y opción "Todos" en los filtros del administrador	E-20, E-21	Bajo	Baja
8	Hacia una versión 2: pagos en línea con una pasarela local, integración con un proveedor de verificación de identidad y aplicación móvil	Alcance	Alto	Futuro

3.6 Conclusiones sobre el estado actual del software

Maestros a un clic cumple su objetivo. Un cliente de Medellín puede encontrar un profesional verificado, reservar una hora que de verdad está libre, coordinar por chat, recibir el servicio y calificarlo, con la misma confianza del lado del profesional. Las 49 historias de usuario están implementadas, y los 44 casos de la batería funcional se aprobaron sobre la aplicación en marcha. Las pruebas automatizadas (220 de backend y 158 de frontend) también pasan.

La revisión final sirvió. Encontró y corrigió 10 errores, 4 de severidad alta:

- un registro de profesional que no funcionaba;
- una fuga de información en la recuperación de contraseña;
- una eliminación de cuenta que no protegía los datos personales;
- reservas que quedaban sin contraparte al eliminar una cuenta.

Ninguno se veía en las pruebas automatizadas. Los encontró la validación funcional sobre la aplicación real, y por eso se propone llevar esa batería a la integración continua.

Está listo para presentar, con trabajo pendiente antes de producción. El software está en un estado estable y presentable para la entrega académica. Para ponerlo en producción con usuarios reales falta, sobre todo:

- quitar los datos de maqueta de los paneles (E-11);
- completar la parte legal de protección de datos (E-13);
- cerrar los límites del servidor (E-22).

Las tres mejoras son de esfuerzo bajo o medio y están priorizadas en la sección 3.5.

Guía de despliegue en Azure

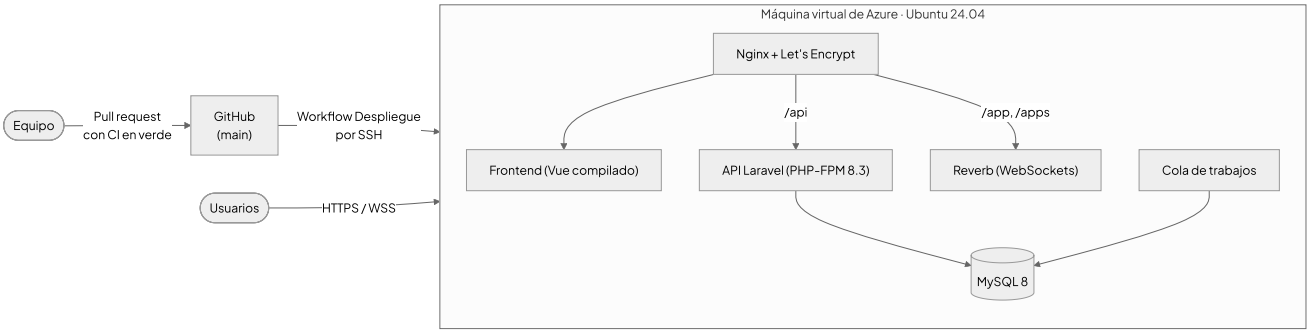
Paso a paso exacto de cómo se puso Maestros a un clic en producción, en el orden en que se hizo, para poder repetirlo:

1. crear la máquina virtual en Azure;
2. prepararla con un solo script;
3. conectarla con GitHub;
4. dejarla desplegando sola cada vez que se fusiona a `main`.

El resultado está en **https://maestrosaunclic.northcentralus.cloudapp.azure.com**.

La referencia de operación (variables, respaldos, monitoreo) está en 06-despliegue-y-operacion.md. Esta guía se centra en **cómo se llega ahí**.

1. Qué se construye



- **Una sola máquina** con todo adentro: suficiente para la demostración y gratuita con *Azure for Students*.
- **Un solo dominio:** el que Azure da gratis, con HTTPS. El frontend, el API y los WebSockets comparten origen, así que no hay CORS que configurar.
- **Despliegue continuo:** cuando el CI de `main` pasa, GitHub Actions publica la nueva versión. Nadie tiene que entrar al servidor.

Dato	Valor
Suscripción	Azure for Students (100 USD de crédito, sin tarjeta)
Grupo de recursos	`maestros-vm_grupo`
Máquina virtual	`maestros-vm`, tamaño **Standard_B2ats_v2** (2 vCPU, 1 GiB de RAM)
Región	North Central US
Imagen	Ubuntu Server 24.04 LTS (x64 Gen2)
Disco	SSD Premium de 64 GiB
IP pública	`20.80.51.83` (estática)
Nombre DNS	`maestrosaunclic.northcentralus.cloudapp.azure.com`
Usuario SSH	`maestros`
Software instalado	Nginx 1.24, PHP 8.3, MySQL 8.0, Supervisor 4.2, Certbot 2.9, Composer 2

2. Requisitos previos

- Una cuenta de **Azure for Students** activa (https://azure.microsoft.com/free/students), creada con el correo institucional.
- Permisos de administrador en los dos repositorios de GitHub, para crear *secrets* y variables.
- En el computador: **Git Bash** (trae `ssh`, `ssh-keygen` y `scp`), **Node.js 24** y, opcionalmente, la CLI de GitHub (`gh`).

3. Paso 1: crear las llaves SSH

Se crean **dos** pares de llaves en el computador, en Git Bash:

```
ssh-keygen -t ed25519 -f ~/.ssh/maestros_azure  -N "" -C "juan-pc@maestros-azure"
ssh-keygen -t ed25519 -f ~/.ssh/maestros_deploy -N "" -C "github-actions@maestros-deploy"
cat ~/.ssh/maestros_azure.pub      # esta línea se pega en el portal de Azure
```

Llave	Para qué	Dónde queda la parte privada
`maestros_azure`	Entrar al servidor como administrador	Solo en el computador del equipo
`maestros_deploy`	Que GitHub Actions despliegue	En los *secrets* de GitHub. En el servidor **solo puede desplegar** (sección 11)

Se separan para poder revocar la de GitHub sin perder el acceso de administrador. Las llaves privadas nunca se suben a un repositorio ni se envían por chat.

4. Paso 2: crear la máquina virtual en el portal

En https://portal.azure.com: **Máquinas virtuales → Crear → Máquina virtual de Azure**.

Pestaña Aspectos básicos:

Campo	Valor
Suscripción	Azure for Students
Grupo de recursos	*Crear nuevo*: `maestros-vm_grupo` (es el que propone el portal)
Nombre de la máquina virtual	`maestros-vm`
Región	**(US) North Central US**. Azure for Students no permite todas las regiones; si una da un error de política (*"not allowed"*), se elige otra
Opciones de disponibilidad	No se requiere redundancia de infraestructura
Tipo de seguridad	Estándar
Imagen	**Ubuntu Server 24.04 LTS – x64 Gen2**
Tamaño	**Standard_B2ats_v2** (2 vCPU, 1 GiB). Debe decir *"Free services eligible"*. También sirve Standard_B1s
Tipo de autenticación	Clave pública SSH
Nombre de usuario	`maestros`
Origen de clave pública SSH	*Usar clave pública existente*: se pega la línea completa de `maestros_azure.pub`
Puertos de entrada públicos	Permitir los seleccionados: **HTTP (80), HTTPS (443) y SSH (22)**

Pestaña Discos: disco del sistema operativo **SSD Premium de 64 GiB (P6)**, que entra en los servicios gratuitos de 12 meses.

Pestaña Redes: todo por defecto. Se marca *"Eliminar la IP pública y la NIC cuando se elimine la VM"*.

Revisar y crear → Crear. Tarda unos dos minutos.

5. Paso 3: ponerle un nombre DNS

El certificado HTTPS necesita un nombre, no solo una IP. Azure da uno gratis: **no hay que crear una zona DNS**, que sirve para dominios propios y además tiene costo.

1. Abrir la máquina virtual, en **Información general**.
2. En **Nombre DNS**, que dice *"No configurado"*, hacer clic en el enlace. Otra forma: clic en la IP pública y luego, en el menú, **Configuración**.
3. En **Etiqueta de nombre DNS**, escribir `maestrosaunclic`. El nombre completo queda así: `maestrosaunclic.northcentralus.cloudapp.azure.com`.
4. Revisar que la **asignación** de la IP sea **Estática**, para que no cambie al reiniciar.
5. **Guardar.**

6. Paso 4: comprobar el acceso

```
nslookup maestrosaunclic.northcentralus.cloudapp.azure.com     # debe responder 20.80.51.83
ssh -i ~/.ssh/maestros_azure maestros@maestrosaunclic.northcentralus.cloudapp.azure.com
```

La primera vez, SSH pregunta si se confía en el servidor: se responde `yes`. Si entra y muestra la consola de Ubuntu, el paso está listo.

7. Paso 5: preparar el servidor con `provision.sh`

Todo lo que el servidor necesita lo hace un solo script del repositorio: `deploy/provision.sh`. Desde el computador:

```
H=maestrosaunclic.northcentralus.cloudapp.azure.com
ssh -i ~/.ssh/maestros_azure maestros@$H
```

Ya dentro del servidor:

```
sudo git clone https://github.com/LordCasta/maestros_a_un_clic_backend.git /var/www/maestros-backend

sudo DOMAIN=maestrosaunclic.northcentralus.cloudapp.azure.com \
     EMAIL=<correo para avisos del certificado> \
     REVERB_APP_KEY=$(openssl rand -hex 16) \
     REVERB_APP_SECRET=$(openssl rand -hex 32) \
     DEPLOY_PUBKEY="<contenido de ~/.ssh/maestros_deploy.pub>" \
     SEED_DEMO=1 \
     bash /var/www/maestros-backend/deploy/provision.sh
```

> Hay que **anotar el** `REVERB_APP_KEY` que se use, porque GitHub lo necesita para compilar el frontend (paso 6). También queda guardado en el `.env` del servidor.

Tarda unos 10 minutos. El script hace, en orden:

#	Paso	Detalle
1	Memoria de intercambio	2 GB de *swap*, porque la máquina tiene 1 GiB de RAM
2	Paquetes	Nginx, MySQL, PHP 8.3 con sus extensiones, Supervisor, Certbot, Git, rsync y Composer
3	MySQL	Ajustado a poca memoria. Crea la base `maestros` y su usuario, con una contraseña aleatoria
4	PHP-FPM	Pocos procesos, y límites de carga para los documentos de verificación
5	Código	El backend en `/var/www/maestros-backend` y una página provisional en `/var/www/maestros-frontend`
6	`.env` de producción	`APP_DEBUG=false`, la URL con HTTPS, claves propias de la app y de Reverb, logs diarios
7	Base de datos	Migraciones, las 21 comunas y corregimientos, y las categorías
8	Datos demo (`SEED_DEMO=1`)	Las cuentas de demostración con una **contraseña aleatoria**: en una URL pública, `password` sería una puerta abierta
9	Nginx	Configuración de `deploy/nginx.conf`: frontend, API y Reverb en el mismo dominio
10	HTTPS	Certificado de Let's Encrypt, con redirección de HTTP a HTTPS y renovación automática
11	Procesos permanentes	Supervisor mantiene vivos la cola y Reverb. cron ejecuta las tareas programadas
12	Llave de despliegue	Se autoriza la llave de GitHub **solo** para desplegar (sección 11)

Al final imprime la dirección de la aplicación y, si se cargó la demo, la contraseña de las cuentas demo. Esa contraseña también queda en el servidor, en `~/demo-password.txt`, para consultarla después.

8. Paso 6: configurar GitHub

En **cada** repositorio, backend y frontend: **Settings → Secrets and variables → Actions**.

Nombre	Pestaña	Valor	Repositorio
`DEPLOY_SSH_KEY`	Secrets	Todo el contenido de `~/.ssh/maestros_deploy` (la llave **privada**)	Los dos
`DEPLOY_KNOWN_HOSTS`	Secrets	La salida de `ssh-keyscan maestrosaunclic.northcentralus.cloudapp.azure.com`	Los dos
`DEPLOY_HOST`	Variables	`maestrosaunclic.northcentralus.cloudapp.azure.com`	Los dos
`DEPLOY_USER`	Variables	`maestros`	Los dos
`VITE_REVERB_APP_KEY`	Variables	El `REVERB_APP_KEY` del paso 5	Solo frontend

Con la CLI de GitHub es lo mismo en una sola tanda, desde Git Bash:

```
H=maestrosaunclic.northcentralus.cloudapp.azure.com
ssh-keyscan $H > known_hosts_prod
for R in LordCasta/maestros_a_un_clic_backend LordCasta/maestros_a_un_clic_frontend; do
  gh secret set DEPLOY_SSH_KEY     -R $R < ~/.ssh/maestros_deploy
  gh secret set DEPLOY_KNOWN_HOSTS -R $R < known_hosts_prod
  gh variable set DEPLOY_HOST      -R $R --body "$H"
  gh variable set DEPLOY_USER      -R $R --body maestros
done
gh variable set VITE_REVERB_APP_KEY -R LordCasta/maestros_a_un_clic_frontend --body "<REVERB_APP_KEY>"
rm known_hosts_prod
```

`DEPLOY_KNOWN_HOSTS` **es la huella del servidor.** Con ella, GitHub comprueba que se conecta a la máquina correcta y no a una impostora. Si la máquina se vuelve a crear, cambia su huella y hay que actualizar este *secret*.

9. Paso 7: el primer despliegue

El servidor quedó con el backend, pero con una página provisional en lugar del frontend. El primer despliegue se puede hacer de dos formas:

- **Automático:** con los workflows ya en `main` (sección 10), en *Actions → Despliegue → Run workflow*, en cada repositorio.
- **A mano**, desde el computador, con la misma llave que usa GitHub. Así se hizo la primera vez:

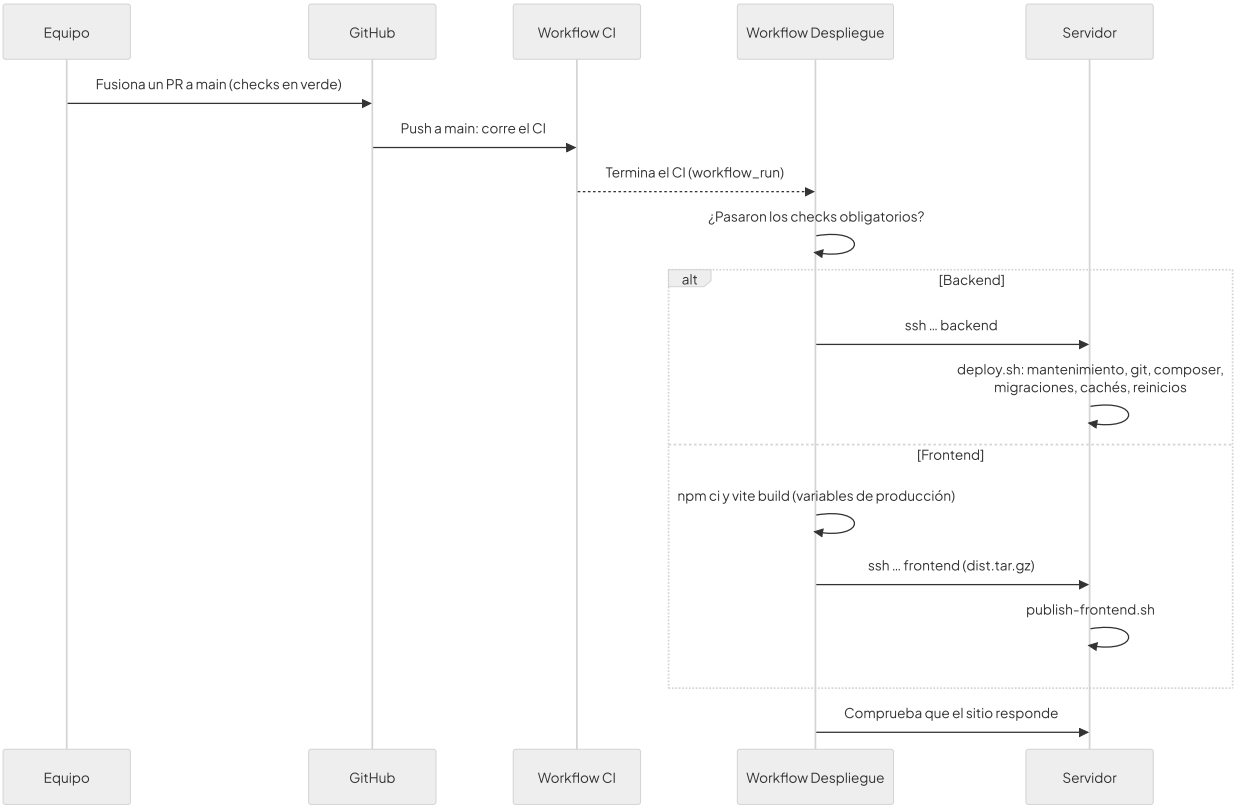
```
H=maestrosaunclic.northcentralus.cloudapp.azure.com
cd frontend-maestros-a-un-clic
VITE_API_BASE_URL=https://$H/api/v1 VITE_REVERB_APP_KEY=<REVERB_APP_KEY> \
VITE_REVERB_HOST=$H VITE_REVERB_PORT=443 VITE_REVERB_SCHEME=https \
  npx vite build
tar -czf - -C dist . | ssh -i ~/.ssh/maestros_deploy maestros@$H frontend   # publica el frontend
ssh -i ~/.ssh/maestros_deploy maestros@$H backend                            # despliega el backend
```

Verificación (la que se hizo el 24 de octubre de 2026):

Prueba	Resultado
`https://<host>/up`, `/api/v1/categories` y `/`	200, con certificado válido
Login con una cuenta demo y la contraseña `password`	401: rechazada
Login con la contraseña aleatoria, por la pantalla, en Chrome	Entra al panel del cliente
WebSockets	Se conectan por `wss://` y los canales privados se autorizan
El cliente envía un mensaje del chat	Le llega al profesional en vivo, sin recargar
La llave de despliegue intenta otro comando (`whoami`)	Rechazado: *"Comando no permitido"*

10. Cómo despliega solo desde GitHub

Cada repositorio tiene el workflow **Despliegue** (`.github/workflows/deploy.yml`).



Cuándo despliega:

- **Solo después de un push a** `main`, es decir, cuando se fusiona un PR. Nunca con un PR abierto.
- **Solo si pasaron los checks obligatorios del CI.**
  - Backend: `Tests (PHP 8.2)` y `Tests (PHP 8.5)`.
  - Frontend: `Lint, tipos, tests y build`.
  - La auditoría de dependencias no cuenta: puede fallar por un aviso nuevo sin bloquear, igual que en los PRs.
- **Uno a la vez.** Si llegan dos fusiones seguidas, la segunda espera a que termine la primera.
- **Solo si existe** `DEPLOY_HOST`**.** Sin servidor configurado, el workflow se salta sin fallar.

Qué hace en el servidor el despliegue del backend (`deploy/deploy.sh`):

1. Pone la aplicación en mantenimiento (`artisan down`).
2. Trae exactamente lo que hay en `main` (`git fetch` y `git reset --hard origin/main`).
3. Instala las dependencias de producción (`composer install --no-dev`).
4. Aplica las migraciones nuevas. Nunca se editan las existentes, así que no se pierden datos.
5. Regenera las cachés de configuración, rutas y eventos.
6. Reinicia la cola y Reverb, y recarga PHP-FPM, para que todo use el código nuevo.
7. Levanta la aplicación (`artisan up`), **aunque algún paso haya fallado**.

Qué hace el despliegue del frontend:

1. GitHub compila `dist/` con las variables de producción.
2. GitHub lo envía comprimido.
3. `deploy/publish-frontend.sh` copia primero los archivos nuevos y al final `index.html`, así la aplicación nunca queda en blanco. Si el paquete no trae `index.html`, no publica nada.

Dónde verlo: en GitHub, pestaña **Actions** → **Despliegue**. Cada ejecución muestra qué checks revisó, la salida del servidor (por ejemplo, *"Desplegado 254e97f: …"*) y la comprobación final. La ejecución queda en verde si el sitio responde.

Desplegar a mano (por ejemplo, después de cambiar una variable): *Actions → Despliegue → Run workflow*.

Volver a una versión anterior: se revierte el PR en GitHub (*Revert*), y esa reversión, al fusionarse, se despliega como cualquier otra.

> **Estado al 24 de octubre de 2026.** Los *secrets*, las variables y el servidor ya están listos. Los dos workflows esperan su revisión en el backend #85 y el frontend #31. Tocan `.github/`, así que necesitan la etiqueta `cambio-de-reglas`, que pone una persona. Desde que se fusionen, cada fusión a `main` llega sola a producción.

11. Seguridad del montaje

- **La llave de GitHub no abre una terminal.** En el servidor, su línea de `~/.ssh/authorized_keys` tiene un comando forzado (`deploy/ssh-gate.sh`): solo acepta `backend` o `frontend`. Si se filtrara, solo serviría para volver a desplegar `main`. Cualquier otra cosa responde *"Comando no permitido"*.
- **Mínimos privilegios en el servidor.** `deploy.sh` corre con el usuario `maestros`, y solo puede ejecutar como administrador tres reinicios concretos (`/etc/sudoers.d/maestros-deploy`). Los comandos de Laravel corren como `www-data`, el mismo usuario de PHP.
- **Secretos fuera del código.** Las contraseñas de MySQL y de la demo, las claves de la app y de Reverb, y la llave privada viven en el `.env` del servidor o en los *secrets* de GitHub. Nunca están en el repositorio.
- **Solo tres puertos abiertos:** 22 con llave (sin contraseña), 80 (redirige) y 443.
- **Producción sin modo de depuración.** `APP_DEBUG=false`: los errores no muestran código ni datos.

12. Costos y cuidado del crédito

- **Lo gratuito de Azure for Students:** el portal marcó la máquina y el disco como *"Free services eligible"*, es decir, dentro de los servicios gratuitos de 12 meses (del orden de 750 horas al mes de máquina pequeña). Lo que exceda esa cuota se descuenta de los 100 USD de crédito. La lista exacta está en el portal, en *Servicios gratuitos*.
- **Sin cobros sorpresa:** la suscripción no tiene tarjeta. Si el crédito se acaba, los servicios se detienen.
- **Detener la máquina** desde el portal cuando no se use. Se conservan la IP, el nombre DNS y los datos, y al encenderla todo arranca solo: Nginx, MySQL, PHP, Supervisor y la renovación del certificado.
- **Consultar el gasto:** en el portal, *Suscripciones → Azure for Students → Análisis de costos*, o en el centro *Education*, que muestra el crédito restante.

13. Solución de problemas

Síntoma	Causa probable	Qué hacer
Al crear la VM: error de política o *"not allowed"*	Azure for Students limita las regiones	Elegir otra región, como se hizo con North Central US
El portal pide crear una "zona DNS"	Es otra cosa: un dominio propio	Usar la **etiqueta de nombre DNS** de la IP pública (paso 3)
`ssh: Permission denied (publickey)`	Se usó otra llave u otro usuario	`ssh -i ~/.ssh/maestros_azure maestros@<host>`
El despliegue falla con *Host key verification failed*	La VM se volvió a crear y cambió su huella	Actualizar `DEPLOY_KNOWN_HOSTS` con `ssh-keyscan` (paso 6)
El despliegue dice *"Comando no permitido"*	Algo intentó usar la llave de GitHub para otra cosa	Es lo esperado: solo acepta `backend` o `frontend`
El workflow aparece como *skipped*	Falta `DEPLOY_HOST`, el CI no pasó o fue un PR	Revisar las variables y el CI de `main`
El sitio muestra 502	PHP-FPM caído	`sudo systemctl restart php8.3-fpm`
No llegan los mensajes en vivo	Reverb detenido	`sudo supervisorctl status` y `sudo supervisorctl restart maestros-reverb`. Log: `/var/log/maestros/reverb.log`
Errores 500	Ver el detalle en el log	`/var/www/maestros-backend/storage/logs/laravel-AAAA-MM-DD.log`
`provision.sh` falla en el certificado	El nombre DNS aún no apunta a la IP, o el puerto 80 está cerrado	Comprobar con `nslookup` y la regla del puerto 80, y volver a correr el script. Es seguro repetirlo
La máquina se queda sin memoria	1 GiB es justo	El *swap* de 2 GB lo absorbe. Si pasa seguido, subir de tamaño desde el portal